

How to Use (Plain) Witness Encryption: Registered ABE, Flexible Broadcast, and More

Cody Freitag
Boston University
freitag@bu.edu

Brent Waters
UT Austin and NTT Research
bwaters@cs.utexas.edu

David J. Wu
UT Austin
dwu4@cs.utexas.edu

Abstract

Witness encryption is a generalization of public-key encryption where the public key can be any NP statement x and the associated decryption key is any witness w for x . While early constructions of witness encryption relied on multilinear maps and indistinguishability obfuscation (iO), recent works have provided direct constructions of witness encryption that are more efficient than iO (and also seem unlikely to yield iO). Motivated by this progress, we revisit the possibility of using witness encryption to realize advanced cryptographic primitives previously known only in “obfustopia.”

In this work, we give new constructions of *trustless* encryption systems from *plain* witness encryption (in conjunction with the learning-with-errors assumption): (1) flexible broadcast encryption (a broadcast encryption scheme where users choose their *own* secret keys and users can encrypt to an *arbitrary* set of public keys); and (2) registered attribute-based encryption (a system where users choose their own keys and then register their public key together with a set of attributes with a deterministic and transparent key curator). Both primitives were previously only known from iO . We also show how to use our techniques to obtain an *optimal* broadcast encryption scheme in the random oracle model.

Underlying our constructions is a novel technique for using witness encryption based on a new primitive which we call *function-binding hash functions*. Whereas a somewhere statistically binding hash function statistically binds a digest to a few bits of the input, a function-binding hash function statistically binds a digest to the *output* of a function of the inputs. As we demonstrate in this work, function-binding hash functions provide us new ways to leverage the power of plain witness encryption and use it as the foundation of advanced cryptographic primitives. Finally, we show how to build function-binding hash functions for the class of disjunctions of block functions from leveled homomorphic encryption; this in combination with witness encryption yields our main results.

1 Introduction

In the last decade, indistinguishability obfuscation (iO) [BGI⁺01, GGH⁺13] has emerged as a central hub for cryptography, and in combination with one-way functions or somewhere statistically binding hash functions [HW15], have yielded solutions to a broad range of cryptographic problems [SW14, BZ14, GGHR14, BPR15, CLP15, KRR17, AL18, CPP20]; we refer to [JLS21, Corollary 1.1] for a more comprehensive list. Witness encryption [GGSW13] is another cryptographic notion that is often considered alongside indistinguishability obfuscation, and has been used successfully to construct a number of cryptographic primitives such as public-key encryption, identity-based encryption, attribute-based encryption [GGSW13], oblivious transfer [BGI⁺17], laconic arguments [FNV17, BISW18], and null- iO [GKW17, WZ17]. However, compared to indistinguishability obfuscation, there has been significantly less success in leveraging witness encryption to realize other cryptographic notions, and indeed, many of the notions implied by witness encryption can be achieved directly from standard number-theoretic or lattice-based assumptions.

Witness encryption. Intuitively, witness encryption [GGSW13] is a generalization of public-key encryption where the public key can be any NP statement x and the associated decryption key is any witness w for x . Here, a user can encrypt a message m with respect to the statement x and anyone who knows a witness w for x can decrypt. The security requirement is that if the statement x is false, then the message is computationally hidden.

Although witness encryption appears to be a substantially weaker primitive than iO (both in terms of its applications and with respect to black-box separations [GMM17]), early constructions nonetheless either relied on iO [GGH⁺13] or techniques that sufficed for iO (e.g., assumptions over multilinear maps [GGSW13, GLW14]). This state of affairs changed recently with several works providing constructions from lattice-based assumptions [CVW18, Tsa22, VWV22] or suggesting new routes from group-based approaches [BIOW20]. Of particular note, the recent results [Tsa22, VWV22] construct witness encryption from the “evasive LWE” assumption, a new knowledge assumption on lattices [Wee22, Tsa22]. These works give a direct construction of witness encryption that is significantly more efficient than existing iO approaches. Moreover, the current techniques do not seem to extend to give iO .

This work: cryptography from *plain* witness encryption. Motivated by the recent progress in constructing witness encryption, our goal in this work is to revisit the possibility of using witness encryption to realize advanced cryptographic primitives. Much like how the punctured programming framework [SW14] and notions like somewhere statistically binding hash functions [HW15] allowed us to leverage the power of iO , our goal is to develop analogous techniques in the setting of (plain) witness encryption. Previous approaches to use witness encryption for realizing new cryptographic primitives have focused on augmenting witness encryption with additional properties such as “position-hiding” [GLW14, GVW19] or “extractability” [GKP⁺13, FNV17]. However, positional witness encryption is only known from iO [GLW14, GVW19] while extractable witness encryption currently relies on a knowledge assumption over multilinear maps or the stronger notion of differing-inputs obfuscation [GKP⁺13]. Moreover, the recent advancements in constructing witness encryption seem insufficient for realizing these stronger notions. As such, we focus on techniques that use *plain* witness encryption.

Function-binding hash functions. In this work, we introduce a new cryptographic primitive called a *function-binding hash function*. Similar to how somewhere statistically binding (SSB) hash functions [HW15] statistically bind to a few bits of an *input* to the hash function, our function-binding hash function statistically binds to the *output* of a *function* of the inputs. In fact, we can view SSB hash functions as a special case of function-binding hash functions for the class of index functions (see Remark 3.2). Similar to the setting of SSB hash function, a function-binding hash function should allow a user to *locally* open one or more bits of the input. The size of the opening should be small compared to the size of the input.

To illustrate the function-binding property, consider a (keyed) hash function H that is statistically binding for the OR function. Suppose that $\text{dig} \leftarrow H(\text{hk}, b_1, \dots, b_n)$ where hk is the hash key and $b_1 = \dots = b_n = 0$. If H is (statistically) function-binding for the OR function, then it should not be possible to locally open *any* index i to a value $b'_i = 1$. This is because the output of the OR function on all inputs $(b'_1, \dots, b'_n)$ is $1 \neq 0$, irrespective of the values of b'_j for $j \neq i$. More generally, a function-binding hash function only supports local openings to values that are consistent with the output of the associated function (i.e., the locally-opened value(s) can be extended into an input to the function that maps to the value associated with the digest).

We provide a formal definition of this notion in Section 3 and a high-level overview in Section 1.1. We then show how to construct a function-binding hash function using leveled homomorphic encryption, which can in turn be based on standard lattice assumptions [BV11, BGV12, Bra12, GSW13]. Our construction here follows a similar structure as the construction of somewhere statistically binding hash functions [HW15] from leveled homomorphic encryption.

Our results. Using function-binding hash functions together with plain witness encryption, we are able to realize new *trustless* encryption notions like flexible broadcast encryption and registered attribute-based encryption (for general policies) [HLWW23] without indistinguishability obfuscation. These examples are intended to illustrate the usefulness of function-binding hash functions in conjunction with witness encryption, and we expect to see further applications in the future. We summarize our main results below:

- **Flexible broadcast encryption.** In a flexible broadcast encryption scheme, users generate their own public/private key-pair (pk, sk) (much like in vanilla public-key encryption) and then post their public key pk to a public bulletin board. The encryption algorithm takes a collection of user public keys $S = (pk_1, \dots, pk_n)$ along with a message m and outputs a single ciphertext ct that can be decrypted by any user who possesses a secret key corresponding to a public key in the broadcast set S . The size of the ciphertext ct scales *polylogarithmically* with the size of the broadcast set S . Flexible broadcast encryption generalizes both the traditional notion of broadcast encryption [FN93] which assumes a central *trusted* authority issues keys to users as well as the

notion of distributed broadcast encryption [BZ14]. In distributed broadcast encryption, users generate their own public and private keys, but the syntax stipulates that each user is associated with a particular “index” (like in standard broadcast encryption); in turn, one can only encrypt to users associated with different indices. Flexible broadcast encryption eliminates all user coordination and supports encryption to an arbitrary set of user public keys. Previously, distributed broadcast encryption was only known from *iO* [BZ14]. In this work, we show how to construct flexible broadcast encryption from plain witness encryption and function-binding hash functions (which implies a distributed broadcast encryption scheme). Previously, it was not even known how to obtain *vanilla* broadcast encryption from witness encryption; instead, previous approaches relied on the stronger notion of positional witness encryption [GLW14, GVW19].

- **Registered attribute-based encryption.** A second application we show is to registered attribute-based encryption [HLWW23]. Much like how flexible broadcast encryption removes the trusted key issuer from broadcast encryption, registered ABE removes the trusted key issuer from traditional ABE [SW05, GPSW06]. In the registered ABE model, instead of a key issuer generating decryption keys for user attributes, users instead generate their own public/secret keys and then *register* their public key along with their attributes with a key curator. The key curator in this case is a deterministic and transparent algorithm that is responsible for aggregating the keys and attributes for the different users into a short master public key (whose size is polylogarithmic in the number of registered users). The aggregated public key functions as a public key for a standard ABE scheme. Since the key curator maintains no long-term secrets and can be publicly audited, registered ABE serves as a new paradigm for enabling the access-control capabilities of ABE without introducing a trusted key-issuing authority into the picture. Previously, the work of [HLWW23] showed how to construct registered ABE for an a priori bounded number of users using pairing-based assumptions in a model with a structured common reference string, as well as a scheme for an unbounded number of users in the common *random* string model from *iO*. In this work, we show how to obtain a registered ABE scheme with the same properties as the previous construction from *iO* using plain witness encryption and function-binding hash functions.
- **Optimal broadcast encryption.** As an additional application, we show how to adapt our flexible broadcast encryption scheme to obtain an *optimal* broadcast encryption scheme (in the traditional centralized model with a trusted key issuer) in the random oracle model from plain witness encryption and function-binding hash functions. An optimal broadcast encryption scheme is one where all of the scheme parameters (public key size, secret key size, and ciphertext size) all scale polylogarithmically with the number of users n , and was previously known from multilinear maps [BWZ14], *positional* witness encryption [GVW19], on combinations of (non-falsifiable or idealized) pairing-based and lattice-based assumptions [AY20, AWY20], or from new (non-falsifiable) lattice assumptions [BV22, Wee22].

By instantiating the underlying witness encryption scheme with the recent constructions [Tsa22, VWW22] based on evasive LWE, we realize for the first time flexible (and distributed) broadcast encryption and registered ABE for general policies from lattice assumptions. Previously, these were only known from *iO*. As we elaborate in Section 1.1, our approach to the above problem is to start from an approach that relies on obfuscation and SSB hash functions, and then roughly speaking, replace *iO* with plain witness encryption and the SSB hash function with a function-binding hash function. In some sense, we are able to substitute the strong notion of *iO* with the comparatively weaker notion of witness encryption by using the more expressive notion of function-binding hash functions in place of an SSB hash function. Broadly speaking, we are optimistic that the techniques we develop will allow us to use witness encryption in place of *iO* in other settings, and thus, bring us closer to simpler and more practical realizations of many advanced cryptographic primitives.

Why witness encryption? Since witness encryption is often regarded as an “obfustopia” primitive, a natural question one might ask is whether it is interesting to study cryptographic constructions from plain witness encryption. We believe the answer is yes. As noted previously, recent advances have introduced new routes [BJK⁺18, BLOW20] and constructions [CVW18, Tsa22, VWW22] for building (plain) witness encryption in ways that do not seem sufficient for indistinguishability obfuscation (nor stronger forms of witness encryption). Moreover, these constructions are simpler and far more efficient than all existing constructions of indistinguishability obfuscation. Thus, witness encryption

can be an easier primitive to build, and provide a more efficient path towards realizing advanced cryptographic notions that are currently only known from obfuscation. In more detail, [BJK⁺18] show how to construct a non-trivial exponentially-efficient witness encryption assuming sub-exponential hardness of the standard learning with errors (LWE) assumption, and [BIOW20] show how to construct witness encryption in the generic (pairing-free) group model under the hypothesis that approximating the minimal distance in linear codes is NP-hard (for super-logarithmic approximation factors). Recent works [Tsa22, VWW22] construct witness encryption under a new “evasive LWE” assumption, a new knowledge assumption that conjectures hardness of LWE even given some trapdoor information. Moreover, at least with respect to black-box separations [GMM17], witness encryption is a demonstrably weaker notion than indistinguishability obfuscation.

1.1 Technical Overview

In this section, we provide a general overview of our notion of function-binding hash functions and how to combine it with witness encryption to realize new cryptographic applications. To demonstrate our approach, we first introduce our notion of flexible broadcast encryption (which generalizes the notion of distributed broadcast encryption from [BZ14]) and describe a simple construction of flexible broadcast encryption from indistinguishability obfuscation and somewhere statistically binding hash functions. We then show how we instantiate an analogous template using witness encryption and function-binding hash functions.

Flexible broadcast encryption. In a traditional broadcast encryption scheme [FN93], each user is associated with an index $i \in [n]$ and a broadcaster can encrypt a message to a set of recipients $S \subseteq [n]$. Any user in the set S can use their private key to decrypt the encrypted broadcast, while users outside the set S should learn no information about the message (even if they combine their keys together). The efficiency requirement is that the size of the ciphertext should be sublinear in the size of S (ideally, polylogarithmic in $|S|$). Note that we assume that the decrypter knows the set S . In traditional broadcast encryption, a central authority is needed to issue the decryption keys to users. The central authority is fully trusted, and if compromised, their long-term secret key can be used to decrypt all ciphertexts. Broadcast encryption is a well-studied primitive and construction are known from a broad range of cryptographic assumptions [FN93,>NNL01, HS02, DF02, BGW05, BW06, GW09, BW13, BZ14, BWZ14, CGW15, GVV19, AY20, AWY20, Wee21, BV22, Wee22].

Distributed broadcast encryption [BZ14] provides an elegant approach to remove the trusted key issuer in the setting of broadcast encryption. Here, instead of a central authority issuing keys, users generate their own key. Like broadcast encryption, each user in a distributed broadcast encryption is still associated with an index $i \in [n]$, and the implicit assumption is that indices for different users are unique. The encrypter in this case can encrypt a message by specifying a set $S \subseteq [n]$ of user indices. As before, the size of the broadcast ciphertext should be sublinear, or preferably, polylogarithmic, in the number of users in the broadcast set. Anyone in the broadcast set can decrypt using their own secret key. Analogous to broadcast encryption where we assume the decrypter knows the set of users associated with an encrypted broadcast, we assume the decrypter in the distributed broadcast scheme knows the set of public keys associated with the broadcast ciphertext. Unlike the case with traditional broadcast encryption, distributed broadcast is currently only known from *iO* [BZ14].

Flexible broadcast encryption is a further generalization of distributed broadcast encryption where we remove the index-dependence for users. Here, users simply generate their public key and post it to the public bulletin board (e.g., a public-key directory). There is no notion of a user “index” and indeed users can generate their keys independently and without any coordination.¹ We do assume that there is a global set of public parameters for the scheme (that users use when generating keys).

An obfuscation-based approach for flexible broadcast encryption. We start by describing a simple approach to construct a flexible broadcast encryption from indistinguishability obfuscation and somewhere statistically binding (SSB) hash functions [HW15]. The approach we take here is an adaptation of the registered ABE scheme

¹Note that we can generically obtain a flexible broadcast encryption scheme from a distributed broadcast encryption scheme by having users sample their index randomly. To support a maximum of ℓ users, we would instantiate the scheme with $n = \Omega(\ell^2)$ indices. This is sufficient if the number of users is *a priori* bounded, though it may incur a *quadratic* blowup in the size of some scheme parameters. If the underlying distributed broadcast encryption scheme supports a super-polynomial number of users (i.e., $n = \lambda^{\omega(1)}$), then it directly implies a flexible broadcast encryption for an arbitrary polynomial number of users.

from [HLWW23] (rather than the Boneh-Zhandry approach [BZ14] based on a reduction to key-agreement).

- The public parameters $pp = (pk, hk)$ for the scheme contains a public key pk for a standard public-key encryption scheme and a hash key hk for an SSB hash function.
- To join the system, a user samples randomness r and computes an encryption of 1 under pk with randomness r : $c = \text{PKE.Enc}(pk, 1; r)$. The user's public key is the ciphertext c and the decryption key is the randomness r .
- To encrypt a message m to an (ordered) set of public keys $(c_1, \dots, c_n)$, the encrypter now proceeds as follows:
 - The encrypter starts by hashing the public keys using the SSB hash function to obtain a digest $\text{dig} = H(hk, (c_1, \dots, c_n))$. Recall that in an SSB hash function, one can give a succinct proof (of size $\text{poly}(\lambda, \log n)$ where λ is a security parameter) that the value at index i is c_i (with respect to (hk, dig)).
 - The encrypter now prepares a program $\mathcal{P}$ that has message m , the public parameters $pp = (hk, pk)$, and the digest dig hard-wired inside. The program takes as input an index $i \in [n]$, a public key c , an opening π , and a secret key r and checks the following:
 - * **Inclusion in broadcast set:** The opening π is a valid opening for c at index i with respect to (hk, dig) .
 - * **Knowledge of secret key:** The pair (c, r) is a valid public/secret key-pair: $c = \text{PKE.Enc}(pk, 1; r)$.
 If all of these properties are satisfied, then the program outputs the message m . Otherwise, it outputs $\perp$.
 - The broadcast ciphertext is an obfuscation of program $\mathcal{P}$: $\text{ct} \leftarrow iO(\mathcal{P})$.
- During decryption, the user know both the ciphertext $\text{ct} = iO(\mathcal{P})$ and the set of public keys $(c_1, \dots, c_n)$ associated with ct .² Suppose the user's public key is $c_i \in (c_1, \dots, c_n)$ and let r_i be their secret key. The user starts by computing $\text{dig} = H(hk, (c_1, \dots, c_n))$ together with an opening π for (i, c_i) with respect to (hk, dig) . It runs the obfuscated program ct on input (i, c_i, π, r_i) .

Correctness of the scheme follows as long as the obfuscation scheme is functionality-preserving. Security follows via a simple hybrid argument:

- In the normal flexible broadcast encryption security game, the challenge ciphertext is an encryption of m to a set $S = (c_1, \dots, c_n)$, where $c_i = \text{PKE.Enc}(pk, 1; r)$. Since the adversary cannot be in the broadcast set in the security game, each public key c_i is an *honestly-generated* encryption of 1: $c_i = \text{PKE.Enc}(pk, 1; r_i)$. We appeal to semantic security of the public-key encryption scheme to replace each c_i with an encryption of 0 (i.e., $c_i = \text{PKE.Enc}(pk, 0; r_i)$).
- Next, we replace the hash key hk with one that is statistically binding at position 1. This is computationally indistinguishable from the normal hash key if the SSB scheme satisfies index hiding. Now $\text{dig} = H(hk, (c_1, \dots, c_n))$ and c_1 is now an encryption of 0. We conclude that every input (i, c, π, r) where $i = 1$ causes the program $\mathcal{P}$ to output $\perp$. This is because the only valid opening for index $i = 1$ is $c = c_1$ (since the hash function is *statistically* binding at index 1). However, c_1 is an encryption of 0, so it cannot be the case that $c_1 = \text{PKE.Enc}(pk, 1; r)$ by (perfect) correctness of PKE. Then, by iO security, we can replace $\mathcal{P}$ with an obfuscation of the program $\mathcal{P}_1$ that outputs $\perp$ whenever $i = 1$.
- More generally, we can let $\mathcal{P}_t$ be the program $\mathcal{P}$ that always outputs $\perp$ whenever the input index satisfies $i \leq t$. By the above argument, if we sample hk to be statistically binding at index t , then the obfuscations of programs $\mathcal{P}_{t-1}$ and $\mathcal{P}_t$ are computationally indistinguishable.
- Finally, the program $\mathcal{P}_n$ outputs $\perp$ on all indices $i \leq n$. This program outputs $\perp$ on *all* inputs, so an obfuscation of $\mathcal{P}_n$ is computationally indistinguishable from an obfuscation of the program that outputs $\perp$ on *all* inputs.

²Note that set of public keys could just be a set of indices if we include a mapping between indices and user public keys (i.e., a public-key directory) as part of the public parameters of the flexible broadcast encryption scheme. This is the setting considered in [BZ14]. More generally, the set of users could admit a succinct description (e.g., "all computer science students") and the decrypter would look up the public keys associated with the members of the set.

In the final hybrid where the ciphertext is an obfuscated program that always outputs $\perp$, the adversary's view is *independent* of the message m , and semantic security holds.

Using witness encryption instead of iO . Suppose we try to implement the above strategy using witness encryption in place of indistinguishability obfuscation. Observe that the program $\mathcal{P}$ defined above is a program that is checking membership in an NP language and outputting m if the input is a valid witness. Specifically, we can view the hard-wired components (hk, pk, dig) as the statement and the input (i, c, π, r) as the witness. The associated NP relation then checks the inclusion-in-broadcast-set and knowledge-of-secret-key properties defined above. Thus, we can derive the same functionality as above by replacing the obfuscation of $\mathcal{P}$ with a witness encryption of m with respect to the statement (hk, pk, dig) . Correctness now follows exactly as before.

The challenge is in the security proof. Suppose we try a similar strategy as above where we first replace the public keys $(c_1, \dots, c_n)$ with encryptions of 0. Let $dig = H(hk, (c_1, \dots, c_n))$ as before. To leverage security of the witness encryption scheme and argue that the message is computationally hidden, we need to show that there does not exist a witness (i, c, π, r) for the statement (hk, pk, dig) . Certainly, any c where $c \in (c_1, \dots, c_n)$ cannot be part of a valid witness. However, we cannot rule out inputs where $c \notin (c_1, \dots, c_n)$ because it is possible for there to *exist* some $c = \text{PKE.Enc}(pk, 1; r)$ and opening π such that π is a valid opening for (i, c) with respect to (hk, dig) . While finding such an opening is computationally hard (by SSB security), such openings can certainly exist for some *index* $i \in [n]$. The crux of the issue is that SSB hash functions can only bind to a *single* component of the input, whereas to appeal to security of witness encryption, we need to argue that the digest dig rules out invalid openings to *all* indices.

Using iO , the ability to statistically bind to a single index was sufficient to argue security because we are able to “save our progress” within the obfuscated program. Namely, we can iteratively rule out inputs (i, c, π, r) where $i \leq t$, and then just focus on ruling out inputs where $i \leq t + 1$. In this step, we only have to consider inputs of the form $(t + 1, c, \pi, r)$ which we can handle by programming the SSB hash function to statistically bind on index $t + 1$. A similar step-by-step approach does not work if we use witness encryption. While we can certainly embed the threshold t into the NP relation (i.e., only accept witnesses (i, c, π, r) where $i > t$), witness encryption does not provide any hiding property for t in this case. Indeed, the adversary can distinguish between a witness encryption ciphertext encrypted with respect to threshold t and one encrypted with respect to threshold $t + 1$. In the case of iO , the thresholds themselves are hidden. In fact, this limitation motivated the formulation of a stronger version of “positional witness encryption” [GLW14, GVW19] which augments witness encryption with an additional “index hiding” property that allows one to secretly embed a threshold within a witness encryption ciphertext. Using positional witness encryption, it is possible to carry out the proof strategy defined above. Unfortunately, positional witness encryption appears to be a much stronger notion than plain witness encryption and the only known instantiations are from iO .

Function-binding hash functions. The key barrier to instantiating the above framework is that SSB hash functions only bind statistically to a *single* input, but to rule out the *existence* of *any* valid tuple (i, c, π, r) , we would seemingly need to statistically bind on all inputs simultaneously. The latter is clearly impossible if we require that the digest be compressing. We take a different approach. Instead of statistically binding to part of the *input* to the hash function, we instead statistically bind to the *output* of a function of the inputs. Namely, let $f : (\{0, 1\}^\ell)^n \rightarrow \{0, 1\}^\ell$ be a function. We say that a keyed function H with hash key hk is statistically function-binding for f if for all inputs $x_1, \dots, x_n \in \{0, 1\}^\ell$ and computing $dig = H(hk, (x_1, \dots, x_n))$, there does not exist an opening π for $(j, \hat{x}_j)$ with respect to (hk, dig) whenever

$$\forall (x'_1, \dots, x'_{j-1}, x'_{j+1}, \dots, x'_n) : f(x'_1, \dots, x'_{j-1}, \hat{x}_j, x'_{j+1}, \dots, x'_n) \neq f(x_1, \dots, x_n).$$

In other words, the only possible openings at *any* index j are to values $\hat{x}_j$ where there exists an assignment to the remaining variables that are consistent with an evaluation of $f(x_1, \dots, x_n)$. As in the case of SSB hash functions, we require succinct local openings to any index. We describe two cases of interest:

- **Index functions:** As a special case, an SSB hash function is a function-binding hash function for the index function $\text{ind}_i : (\{0, 1\}^\ell)^n \rightarrow \{0, 1\}^\ell$ where $\text{ind}_i(x_1, \dots, x_n) := x_i$.
- **Disjunction of predicates:** Let $g : \{0, 1\}^\ell \rightarrow \{0, 1\}$ be a binary-valued predicate and let $f_g : (\{0, 1\}^\ell)^n \rightarrow \{0, 1\}$ be the block-wise disjunction function

$$f_g(x_1, \dots, x_n) := \bigvee_{i \in [n]} g(x_i). \quad (1.1)$$

Suppose a hash key hk is statistically binding for the function f_g , and suppose $\text{dig} = H(hk, (x_1, \dots, x_n))$ for an input $(x_1, \dots, x_n)$ where $g(x_i) = 0$ for all $i \in [n]$. Function-binding security now says that there does *not* exist any opening $(j, \hat{x}_j)$ for any $j \in [n]$ and $\hat{x}_j \in \{0, 1\}^\ell$ where $g(\hat{x}_j) = 1$.

Like index hiding in the context of SSB hash functions, we additionally require that the hash key hk associated with a predicate f computationally hides the function f . Similarly, we require that given a digest $\text{dig} = H(hk, (x_1, \dots, x_n))$, one can efficiently compute a succinct local opening π to any index (j, x_j) , where $|\pi| = \text{poly}(\lambda, \ell, \log n)$, where ℓ is the output length of f . We provide the formal definition in [Section 3](#).

Using function-binding hash functions. Consider again our template for constructing flexible broadcast encryption, except we replace the SSB hash function in the construction with a function-binding hash function. Our security reduction now proceeds as follows:

- As before, we start by switching the honest users' public keys $c_1, \dots, c_n \leftarrow \text{PKE.Enc}(\text{pk}, 0)$ to encryptions of 0.
- Next, we sample the hash key hk to be statistically function-binding for the function $f_g(c_1, \dots, c_n)$ where f_g is the disjunction-of-predicates function from [Eq. \(1.1\)](#) and $g(c) = \text{PKE.Dec}(\text{sk}, c)$, and sk is the decryption key associated with pk in the public-key encryption scheme. Observe now that in the challenge ciphertext, $f_g(c_1, \dots, c_n) = 0$ since each c_i is an encryption of 0.
- Security now follows by security of the witness encryption scheme. We claim that with overwhelming probability over the choice of the hash key hk , the statement $(hk, \text{pk}, \text{dig})$ is false. Take any candidate witness (i, c, π, r) . For this to be a valid witness, it must be the case that $c = \text{PKE.Enc}(\text{pk}, 1; r)$ and π is a valid opening for (i, c) with respect to (hk, dig) where $\text{dig} = H(hk, (c_1, \dots, c_n))$. Since c is an encryption of 1, $g(c) = 1$. Since f_g is a disjunction, the output of f_g on any tuple of inputs that includes c is necessarily $1 \neq 0 = f_g(c_1, \dots, c_n)$. Function-binding security now says that there does not exist any valid π for (i, c) with respect to (hk, dig) .

The use of the function-binding hash function eliminates the need for a step-by-step hybrid strategy. As such, we can rely directly on *plain* witness encryption to complete the analysis. This yields the first flexible (and correspondingly, distributed) broadcast encryption scheme that does not rely on *iO*. Like the Boneh-Zhandry distributed broadcast encryption scheme from *iO*, the size of the ciphertexts in our scheme scale with $\text{poly}(\lambda, \log n)$, where n is the number of users in the broadcast set. We provide the full construction and analysis in [Section 4](#).

Optimal broadcast encryption. A flexible broadcast encryption scheme immediately implies a traditional broadcast encryption scheme with a central key issuer. Here, the central key issuer would generate the public/private keys for all of the users and publish the public keys for each user as the master public key for the broadcast encryption system. Instantiated with our flexible broadcast encryption scheme, this yields a construction with a linear-size public key, but short ciphertexts and secret keys (scaling polylogarithmically with the number of users). It is straightforward to extend our scheme to an *optimal* broadcast encryption where the size of the public key also scales polylogarithmically with the number of users by working in the random oracle model.

The reason our basic approach has a linear-size public key is because we concatenate n user keys $c_1, \dots, c_n$ to form the master public key for the broadcast encryption scheme. The key idea behind our optimal broadcast encryption scheme is to instead derive each user's public key from a hash of the user index i : that is, we compute $c_i \leftarrow h(i)$, where h is modeled as a random oracle. On the one hand, the master public key only needs to contain the public parameters for our flexible broadcast encryption system, which is now independent of the number of users in the scheme. On the flip side, we need a way to generate decryption keys for the users.

To recover functionality, we start with a more general view of our scheme. Recall that in our scheme, an honestly-generated public key c_i is an encryption of 1 and the associated secret key is a proof that c_i is an encryption of 1. In this case, the proof is the encryption randomness. In the security analysis, each c_i is replaced with an encryption of 0. In this case, there no longer exists a proof (i.e., encryption randomness) that asserts c_i to be an encryption of 1. We can more abstractly view this as implementing a set system where the honestly-generated public keys c_i are elements of some set S (i.e., encryptions of 1 in our case), and the associated secret key is a (statistically sound) proof of membership that $c_i \in S$. In the security proof, the public keys are sampled from some set T that is disjoint from S (i.e., encryptions of 0 in our case). Finally, we also require that a random element in S should be computationally indistinguishable from a random element in T so we can change the distribution of the honest users' keys in the

security proof. Now, suppose we can construct sets $S, T \subseteq \{0, 1\}^k$ where S is dense (i.e., a random element in $\{0, 1\}^k$ is contained in S with all but negligible probability), and moreover, given a trapdoor, it is possible to construct a proof of membership for any element $c \in S$. We refer to this notion as a “trapdoor proof system.” Trapdoor proof systems can be constructed in a straightforward manner from a non-interactive zero-knowledge proof in conjunction with a public-key encryption scheme with pseudorandom ciphertexts (see [Section 5.1](#)). Using a trapdoor proof system, we can now adapt our flexible broadcast encryption scheme to obtain an optimal broadcast encryption scheme in the random oracle model:

- The setup algorithm for the broadcast encryption scheme samples a trapdoor td for our trapdoor proof system. Let $S, T \subseteq \{0, 1\}^k$ be the sets associated with the trapdoor proof system. Let $h: [n] \rightarrow \{0, 1\}^k$ be a hash function that is modeled as a random oracle.
- The public key for user i is $c_i \leftarrow h(i)$. With overwhelming probability, $c_i \in S$. The secret key r_i is a proof of membership that $c_i \in S$, which can be sampled using the trapdoor td for the trapdoor proof system.
- Encryption and decryption proceed exactly as in the flexible broadcast encryption scheme.

The security analysis also proceeds analogously to that for our flexible broadcast encryption scheme. Namely, we first move the honest users’ public keys from S to T (so no proofs of membership exist), and then appeal to statistical function-binding and security of witness encryption. We refer to [Section 5](#) for the full details.

Registered attribute-based encryption. In [Section 6](#), we show that function-binding hash functions and plain witness encryption can also be used to obtain a registered ABE scheme that supports an arbitrary number of users and supporting general policies. Previously, this was also only known from indistinguishability obfuscation [[HLWW23](#)]. Non-obfuscation-based constructions of registered ABE could only support a bounded number of users [[HLWW23](#)] or support less-expressive policies such as Boolean formulas [[HLWW23](#)] or equality predicates [[GHMR18](#), [GHM⁺19](#), [GV20](#), [CES21](#), [GKMR23](#)]. We sketch the basic idea here (which closely parallels the *iO*-based approach from [[HLWW23](#)]):

- The common reference string for the registered ABE scheme consists of a public key pk for a public-key encryption scheme and the hash key for a function-binding hash function.
- As in our flexible broadcast encryption scheme, to generate a public key, a user samples randomness r and computes their public key as $c = \text{PKE.Enc}(pk, 1; r)$. Their secret key is the encryption randomness r .
- Suppose n users register with public keys $(c_1, \dots, c_n)$ and attributes $(x_1, \dots, x_n)$. The aggregated public key is a hash of the public keys and associated attributes: $\text{mpk} = \text{dig} = H(\text{hk}, ((c_1, x_1), \dots, (c_n, x_n)))$. The helper decryption key for user i is an opening π_i for (c_i, x_i) with respect to (hk, dig) .
- An encryption of message m under a policy $\mathcal{P}$ consists of a witness encryption ciphertext for m with respect to the statement $(pk, \text{hk}, \text{dig}, \mathcal{P})$. A witness (i, c, x, π, r) is valid if the following holds:
 - **Valid opening:** The opening π is a valid opening for (c, x) at index i with respect to (hk, dig) .
 - **Policy satisfiability:** The associated attribute satisfies the policy: $\mathcal{P}(x) = 1$.
 - **Knowledge of secret key:** The pair (c, r) is a valid public/secret key-pair: $c = \text{PKE.Enc}(pk, 1; r)$.

We show that this scheme is secure in a *selective* model where the adversary commits to its challenge policy $\mathcal{P}$ at the beginning of the security game. The security proof follows a similar hybrid structure as the security proof for our flexible broadcast encryption scheme:

- We first replace the honest users’ public keys with encryptions of 0.
- Next, we sample the hash key to be statistically function-binding for the function $f_g((c_1, x_1), \dots, (c_n, x_n))$ where f_g is the disjunction-of-predicates function from [Eq. \(1.1\)](#) and $g(c, x) := \text{PKE.Dec}(\text{sk}, c) \wedge \mathcal{P}(x)$. Since we need to program the challenge policy $\mathcal{P}$ into the hash key, we are only able to argue selective security.

- Security now follows by security of the witness encryption scheme. Specifically, with overwhelming probability over the choice of the hash key, the statement (hk, pk, dig) is false. This is because $f_g((c_1, x_1), \dots, (c_n, x_n)) = 0$ by construction³ whereas a valid witness (i, c, x, π, r) must satisfy $g(c, x) = 1$.

Constructing function-binding hash functions. To complete the loop, we finally show how to construct function-binding hash functions. In this work, we describe a construction that supports the disjunction-of-blocks family of functions f_g (for arbitrary g) from Eq. (1.1). This is the function family used in our applications to flexible broadcast encryption and registered ABE. Our construction proceeds very similarly to the construction of SSB hash functions from fully homomorphic encryption [HW15].

Specifically, let $(x_1, \dots, x_n)$ be the input to our function-binding hash function; for ease of exposition, suppose that $n = 2^d$ is a power of two. We combine a Merkle tree [Mer89] with a (leveled) homomorphic encryption scheme. We construct a complete binary tree of depth d , where the leaves of the binary tree are associated with the labels $x_1, \dots, x_n$. At the base level (the leaves), we homomorphically evaluate the function g on the input. Note that the description of g itself is encrypted as part of the hash key, since we require the hash key to hide the function. The value of each internal node is then obtained by homomorphically evaluating the OR function on its child ciphertexts. Observe that at the end of this process, the root of the tree will be an encryption of $f_g(x_1, \dots, x_n)$. The function-binding property now follows from (perfect) correctness of the encryption scheme. We provide the full details and proof in Section 3.1.

From selective to adaptive security in the random oracle model. The aforementioned constructions of flexible/optimal broadcast encryption and registered ABE all satisfy a selective notion of security where the adversary has to pre-commit to the set of corrupted users in the security game. We do note that our flexible/optimal broadcast encryption schemes, however, satisfy a stronger notion of “semi-static security” [GW09] where the adversary only needs to commit to a *super-set* of its challenge set (but is not allowed to corrupt any users in its committed set). Previously, Gentry and Waters [GW09] showed that a broadcast encryption scheme with semi-static security implies an adaptively-secure scheme in the random oracle model. Using the techniques of [GW09], we show in Appendix B how to transform the flexible broadcast encryption scheme above into one that satisfies adaptive security in the random oracle model. Similarly, our registered ABE construction described above satisfies a weaker policy-selective security without corruption queries. Namely, in the security game, the adversary has to declare its challenge policy upfront and it is not allowed to corrupt honestly-generated keys (but it can still generate its own keys and register those). We show in Appendix C that in the random oracle model, we can apply a similar approach based on [GW09] to support corruption queries in our registered ABE scheme.⁴ We note that the *iO*-based construction of registered ABE of [HLWW23] achieves full adaptive security. We leave it as an open question to satisfy this stronger notion from plain witness encryption.

Flexible broadcast encryption from registered ABE. As a final contribution, we also show that a registered ABE scheme *generically* implies a flexible broadcast encryption scheme.⁵ The idea is simple: the public parameters for the flexible broadcast encryption scheme is the CRS for the registered ABE scheme. Now, to broadcast to a set of public keys $(pk_1, \dots, pk_n)$, the encrypter initializes a registered ABE scheme and registers $pk_1, \dots, pk_n$ to a dummy attribute x to obtain a master public key mpk . It then encrypts the message m with respect to the master public key and a dummy policy P where $P(x) = 1$. To decrypt, a user would run the registration algorithm itself, derive the associated helper decryption key, and combine with their own secret key to recover the message m . We describe this transformation in Section 7.

By applying the generic transformation to our registered ABE scheme, we obtain another approach for constructing flexible broadcast encryption from witness encryption and function-binding hash functions. We do note though that the flexible broadcast encryption scheme obtained via this generic approach satisfies a *weaker* notion of static security

³Specifically, if c_i is the public key of an honest user, then $\text{PKE.Dec}(\text{sk}, c_i) = 0$ whereas if x_i belongs to a corrupted user, then $\mathcal{P}(x_i) = 0$ by the admissibility restriction on the registered ABE adversary. In either case, $g(c_i, x_i) = 0$ for all $i \in [n]$.

⁴The transformed scheme is still policy-selective; however, this can be removed generically via complexity leveraging and assuming subexponential hardness. Note that one cannot use complexity leveraging to handle corruption queries without having the ciphertext size grow with the number of registered users.

⁵Note that the direct approach of encoding the broadcast set as part of the policy in the ABE scheme does *not* yield a flexible broadcast encryption scheme with the required efficiency. Namely, in existing registered ABE schemes (including the one in this work), the size of the ciphertext scales with the size of the policy. Using this to implement broadcast encryption yields a flexible broadcast encryption scheme where the size of the ciphertext scales *linearly* with the number of users.

(as opposed to semi-static security). Thus, the generic transformation does not subsume our direct construction of flexible broadcast encryption which satisfies semi-static security. As noted above, a semi-statically secure scheme can be bootstrapped to an adaptively-secure one in the random oracle model, but a similar transformation is not known if we start from a statically-secure scheme.

The transformation described here critically assumes that the key-generation process is “stateless:” namely, users can independently generate their keys without knowledge of the current state of the key curator. In schemes like the pairing-based construction of [HLWW23], the registration process is not stateless and users need to know the current number of registered users when registering. In this setting, our transformation can still be applied to obtain a *distributed* broadcast encryption scheme from assumptions over pairing groups. We provide additional details in Remark 7.6. Ultimately, in conjunction with [HLWW23], we also obtain the first distributed broadcast encryption scheme from pairing-based assumptions (over a composite-order group). The resulting scheme supports an *a priori bounded* number of users, requires a quadratic-size CRS (inherited from [HLWW23]), and satisfies static security.

1.2 Concurrent Work of Brakerski et al. [BBK⁺23]

In an independent and concurrent work, Brakerski et al. [BBK⁺23] constructed succinct non-interactive arguments (SNARGs) for monotone policy batch NP. A key technical ingredient introduced in their work is the notion of a predicate-extractable hash family. Their notion shares some similarity to our notion of function-binding hash functions. In both primitives, the hash key is associated with some function $f \in \mathcal{F}$ from a family $\mathcal{F}$, and the evaluation of the hash function on some input x will in some way “bind” to $f(x)$. Namely, the hash value reflects some global property of the hashed data. Moreover, the function f associated with the hash key should be computationally hidden. However, there are some important distinctions between the notions that we highlight below.

The work of [BBK⁺23] presents two sets of definitions. The paper first provides a broad definition of a predicate-extractable hash family for a general function family $\mathcal{F}$ (c.f., [BBK⁺23, Definition 5.1]). Next, they consider a specialized definition tailored for bit-fixing predicates (i.e., a bit-fixing predicate takes as input a bitstring and compares the values on a fixed set of indices to a particular target value y , and outputs 1 if they match and 0 otherwise) [BBK⁺23, Definition 5.2]. In addition to restricting the functionality, the predicate-extractable hash for bit-fixing predicates definition introduces additional security properties beyond those inherited from the main definition for general functions. The predicate-extractable hash function for bit-fixing predicates is used to obtain SNARGs for monotone policy batch NP.⁶

Our notion of function-binding hash functions differs from the general notion of predicate-extractable hash functions from [BBK⁺23, Definition 5.1] in two important ways:

- **Statistical vs. computational binding.** First, our binding property is *statistical* in that even a computationally unbounded adversary cannot construct a local opening for an “incompatible” input (i.e., an input that cannot be extended to one that is consistent with the function output stored in the hash). In contrast, the binding property in [BBK⁺23, Definition 5.1] is computational. (This also means that the notion of somewhere statistically binding hash functions [HW15] is not captured by their security definition.) This statistical binding property is critical for realizing our applications from plain witness encryption, but not necessarily needed for constructing proof systems. Note that while predicate-extractable hash functions require an explicit extraction algorithm, this property is defined independently of the verification algorithm. Thus, the combination of computational binding and extraction correctness does not imply a statistical binding property on the hash function.
- **Function binding.** Second, the (statistical) binding property in our definition is tightly coupled to the function f associated with the hash key chosen at setup. In contrast, the computational binding property from [BBK⁺23] is defined independently of the function f associated with the hash key and only requires that it be computationally difficult to open an index to both a 1 and 0. The more specialized definition of predicate-extractable hash for bit-fixing predicates adds an additional property they call “consistency of extraction” which does couple the security of the verify algorithm to the global hash property. This additional consistency property of the hash function is important for realizing their SNARG construction.

⁶There is also a third definition of predicate-extractable hash functions with tags for bit-fixing predicates. For simplicity, our discussion is limited to the first two.

The two notions also have some additional minor differences. Unlike [BBK⁺23], our definition does not require an algorithm to extract the function value associated with the hashed input. In addition, for certain function classes under our definition, it may be possible to open an input bit to both 0 and 1. As a trivial example, consider a function family $\mathcal{F}$ for which all functions $f \in \mathcal{F}$ ignore a certain bit. Finally, [BBK⁺23] consider function families as predicates with one-bit output, whereas we consider multi-bit outputs. For certain security definitions it might be possible to go from supporting one-bit output to supporting ℓ -bit outputs by concatenating ℓ hash values. This is conceptually similar to the observation that one can obtain the indistinguishability notion of functional encryption [BSW11] from earlier notions of predicate encryption [BW07, KSW08].

2 Preliminaries

In this section, we introduce the notation and definitions of standard cryptographic primitives that we use in this paper. We let $\mathbb{N} = \{1, 2, 3, \dots\}$ denote the set of natural numbers, and for any $n \in \mathbb{N}$, we write $[n]$ to denote the set $[n] = \{1, \dots, n\}$. For integers $a, b \in \mathbb{Z}$ with $a \leq b$, we write $[a, b]$ to denote the set $\{a, a+1, \dots, b\}$. For a set Σ , referred to as the *alphabet*, we denote Σ^* the set of strings consisting of 0 or more elements from Σ . We let Σ^n denote the set of n -character strings from Σ and $\Sigma^{\leq n}$ the set of string of length at most n . For a string $s \in \Sigma^*$, we let $|s|$ denote the length of s . Unless specified otherwise, we assume that a string s is defined over the binary alphabet $\{0, 1\}$.

We write $\lambda \in \mathbb{N}$ to denote the security parameter. We say that a function $p: \mathbb{N} \rightarrow \mathbb{N}$ is in the set $\text{poly}(\lambda)$ and is *polynomially-bounded* if there exists a constant c and an index $i \in \mathbb{N}$ such that $p(\lambda) \leq \lambda^c$ for all $\lambda \geq i$. We say that a function $\text{negl}: \mathbb{N} \rightarrow \mathbb{R}$ is negligible if for every constant $c > 0$ there exists $i \in \mathbb{N}$ such that $\text{negl}(\lambda) \leq \lambda^{-c}$ for all $\lambda \geq i$.

We use PPT to denote the acronym *probabilistic polynomial time*. We say that an algorithm is *stateful* if when invoked multiple times in succession, it implicitly takes its entire view so far, including previous inputs and random coins used, as input. We say that a function $f: \mathbb{N} \rightarrow \mathbb{N}$ is efficiently computable if there exists a PPT algorithm that computes f on all inputs.

For a distribution X , we write $x \leftarrow X$ to denote the process of sampling a value x from the distribution X . For a set X , we use $x \xleftarrow{\mathbb{R}} X$ to denote the process of sampling a value x from the uniform distribution over X . We use $x = A(\cdot)$ to denote the output of a deterministic algorithm and $x \leftarrow A(\cdot)$ to denote the output of a randomized algorithm. We write $x \leftarrow A(y; r)$ to denote the output of running the randomized algorithm A on input y with *explicit* randomness r . We write $x := y$ to denote the assignment of value y to x . For a distribution D , we define $\text{Supp}(D)$ to denote the support of the distribution D .

Boolean circuits. A Boolean circuit consist of input and output wires (each labeled with a bit) and gates (either AND, OR, or NOT gates unless specified otherwise) that are arranged according to a directed acyclic graph. We define the size of a circuit to be the number of input/output wires and gates in the circuit. We assume a canonical description for all Boolean circuits of a given size. We define the circuit depth to be the depth of the graph representing the circuit.

Message spaces. For a function $m: \mathbb{N} \rightarrow \mathbb{N}$, we say that a sequence $\mathcal{M} = \{\mathcal{M}_\lambda\}_{\lambda \in \mathbb{N}}$ is a *message space with message-length m* if for every $\text{msg} \in \mathcal{M}_\lambda$, $|\text{msg}| = m(\lambda)$. We reserve the special character $\perp$ to denote an invalid message.

Languages and witness relations. We consider sequences of languages $\mathcal{L} = \{\mathcal{L}_\lambda\}_{\lambda \in \mathbb{N}}$ that are parameterized by a security parameter λ . We say that a language has instance length $m = m(\lambda)$ if for all security parameters $\lambda \in \mathbb{N}$ and statements $x \in \mathcal{L}_\lambda$, we have that $|x| = m(\lambda)$. We say a language $\mathcal{L} = \{\mathcal{L}_\lambda\}_{\lambda \in \mathbb{N}}$ is polynomially-bounded if there exists a polynomial p such that for all $\lambda \in \mathbb{N}$, $\max\{|x| : x \in \mathcal{L}_\lambda\} \leq p(\lambda)$. A witness relation for an NP language $\mathcal{L} = \{\mathcal{L}_\lambda\}_{\lambda \in \mathbb{N}}$ with instances of length m is a deterministic, binary relation $\mathbf{R}_\mathcal{L}$ that can be computed in time $\text{poly}(\lambda, m(\lambda))$ and characterizes $\mathcal{L}$ by $\mathcal{L}_\lambda = \{x : \exists w, (1^\lambda, x, w) \in \mathbf{R}_\mathcal{L}\}$. We say that a string $w \in \{0, 1\}^*$ is a *witness* for an instance $x \in \mathcal{L}_\lambda$ if $(1^\lambda, x, w) \in \mathbf{R}_\mathcal{L}$. We let $\mathbf{R}_\mathcal{L}(1^\lambda, x)$ denote the set of all witnesses for x : namely, $\mathbf{R}_\mathcal{L}(1^\lambda, x) = \{w : \mathbf{R}_\mathcal{L}(1^\lambda, x, w) = 1\}$. We say that $\mathcal{L}$ is efficiently samplable with ℓ uniform bits if there exists an efficient algorithm Sample such that

$$\{x \leftarrow \text{Sample}(1^\lambda; r) : r \xleftarrow{\mathbb{R}} \{0, 1\}^\ell\} \equiv \{x \xleftarrow{\mathbb{R}} \mathcal{L}_\lambda\}.$$

We say that $\mathcal{L}$ is efficiently-recognizable if there exists an efficient algorithm Decide that takes as input the security parameter λ and an instance $x \in \{0, 1\}^m$ and outputs whether $x \in \mathcal{L}_\lambda$.

2.1 Witness Encryption

Witness encryption [GGSW13] allows users to encrypt a message with respect to an instance x of an NP language $\mathcal{L}$. Anyone who has a corresponding witness w for x should be able to decrypt, whereas if x is false, the ciphertext should computationally hide the underlying message. We formalize this as follows.

Definition 2.1 (Witness Encryption). Let $m \in \text{poly}(\lambda)$ and $\mathcal{M} = \{\mathcal{M}_\lambda\}_{\lambda \in \mathbb{N}}$ be a message space of length m . A witness encryption scheme WE for an NP language $\mathcal{L} = \{\mathcal{L}_\lambda\}_{\lambda \in \mathbb{N}}$ with instance length $n = n(\lambda)$ and witness relation $\mathbf{R}_{\mathcal{L}}$ consists of polynomial-time algorithms (Enc, Dec) with the following syntax:

- $\text{Enc}(1^\lambda, \text{msg}, x) \rightarrow \text{ct}$: A probabilistic algorithm that on input a security parameter λ , a message $\text{msg} \in \mathcal{M}_\lambda$, and an instance x for the language $\mathcal{L}$, outputs a ciphertext ct . We implicitly assume that ct includes 1^λ and x .
- $\text{Dec}(\text{ct}, w) \rightarrow \text{msg}$: A deterministic algorithm that on input a ciphertext ct and a witness w , outputs a message $\text{msg} \in \mathcal{M}_\lambda \cup \{\perp\}$.

We require that (Enc, Dec) satisfy the following properties:

- **(Perfect) correctness**: For all $\lambda \in \mathbb{N}$, messages $\text{msg} \in \mathcal{M}_\lambda$, and tuples $(1^\lambda, x, w) \in \mathbf{R}_{\mathcal{L}}$, it holds that

$$\Pr \left[\text{ct} \leftarrow \text{Enc}(1^\lambda, \text{msg}, x) : \text{Dec}(\text{ct}, w) = \text{msg} \right] = 1.$$

- **Message indistinguishability**: For all stateful PPT algorithms A , there exists a negligible function negl such that for all $\lambda \in \mathbb{N}$, it holds that

$$\Pr \left[\begin{array}{l} (x, \text{msg}_0, \text{msg}_1) \leftarrow A(1^\lambda) \\ b \xleftarrow{\mathcal{R}} \{0, 1\} \\ \text{ct}^* \leftarrow \text{Enc}(1^\lambda, \text{msg}_b, x) \\ b' \leftarrow A(\text{ct}^*) \end{array} : x \notin \mathcal{L}_\lambda \wedge b' = b \right] \leq 1/2 + \text{negl}(\lambda).$$

2.2 Public-Key Encryption

We recall the standard definition of semantically-secure public-key encryption.

Definition 2.2 (Public-Key Encryption). Let $m \in \text{poly}(\lambda)$ and $\mathcal{M} = \{\mathcal{M}_\lambda\}_{\lambda \in \mathbb{N}}$ be a message space of length m . A (semantically-secure) public-key encryption scheme PKE for message space $\mathcal{M}$ consists of polynomial-time algorithms (KeyGen, Enc, Dec) with the following syntax:

- $\text{KeyGen}(1^\lambda) \rightarrow (\text{pk}, \text{sk})$: A probabilistic algorithm that on input a security parameter λ , outputs a public key pk and a secret key sk . We implicitly assume that pk includes 1^λ and sk includes pk .
- $\text{Enc}(\text{pk}, \text{msg}) \rightarrow \text{ct}$: A probabilistic algorithm that on input a public key pk and a message $\text{msg} \in \mathcal{M}_\lambda$, outputs a ciphertext ct .
- $\text{Dec}(\text{sk}, \text{ct}) \rightarrow \text{msg}$: A deterministic algorithm that on input a secret key sk and a ciphertext ct , outputs a message $\text{msg} \in \mathcal{M}_\lambda \cup \{\perp\}$.

We require that (KeyGen, Enc, Dec) satisfy the following properties:

- **(Perfect) correctness**: For all $\lambda \in \mathbb{N}$, messages $\text{msg} \in \mathcal{M}_\lambda$, it holds that

$$\Pr \left[\begin{array}{l} (\text{pk}, \text{sk}) \leftarrow \text{KeyGen}(1^\lambda) \\ \text{ct} \leftarrow \text{Enc}(\text{pk}, \text{msg}) \end{array} : \text{Dec}(\text{sk}, \text{ct}) = \text{msg} \right] = 1.$$

- **Semantic security:** For all stateful PPT algorithms A , there exists a negligible function negl such that for all $\lambda \in \mathbb{N}$, it holds that

$$\Pr \left[\begin{array}{l} (\text{pk}, \text{sk}) \leftarrow \text{KeyGen}(1^\lambda) \\ (\text{msg}_0, \text{msg}_1) \leftarrow A(\text{pk}) \\ b \xleftarrow{\mathbb{R}} \{0, 1\} \\ \text{ct}^* \leftarrow \text{Enc}(\text{pk}, \text{msg}_b) \\ b' \leftarrow A(\text{ct}^*) \end{array} : b' = b \right] \leq 1/2 + \text{negl}(\lambda).$$

We say that the encryption scheme is a bit encryption scheme if $\mathcal{M}_\lambda = \{0, 1\}$ for all $\lambda \in \mathbb{N}$.

Remark 2.3 (Randomness Complexity). We assume for simplicity that the algorithms KeyGen , Enc in [Definition 2.2](#) use at most λ bits of randomness. This is without loss of generality since we can always expand the randomness using a pseudorandom generator (PRG).

2.3 Leveled Homomorphic Encryption

A (public-key) leveled homomorphic encryption scheme [[Gen09](#)] enables homomorphic evaluation on encrypted messages. Specifically, given any ciphertext ct encrypting a message x and a circuit C , the evaluation algorithm outputs a ciphertext for the message $C(x)$. A leveled homomorphic encryption scheme can only support circuits C with a priori bounded depth (specified during key generation). We formalize this as follows:

Definition 2.4 (Leveled Homomorphic Encryption). Let $m \in \text{poly}(\lambda)$ and $\mathcal{M} = \{\mathcal{M}_\lambda\}_{\lambda \in \mathbb{N}}$ be a message space of length m . A (public-key) leveled homomorphic encryption scheme LHE for a message space $\mathcal{M}$ consists of polynomial-time algorithms $(\text{KeyGen}, \text{Enc}, \text{Dec}, \text{Eval})$, where (Enc, Dec) have syntax corresponding to a standard public-key encryption scheme, and $(\text{KeyGen}, \text{Eval})$ have the following syntax:

- $\text{KeyGen}(1^\lambda, 1^L) \rightarrow (\text{pk}, \text{sk})$: A probabilistic algorithm that on input a security parameter λ and a depth bound L , outputs a public key pk and a secret key sk . We implicitly assume that pk includes 1^λ and 1^L and sk includes pk .
- $\text{Eval}(\text{pk}, C, \text{ct}) \rightarrow \text{ct}'$: A deterministic algorithm that on input a public key pk , a Boolean circuit C , and a ciphertext ct , outputs another ciphertext ct' .

We require that $(\text{KeyGen}, \text{Enc}, \text{Dec})$ is a correct public-key encryption scheme for all depth bounds $L \in \mathbb{N}$ given to KeyGen and satisfies semantic security for all polynomially-bounded depth bounds $L(\lambda)$ given to KeyGen . We additionally require the following properties:

- **Correct evaluation:** For any $\lambda \in \mathbb{N}$, $L \in \mathbb{N}$, $x \in \mathcal{M}_\lambda$, and Boolean circuits C of depth at most L , it holds that

$$\Pr \left[\begin{array}{l} (\text{pk}, \text{sk}) \leftarrow \text{KeyGen}(1^\lambda, 1^L) \\ \text{ct} \leftarrow \text{Enc}(\text{pk}, x) \\ \text{ct}' = \text{Eval}(\text{pk}, C, \text{ct}) \end{array} : \text{Dec}(\text{sk}, \text{ct}') = C(x) \right] = 1.$$

- **Compactness:** There exists a polynomial p such that for any $\lambda \in \mathbb{N}$, $L \in \mathbb{N}$, public/private key-pair $(\text{pk}, \text{sk}) \in \text{Supp}(\text{KeyGen}(1^\lambda, 1^L))$, message $x \in \mathcal{M}_\lambda$, $\text{ct} \in \text{Supp}(\text{Enc}(\text{pk}, x))$, and Boolean circuit C of depth at most L , it holds that

$$|\text{Eval}(\text{pk}, C, \text{ct})| \leq p(\lambda, L, |C(x)|).$$

We emphasize that the definition above stipulates perfect correctness, meaning that any honestly generated ciphertext (output from either encryption or the result of evaluation) corresponding to a message m will correctly decrypt to m . We use this property to guarantee statistical function binding in our construction of [Section 3](#).

Polylogarithmic depth decryption. A long sequence of works (c.f., [[BV11](#), [BGV12](#), [Bra12](#), [GSW13](#)]) have shown how to construct leveled homomorphic encryption schemes from the plain learning with errors (LWE) assumption. Moreover, in these constructions, the decryption circuit can be computed by a circuit of polylogarithmic depth, which we formally define as follows:

Definition 2.5 (Polylogarithmic Depth Decryption). We say that a leveled homomorphic encryption scheme $\text{LHE} = (\text{KeyGen}, \text{Enc}, \text{Dec}, \text{Eval})$ supports *polylogarithmic depth decryption* if there exists a polynomial p such that for any $\lambda \in \mathbb{N}, L \in \mathbb{N}$, public/private key-pair $(\text{pk}, \text{sk}) \in \text{Supp}(\text{KeyGen}(1^\lambda, 1^L))$, the depth of the decryption circuit computing $\text{Dec}(\text{sk}, \cdot)$ is at most $p(\log \lambda, \log L)$.

Theorem 2.6 (Leveled Homomorphic Encryption, [BV11, BGV12, Bra12, GSW13]). *Assuming the plain learning with errors (LWE) problem, there exists a leveled homomorphic encryption scheme LHE with polylogarithmic depth decryption.*

3 Function-Binding Hash Functions

In this section, we introduce the notion of a *function-binding* hash function. In the following, we consider the setting where the hash function satisfies *statistical function binding* and *computational function hiding*. This is the notion we rely on for our applications. However, we note that we can easily consider alternative notions where function binding is computational or function hiding is statistical.

Definition 3.1 (Function-Binding Hash Function). Let $\ell_{\text{blk}} = \ell_{\text{blk}}(\lambda)$ be the block size, $\ell_{\text{out}} = \ell_{\text{out}}(\lambda)$ be the output size. Let $\mathcal{F} = \{\mathcal{F}_\lambda\}_{\lambda \in \mathbb{N}}$ be a class of functions where each $\mathcal{F}_\lambda$ is a collection of functions $f: (\{0, 1\}^{\ell_{\text{blk}}(\lambda)})^* \rightarrow \{0, 1\}^{\ell_{\text{out}}(\lambda)}$ implementable by a circuit of size at most $s(\lambda) \cdot \text{poly}(k)$, where k is the number of input blocks to f . A *function-binding hash function* for $\mathcal{F}$ is a tuple of polynomial-time algorithms (Setup, SetupBinding, Hash, ProveOpen, VerOpen) with the following syntax:

- $\text{Setup}(1^\lambda, n) \rightarrow \text{hk}$: A probabilistic algorithm that on input a security parameter λ and a bound on the number of input blocks n (in *binary*), outputs a hash key hk . We implicitly assume that hk includes 1^λ and n .
- $\text{SetupBinding}(1^\lambda, n, f) \rightarrow \text{hk}$: A probabilistic algorithm that on input a security parameter λ , a bound on the number of input blocks n , and a function $f \in \mathcal{F}_\lambda$, outputs a hash key hk . We implicitly assume that hk includes 1^λ and n .
- $\text{Hash}(\text{hk}, (\text{hinp}_1, \dots, \text{hinp}_k)) \rightarrow \text{dig}$: A deterministic algorithm that on input a hash key hk and hash inputs $(\text{hinp}_1, \dots, \text{hinp}_k) \in (\{0, 1\}^{\ell_{\text{blk}}(\lambda)})^k$ for some $k \leq n$, outputs a digest dig .
- $\text{ProveOpen}(\text{hk}, (\text{hinp}_1, \dots, \text{hinp}_k), S) \rightarrow \pi$: A deterministic algorithm that on input a hash key hk , a sequence of hash inputs $(\text{hinp}_1, \dots, \text{hinp}_k) \in (\{0, 1\}^{\ell_{\text{blk}}(\lambda)})^k$ for any $k \leq n$, and a subset of indices $S \subseteq [k]$, outputs a proof π .
- $\text{VerOpen}(\text{hk}, \text{dig}, S, \{(i, \text{hinp}_i)\}_{i \in S}, \pi) \rightarrow \{0, 1\}$: A deterministic algorithm that on input a hash key hk , a digest dig , a subset $S \subseteq [k]$ for any $k \leq n$, a set of hash inputs $\{(i, \text{hinp}_i)\}_{i \in S}$ where each $\text{hinp}_i \in \{0, 1\}^{\ell_{\text{blk}}(\lambda)}$, and a proof π , outputs a bit $b \in \{0, 1\}$.

We require that (Setup, SetupBinding, Hash, ProveOpen, VerOpen) satisfy the following properties:

- **Efficiency:** There exist polynomials p_1, p_2, p_3 such that for all parameters $\lambda \in \mathbb{N}, k \in \mathbb{N}, n \in \mathbb{N}$ with $k \leq n$, any function $f \in \mathcal{F}_\lambda$, any hash key $\text{hk} \in \text{Supp}(\text{Setup}(1^\lambda, n))$, any tuple of inputs $(\text{hinp}_1, \dots, \text{hinp}_k) \in (\{0, 1\}^{\ell_{\text{blk}}(\lambda)})^k$, any digest $\text{dig} \in \text{Supp}(\text{Hash}(\text{hk}, (\text{hinp}_1, \dots, \text{hinp}_k)))$, any subset $S \subseteq [k]$, and any proof $\pi \in \text{Supp}(\text{ProveOpen}(\text{hk}, (\text{hinp}_1, \dots, \text{hinp}_k), S))$, the following hold:
 - $\text{Setup}(1^\lambda, n)$ and $\text{SetupBinding}(1^\lambda, n, f)$ run in time $p_1(\lambda, s(\lambda), \log n)$;
 - $|\text{dig}| \leq p_2(\lambda, \ell_{\text{out}}(\lambda), \log k)$; and
 - $|\pi| \leq |S| \cdot p_3(\lambda, \ell_{\text{out}}(\lambda), \log k)$.
- **Perfect completeness:** For all parameters $\lambda \in \mathbb{N}, k \in \mathbb{N}, n \in \mathbb{N}$ with $k \leq n$, inputs $(\text{hinp}_1, \dots, \text{hinp}_k) \in (\{0, 1\}^{\ell_{\text{blk}}(\lambda)})^k$, and subsets $S \subseteq [k]$, it holds that

$$\Pr \left[\begin{array}{l} \text{hk} \leftarrow \text{Setup}(1^\lambda, n) \\ \text{dig} = \text{Hash}(\text{hk}, (\text{hinp}_1, \dots, \text{hinp}_k)) \\ \pi = \text{ProveOpen}(\text{hk}, (\text{hinp}_1, \dots, \text{hinp}_k), S) \\ b = \text{VerOpen}(\text{hk}, \text{dig}, S, \{(i, \text{hinp}_i)\}_{i \in S}, \pi) \end{array} : b = 1 \right] = 1.$$

- **Statistical function binding:** For all stateful unbounded adversaries A and efficiently-computable functions $n \in \text{poly}(\lambda)$, there exists a negligible function negl such that for all $\lambda \in \mathbb{N}$,

$$\Pr \left[\text{Expt}_{A,n(\lambda)}^{\text{FB}}(\lambda) = 1 \right] \leq \text{negl}(\lambda),$$

where $\text{Expt}_{A,n}^{\text{FB}}(\lambda)$ is defined via the following security game between the adversary A and a challenger on common input 1^λ :

1. On input the security parameter λ , A outputs a function $f \in \mathcal{F}_\lambda$.
2. The challenger samples a hash key $\text{hk} \leftarrow \text{SetupBinding}(1^\lambda, n, f)$ for the function f and gives hk to A .
3. Algorithm A outputs a sequences of hash inputs $(\text{hinp}_1, \dots, \text{hinp}_k)$ where each $\text{hinp}_i \in \{0, 1\}^{\ell_{\text{blk}}(\lambda)}$ and $k \leq n$. Additionally, A outputs a set $S \subseteq [k]$ with associated values $\{(i, \text{hinp}_i^*)\}_{i \in S}$ and a proof π .
4. The experiment outputs 0 if there exists an extension of remaining inputs $\{(j, \text{hinp}_j^*)\}_{j \in [k] \setminus S}$ such that $f(\text{hinp}_1, \dots, \text{hinp}_k) = f(\text{hinp}_1^*, \dots, \text{hinp}_k^*)$. Otherwise, let $\text{dig} = \text{Hash}(\text{hk}, (\text{hinp}_1, \dots, \text{hinp}_k))$. The output of the experiment is $\text{VerOpen}(\text{hk}, \text{dig}, S, \{(i, \text{hinp}_i^*)\}_{i \in S}, \pi)$.

- **Computational function hiding:** For all stateful PPT algorithms A and efficiently-computable functions $n \in \text{poly}(\lambda)$, there exists a negligible function negl such that for all $\lambda \in \mathbb{N}$, it holds that

$$\Pr \left[\begin{array}{l} f \leftarrow A(1^\lambda) \\ \text{hk}_0 \leftarrow \text{Setup}(1^\lambda, n(\lambda)) \\ \text{hk}_1 \leftarrow \text{SetupBinding}(1^\lambda, n(\lambda), f) \\ b \xleftarrow{\mathcal{R}} \{0, 1\} \end{array} : f \in \mathcal{F}_\lambda \wedge A(\text{hk}_b) = b \right] \leq 1/2 + \text{negl}(\lambda).$$

Remark 3.2 (Somewhere Statistically Binding Hash Functions). [Definition 3.1](#) captures the notion of a somewhere statistically binding hashing [\[HW15\]](#) as a special case where the function class $\mathcal{F}$ is the class of *index functions* (i.e., functions f_i where $f_i(\text{hinp}_1, \dots, \text{hinp}_n) = \text{hinp}_i$).

3.1 Function-Binding Hash Functions for Disjunctions of Block Functions

We construct a function-binding hash for the function class consisting of disjunctions of block functions:

Definition 3.3 (Disjunction of Block Functions). Let $\ell_{\text{blk}} = \ell_{\text{blk}}(\lambda)$ be an input length parameter. We say a function $f_g: (\{0, 1\}^{\ell_{\text{blk}}})^* \rightarrow \{0, 1\}$ is a disjunction of block function if we can express it as

$$f_g(x_1, \dots, x_k) = \bigvee_{i \in [k]} g(x_i), \quad (3.1)$$

and where the function $g: \{0, 1\}^{\ell_{\text{blk}}} \rightarrow \{0, 1\}$ is a predicate on ℓ_{blk} -bit inputs. We say that $\mathcal{F} = \{\mathcal{F}_\lambda\}_{\lambda \in \mathbb{N}}$ is the class of disjunctions for block functions with input length $\ell_{\text{blk}} = \ell_{\text{blk}}(\lambda)$, size $s = s(\lambda)$, and depth $d = d(\lambda)$ if $\mathcal{F}_\lambda$ contains all functions $f_g: (\{0, 1\}^{\ell_{\text{blk}}})^* \rightarrow \{0, 1\}$ where the function g can be computed by a Boolean circuit of size s and depth d .

Our construction closely follows the construction of somewhere statistically binding hash functions of [\[HW15\]](#), which uses a Merkle tree [\[Mer89\]](#) where the underlying two-to-one hash function is built from a leveled homomorphic encryption scheme. We first introduce some preliminary notation and background on Merkle trees and then describe our full construction.

Merkle tree preliminaries. We recall some preliminary definitions regarding Merkle trees [\[Mer89\]](#) that we will use in our construction. A Merkle tree MT with depth α is a complete binary tree with 2^α leaf nodes at level 0 in the tree. Each leaf node is associated with an arbitrary fixed-length value. The values of the intermediate nodes in level $i \in [0, \alpha - 1]$ is defined by applying a two-to-one hash function to the two values associated with its child nodes. As such, there are $2^{\alpha-j}$ nodes at level j in the tree MT, and a single node at level α , which we refer to as the *root* node. We view the Merkle tree MT as a directed acyclic graph with edges running from each node to its parent. We index

each node in the tree by a pair $(j, i) \in [0, \alpha] \times [1, 2^{\alpha-j}]$ corresponding to its level j and its index i at level j , ordered from left to right.

Merkle tree notation. We also introduce some helpful notation for referring to different sets of nodes in the tree. Our treatment follows the generalization of authentication paths in Merkle trees used by [EFKP20]. Let MT be a depth α Merkle tree. For a leaf at index $i \in [2^\alpha]$, we write $\text{path}(i)$ to denote the set of nodes along the directed path from leaf i to the root. For a set of leaves $S \subseteq [2^\alpha]$, we define the subtree corresponding to S , denoted $\text{ST}(S)$, to be the union of all paths associated with the leaf nodes $i \in S$: $\text{ST}(S) = \bigcup_{i \in S} \text{path}(i)$. This is sometimes referred to as the Steiner tree for S (c.f., [NNL01]). We define the sibling of a node (j, i) , denoted $\text{sib}((j, i))$, to be the adjacent node in the tree who shares a parent with the node; this is either node $(j, i-1)$ if i is even or node $(j, i+1)$ if i is odd.

Definition 3.4 (Dangling Nodes). Let MT be a depth α Merkle tree, and let $S \subseteq [2^\alpha]$. We define the set $\text{dangling}(S)$ to be the set of nodes “dangling” off of the subtree $\text{ST}(S)$. These are the siblings of nodes in $\text{ST}(S)$ which are not contained in $\text{ST}(S)$. Formally,

$$\text{dangling}(S) = \{\text{sib}(\text{node}) : \text{node} \in \text{ST}(S) \wedge \text{sib}(\text{node}) \notin \text{ST}(S)\}.$$

Normally, one can prove membership of a leaf node i in a Merkle tree (with respect to a Merkle root) by providing the values associated with $\text{dangling}(\{i\})$; this is referred to as the authenticating path of i . To check the proof of membership, the verifier uses the value of node i together with the values of the nodes in $\text{dangling}(\{i\})$ to compute the root of the Merkle tree and compare with the given value. More generally, $\text{dangling}(S)$ defines an authentication path for a set of leaves. Namely, given the values of the nodes in S together with those in $\text{dangling}(S)$, one can again compute the value of the root. Assuming the underlying two-to-one hash function is collision-resistant, the value at the root is computationally unique, so the values in $\text{dangling}(S)$ can be used to authenticate the values of S with respect to the root.

Construction 3.5 (Function-Binding Hash for Disjunctions). Let $\ell_{\text{blk}} = \ell_{\text{blk}}(\lambda)$ be an input length parameter. Fix a size parameter $s_g = s_g(\lambda)$ and depth parameter $d_g = d_g(\lambda)$. We define the function class $\mathcal{F} = \{\mathcal{F}_\lambda\}_{\lambda \in \mathbb{N}}$ to consist of all disjunction-of-block-functions $f_g: (\{0, 1\}^{\ell_{\text{blk}}})^* \rightarrow \{0, 1\}$ where

$$f_g(\text{hinp}_1, \dots, \text{hinp}_k) = \bigvee_{i \in [k]} g(\text{hinp}_i), \quad (3.2)$$

and $g: \{0, 1\}^{\ell_{\text{blk}}} \rightarrow \{0, 1\}$ is a function that can be computed by a Boolean circuit C_g of size s_g and depth d_g . In particular, the function f_g can be computed by a circuit of size at most $s_g(\lambda) \cdot (2k + 1) \in s_g(\lambda) \cdot \text{poly}(k)$. Our construction relies on the following building blocks:

- Let $U(\cdot, \cdot)$ be a universal circuit that takes as input a circuit C_g of size s_g and depth d_g as well as an input $x \in \{0, 1\}^{\ell_{\text{blk}}}$ and outputs $C_g(x)$. Such a universal circuit exists with size $s \in \text{poly}(s_g)$ and depth $d_0 \in O(d_g)$ [CH85].
- Let LHE = (KeyGen, Enc, Dec, Eval) be a leveled homomorphic encryption scheme with polylogarithmic depth decryption (Definition 2.5). Let $d_0(\lambda) \in O(d_g(\lambda))$ be the depth of the universal circuit U defined above, and for $j = 1, 2, \dots, \lceil \log_2 n \rceil$, we recursively define $d_j(\lambda)$ to be the depth of the decryption circuit LHE.Dec instantiated with security parameter λ and depth parameter $d_{j-1}(\lambda)$. Note that since LHE supports polylogarithmic depth decryption, there exists a polynomial p such that $d_j(\lambda) \leq p(\lambda, \log j)$ for all $j \in [0, \lceil \log_2 n \rceil]$.

We show how to construct a function-binding hash for the function family $\mathcal{F}$. We construct the function-binding hash function FBH = (Setup, SetupBinding, Hash, ProveOpen, VerOpen) for the class of functions $\mathcal{F}$ as follows:

- **Setup**($1^\lambda, n$): On input the security parameter λ and a bound on the number of inputs $n \in \mathbb{N}$, let $\alpha = \lceil \log_2 n \rceil$. For $j \in [0, \alpha]$, sample encryption keys $(\text{pk}_j, \text{sk}_j) \leftarrow \text{LHE.Setup}(1^\lambda, 1^{d_j(\lambda)+1})$, where $d_j(\lambda)$ is defined above. Let $c_j \leftarrow \text{LHE.Enc}(\text{pk}_j, \text{sk}_{j-1})$ for $j \in [\alpha]$. Let $c_g \leftarrow \text{LHE.Enc}(\text{pk}_0, g_\perp)$ where $g_\perp: \{0, 1\}^{\ell_{\text{blk}}(\lambda)} \rightarrow \{0, 1\}$ is a dummy circuit of size $s(\lambda)$ that outputs 0 on all inputs. Output the hash key $\text{hk} = (\text{pk}_0, \dots, \text{pk}_\alpha, c_1, \dots, c_\alpha, c_g)$.
- **SetupBinding**($1^\lambda, n, f_g$): On input the security parameter λ , a bound on the number of inputs n , and a target function $f_g \in \mathcal{F}$, the binding setup function samples $(\text{pk}_0, \dots, \text{pk}_\alpha, c_1, \dots, c_\alpha)$ exactly as in **Setup**($1^\lambda, n$). It then samples $c_g \leftarrow \text{LHE.Enc}(\text{pk}_0, g)$ and outputs the hash key $\text{hk} = (\text{pk}_0, \dots, \text{pk}_\alpha, c_1, \dots, c_\alpha, c_g)$.

- **Hash**(hk, (hinp₁, ..., hinp_k)): On input the hash key hk = (pk₀, ..., pk_α, c₁, ..., c_α, c_g), and a tuple of inputs (hinp₁, ..., hinp_k) ∈ ({0, 1}^{ℓ_{blk}})^k, the hash algorithm sets α' = ⌈log₂ k⌉ and k' = 2^{α'}. Then, for each i ∈ [k'], it constructs ciphertexts as follows:
 - If i ≤ k, it homomorphically computes ct_i = LHE.Eval(pk₀, U(·, hinp_i), c_g) where U(·, hinp_i) is the universal circuit defined above.
 - If i > k, it deterministically sets ct_i ← LHE.Enc(pk₀, 0; 0^λ) using fixed randomness 0^λ.

The hash algorithm now constructs a Merkle tree MT of depth α':

- It associates leaf i with the value ct_i.
- It uses the following two-to-one hash function h_j to compute the values of the nodes at level j ∈ [0, α'] in the tree (recall that level 0 corresponds to the leaves):

$$h_j(\text{node}_1, \text{node}_2) := \text{LHE.Eval}(\text{pk}_j, f_{\text{node}_1, \text{node}_2, j}, c_j), \quad (3.3)$$

where

$$f_{\text{node}_1, \text{node}_2, j}(\text{sk}) := \begin{cases} 1 & \text{LHE.Dec}(\text{sk}, \text{node}_1) = 1 \vee \text{LHE.Dec}(\text{sk}, \text{node}_2) = 1 \\ 0 & \text{otherwise.} \end{cases}$$

By design, f_{node₁, node₂, j} can be computed by a circuit of depth d_j(λ) + 1.

Output dig = (root, k) where root is the value associated with the root of the Merkle tree MT.

- **ProveOpen**(hk, (hinp₁, ..., hinp_k), S): On input the hash key hk and a tuple of inputs (hinp₁, ..., hinp_k) ∈ ({0, 1}^{ℓ_{blk}})^k, the prove algorithm starts by computing the Merkle tree MT according to the specification of Hash(hk, (hinp₁, ..., hinp_k)). Let dvals to be a map consisting of the values associated with the nodes in dangling(S) in MT. Output π = dvals.
- **VerOpen**(hk, dig, S, {(i, hinp_i)}_{i∈S}, π): On input the hash key hk = (pk₀, ..., pk_α, c₁, ..., c_α, c_g), a digest dig = (root, k), a set S, values {(i, hinp_i)}_{i∈S}, and a proof π = dvals, the verification algorithm first checks that S ⊆ [k] and moreover that dvals corresponds to dangling(S) for a Merkle tree with k' = 2^{⌈log₂(k)⌉} leaves. If either check fails, it outputs 0. Otherwise, it computes ct_i = LHE.Eval(pk₀, U(·, hinp_i), c_g) for each i ∈ S. Then, using the values of {ct_i}_{i∈S} together with the values in dvals for the set dangling(S), the verification algorithm computes the root root' of the Merkle tree using the two-to-one hash function from the specification of Hash(hk, ·) (Eq. (3.3)). If root = root', it outputs 1. Otherwise, it outputs 0.

Correctness and security analysis. We now show that FBH from [Construction 3.5](#) is a secure function binding hash for $\mathcal{F}$. First, we note that completeness of [Construction 3.5](#) holds by construction since VerOpen and ProveOpen perform identical operations to compute the dig. As such, VerOpen will always accept on honestly generated proofs. We proceed to prove efficiency, computational function hiding, and statistical function binding in [Theorems 3.6, 3.7](#) and [3.10](#) below:

Theorem 3.6 (Efficiency). *Let p = p(·, ·) be a fixed polynomial. Assuming that LHE satisfies efficiency and compactness and d_j(λ) ≤ p(λ, log j) for all λ ∈ ℕ and j ∈ [0, ⌈log₂ n⌉], then [Construction 3.5](#) satisfies efficiency.*

Proof. We show each of the efficiency properties individually:

- **Setup efficiency:** First, we analyze the efficiency of Setup and SetupBinding. Note that Setup runs LHE.Setup(1^λ, 1^{d_j(λ)+1}) for each j ∈ [⌈log₂ n⌉]. As d_j(λ) ≤ p(λ, log j), this takes time poly(λ, log n). Computing ciphertexts c₁, ..., c_α then also takes time poly(λ, log n) by the efficiency of LHE. Computing c_g takes time poly(λ, s_g(λ), log n) as g_⊥ is of size poly(λ, s_g(λ)). So, Setup (and SetupBinding by the same argument) runs in total time poly(λ, s_g(λ), log n), as required.

- **Digest size:** Note that root is an encryption of a bit under LHE for public key $pk_{\alpha'}$ where $\alpha' = \lceil \log_2 k \rceil$. As $d_{\alpha'}(\lambda) \leq p(\lambda, \log \alpha')$, it follows by the efficiency and compactness of LHE that $|\text{root}| \in \text{poly}(\lambda, \log k)$. Also, $|k| \leq \alpha'$, so together $|\text{dig}| \in \text{poly}(\lambda, \log k)$.
- **Proof size:** The proof consists of at most $|S| \cdot \alpha'$ ciphertexts corresponding to the values of nodes in the Merkle tree. As argued for the digest size, each of the ciphertexts are of size $\text{poly}(\lambda, \log k)$, so in total the size of the proof $\pi \in |S| \cdot \text{poly}(\lambda, \log k)$, as required. $\square$

Theorem 3.7 (Computational Function Hiding). *Assuming LHE is semantically secure, then [Construction 3.5](#) satisfies computational function hiding.*

Proof. Suppose function hiding does not hold. Namely, there exists a stateful PPT algorithm A , a function $n \in \text{poly}(\lambda)$, and a polynomial q such that for infinitely many $\lambda \in \mathbb{N}$,

$$\Pr \left[\begin{array}{l} f \leftarrow A(1^\lambda) \\ \text{hk}_0 \leftarrow \text{Setup}(1^\lambda, n(\lambda)) \\ \text{hk}_1 \leftarrow \text{SetupBinding}(1^\lambda, n(\lambda), f) \\ b \xleftarrow{\mathbb{R}} \{0, 1\} \end{array} : f \in \mathcal{F}_\lambda \wedge A(\text{hk}_b) = b \right] \geq 1/2 + 1/q(\lambda).$$

We will reach a contradiction via a sequence of hybrids on how the hash key hk is computed in the above experiment. Specifically, for any block function g' and index $j' \in [0, \alpha]$, we define a new setup algorithm $\text{Setup}_{g',j'}(1^\lambda, n(\lambda))$ as follows:

- $\text{Setup}_{g',j'}(1^\lambda, n(\lambda)) \rightarrow \text{hk}_{g',j'}$: First, sample encryption keys $(pk_j, sk_j) \leftarrow \text{LHE.Setup}(1^\lambda, 1^{d_j(\lambda)+1})$ for each $j \in [0, \alpha]$ and compute $c_g \leftarrow \text{LHE.Enc}(pk_0, g')$ exactly as in $\text{SetupBinding}(1^\lambda, n(\lambda), f_{g'})$. For $j \in [1, j']$, compute $c_j \leftarrow \text{LHE.Enc}(pk_j, sk_{j-1})$ again as in Setup . However, for $j \in [j' + 1, \alpha]$, instead compute $c_j \leftarrow \text{LHE.Enc}(pk_j, 0^{|sk_{j-1}|})$. Output $\text{hk}_{g',j'} = (pk_0, \dots, pk_\alpha, c_1, \dots, c_\alpha, c_g)$.

In the following two claims, we argue that the distribution of hash keys $\text{hk}_{g',j'} \leftarrow \text{Setup}_{g',j'}(1^\lambda, n(\lambda))$ and $\text{hk}_{g',j'-1} \leftarrow \text{Setup}_{g',j'-1}(1^\lambda, n(\lambda))$ for each $j' \in [\alpha]$ and then argue indistinguishability of $\text{hk}_{g_0,0}$ and $\text{hk}_{g_1,0}$ for any two same-size block functions g_0, g_1 .

Claim 3.8. *For any block function g' and index $j' \in [\alpha]$, there exists a negligible function μ such that for all $\lambda \in \mathbb{N}$, it holds that*

$$|\Pr[A(\text{hk}_{g',j'}) = 1] - \Pr[A(\text{hk}_{g',j'-1}) = 1]| \leq \mu(\lambda).$$

Proof. This follows via a reduction to the semantic security of LHE. Namely, observe that the only difference between how $\text{hk}_{g',j'-1}$ and $\text{hk}_{g',j'}$ are generated is in the ciphertext $c_{j'}$. The ciphertext $c_{j'}$ is computed as $\text{LHE.Enc}(pk_{j'}, 0^{|sk_{j'-1}|})$ for $\text{hk}_{g',j'-1}$ and as $\text{LHE.Enc}(pk_{j'}, sk_{j'-1})$ for $\text{hk}_{g',j'}$. We use this observation to construct a stateful algorithm B that breaks the semantic security of LHE with depth parameter $d_{j'}(\lambda) + 1$ as follows:

1. Algorithm B first takes as input a uniformly sampled public key $\hat{pk}$ for the LHE scheme with depth parameter $d_{j'}(\lambda) + 1$ and set $pk_{j'} = \hat{pk}$. For each $j \neq j' \in [0, \alpha]$, B samples encryption keys $(pk_j, sk_j) \leftarrow \text{LHE.Setup}(1^\lambda, 1^{d_j(\lambda)+1})$. It then outputs $(\text{msg}_0, \text{msg}_1)$ where $\text{msg}_0 = sk_{j'-1}$ and $\text{msg}_1 = 0^{|sk_{j'-1}|}$.
2. Algorithm B receives as input a challenge ciphertext ct^* corresponding to msg_b for a random $b \xleftarrow{\mathbb{R}} \{0, 1\}$. Algorithm B sets $c_{j'} = \text{ct}^*$ and then computes “lower level” ciphertexts as $c_j \leftarrow \text{LHE.Enc}(pk_j, sk_{j-1})$ for $j \in [0, j' - 1]$ and “higher level” ciphertexts as $c_j \leftarrow \text{LHE.Enc}(pk_j, 0^{|sk_{j-1}|})$ for $j \in [j' + 1, \alpha]$ (relying on the fact that the size of $sk_{j'-1}$ is known from the description of $\text{LHE.Setup}(1^\lambda, 1^{d_{j'}(\lambda)+1})$). Algorithm B also samples $c_g \leftarrow \text{LHE.Enc}(pk_0, g')$. Algorithm B then sets $\text{hk}^* = (pk_0, \dots, pk_\alpha, c_1, \dots, c_\alpha, c_g)$ and outputs $b' \leftarrow A(\text{hk}^*)$.

By construction, if algorithm A is efficient, then so is B . It follows that B can succeed at distinguishing with at most negligible probability. However, B 's distinguishing advantage is exactly the distinguishing advantage of A on inputs $\text{hk}_{g',j'}$ and $\text{hk}_{g',j'-1}$ by construction. This is because whenever the challenge bit $b = 1$, hk^* is distributed identically to $\text{hk}_{g',j'}$ since $c_{j'}$ is an encryption of $sk_{j'-1}$, and whenever the challenge bit $b = 0$, hk is distributed identically to $\text{hk}_{g',j'-1}$ since $c_{j'}$ is an encryption of the all-zeroes string. Thus, the claim follows. $\square$

Claim 3.9. For any two same-size block functions g_0, g_1 , there exists a negligible function μ such that for all $\lambda \in \mathbb{N}$, it holds that

$$|\Pr[A(\text{hk}_{g_0,0}) = 1] - \Pr[A(\text{hk}_{g_1,0}) = 1]| \leq \mu(\lambda).$$

Proof. This follows by a nearly identical reduction to the semantic security of LHE as in Claim 3.8. The only difference is we indistinguishably replace the ciphertext c_g using the block functions g_0 and g_1 in the reduction. Note that all other “higher level” ciphertexts can be efficiently simulated as they are simply LHE encryptions of the all-zeroes string. $\square$

Now, to finish the proof of computational function hiding, we observe that hk_0 is identically distributed to $\text{hk}_{g_\perp, \alpha}$ and hk_1 is identically distributed to $\text{hk}_{g, \alpha}$. So, by considering the sequence of $2\alpha + 1 = 2\lceil \log_2 n(\lambda) \rceil + 1 \in \text{poly}(\lambda)$ indistinguishable hybrids

$$A(\text{hk}_0) \equiv A(\text{hk}_{g_\perp, \alpha}) \stackrel{c}{\approx} \dots \stackrel{c}{\approx} A(\text{hk}_{g_\perp, 0}) \stackrel{c}{\approx} A(\text{hk}_{g, 0}) \stackrel{c}{\approx} \dots \stackrel{c}{\approx} A(\text{hk}_{g, \alpha}) \equiv A(\text{hk}_1),$$

we reach a contradiction with our initial assumption, where we write $\mathcal{D}_0 \stackrel{c}{\approx} \mathcal{D}_1$ to denote that the distributions $\mathcal{D}_0$ and $\mathcal{D}_1$ are computationally indistinguishable. Namely, there cannot exist a pair of adjacent hybrids that A distinguishes with better than $1/((2\alpha + 1) \cdot q(\lambda))$ advantage for infinitely many $\lambda \in \mathbb{N}$ by Claims 3.8 and 3.9, so computational function hiding must hold. $\square$

Theorem 3.10 (Statistical Function Binding). Assuming LHE satisfies perfect correctness, then Construction 3.5 satisfies function binding.

Proof. Consider any stateful unbounded algorithm A and function $n \in \text{poly}(\lambda)$. We prove a stronger statement and show that *perfect* function binding holds (i.e., function binding holds for the negligible function $\text{negl}(\lambda) = 0$ for all $\lambda \in \mathbb{N}$). Let $f_g \in \mathcal{F}_\lambda$ be any initial function output by A , and let $\text{hk} \in \text{Supp}(\text{SetupBinding}(1^\lambda, n(\lambda), f_g))$. Let $(\text{hinp}_1, \dots, \text{hinp}_k)$ for $k \leq n(\lambda)$ be any set of hash inputs (that may depend on hk) and $(\text{root}, k) = \text{dig} = \text{Hash}(\text{hk}, (\text{hinp}_1, \dots, \text{hinp}_k))$. For any set $S \subseteq [k]$, set of inputs $\{(i, \text{hinp}_i^*)\}_{i \in S}$, and proof $\pi = \text{dvals}$, let $b = \text{VerOpen}(\text{hk}, \text{dig}, S, \{(i, \text{hinp}_i^*)\}_{i \in S}, \pi)$. We show that

$$\left(\nexists \{(j, \text{hinp}_j^*)\}_{j \in [k] \setminus S} : f_g(\text{hinp}_1, \dots, \text{hinp}_k) = f_g(\text{hinp}_1^*, \dots, \text{hinp}_k^*) \right) \Rightarrow b = 0, \quad (3.4)$$

which suffices to prove the claim. Specifically, we show the following invariant holds for all of the nodes at level $j \in [0, \alpha']$ in the tree:

- If $g(\text{hinp}_i) = 0$ for all $i \in [k]$, then all of the nodes at level j in the tree are encryptions of 0 under pk_j .
- Otherwise, if there exists some index $i \in [k]$ such that $g(\text{hinp}_i) = 1$, then there exists some node at level j that is an encryption of 1 under pk_j .

For the root node (at level α'), there is just a single node, so it is either an encryption of 1 if there exists $i \in [k]$ such that $g(\text{hinp}_i) = 1$ (i.e., $f_g(\text{hinp}_1, \dots, \text{hinp}_k) = 1$) and is an encryption of 0 otherwise (i.e., $f_g(\text{hinp}_1, \dots, \text{hinp}_k) = 0$). We now show the invariant inductively:

- **Base case:** For the base case $j = 0$, each leaf node $i \in [k]$ has value $\text{LHE.Eval}(\text{pk}_0, U(\cdot, \text{hinp}_i), c_g)$. By correctness of LHE, this is an encryption of $g(\text{hinp}_i)$. For indices $i \geq k$, each leaf node has value $\text{LHE.Eval}(\text{pk}_0, U(\cdot, \text{hinp}_i), c_\perp)$, which is an encryption of 0 by definition of $c_\perp$ and correctness of LHE. If $g(\text{hinp}_i) = 0$ for all $i \in [k]$, then every leaf node is an encryption of 0 under pk_0 . Otherwise, if $g(\text{hinp}_i) = 1$ for some $i \in [k]$, then the i^{th} leaf node is an encryption of 1 under pk_0 . The invariant holds.
- **Inductive step:** This follows by definition of h_j in Eq. (3.3). Specifically, suppose there exists a node that is an encryption of 1 under pk_j at level j . Then, this will result in an encryption of 1 under pk_{j+1} in the next level $j + 1$ (specifically, the *parent* node of any node that encrypts 1 in level $j + 1$ will also be an encryption of 1 by construction of Eq. (3.3)). Alternatively, if all of the nodes at level j are encryptions of 0 under pk_j , then the nodes at level $j + 1$ will be encryptions of 0 under pk_{j+1} .

To complete the argument, we now consider the following cases and show that the implication in Eq. (3.4) holds in each case:

- **Case 1a:** Suppose $f_g(\text{hinp}_1, \dots, \text{hinp}_k) = 1$ and $S = [k]$. In this case, the inputs $\text{hinp}_1^*, \dots, \text{hinp}_k^*$ are all fixed since $S = [k]$. To show Eq. (3.4) holds, it suffices to show that if $f_g(\text{hinp}_1^*, \dots, \text{hinp}_k^*) = 0$, then $b = 0$. Let root' be the root of the Merkle tree computed on inputs $(\text{hinp}_1^*, \dots, \text{hinp}_k^*)$. By our invariant above, root' is an encryption of 1 under $\text{pk}_{\alpha'}$. At the same time, root is an encryption of $0 = f_g(\text{hinp}_1, \dots, \text{hinp}_k)$ under $\text{pk}_{\alpha'}$. Perfect correctness of LHE now implies that $\text{root} \neq \text{root}'$, in which case VerOpen outputs $b = 0$.
- **Case 1b:** Suppose $f_g(\text{hinp}_1, \dots, \text{hinp}_k) = 1$ and $S \neq [k]$. Since $S \neq [k]$, we can take any hinp_j^* for $j \notin S$ such that $g(\text{hinp}_j^*) = 1$. Note that if such an hinp_j^* does not exist, then it cannot be the case that $f_g(\text{hinp}_1, \dots, \text{hinp}_k) = 1$. Thus, there always exists a set of inputs $\{(j, \text{hinp}_j^*)\}_{j \in S \setminus [k]}$ such that $f_g(\text{hinp}_1^*, \dots, \text{hinp}_k^*) = 1$. Eq. (3.4) now holds trivially as the hypothesis is always false in this case.
- **Case 2:** Suppose $f_g(\text{hinp}_1, \dots, \text{hinp}_k) = 0$. Suppose there exists $i \in S$ where $g(\text{hinp}_i^*) = 1$ for some $i \in S$; otherwise there always exists values hinp_j^* for $j \in [k] \setminus S$ that make $f_g(\text{hinp}_1^*, \dots, \text{hinp}_k^*) = 0$. Let root' be the root of the Merkle tree computed on an input that contains hinp_i^* . By our invariant above, this means that root' is an encryption of 1 under $\text{pk}_{\alpha'}$. However, root is an encryption of $0 = f_g(\text{hinp}_1, \dots, \text{hinp}_k)$ under $\text{pk}_{\alpha'}$. It again follows that $\text{root} \neq \text{root}'$ by perfect correctness of LHE, so VerOpen outputs $b = 0$, as required. $\square$

4 Flexible Broadcast Encryption

In this section, we introduce the notion of flexible broadcast encryption. As described in Section 1.1, a flexible broadcast encryption allows anyone to encrypt a message to an arbitrary set of public keys with a ciphertext whose size scales sublinearly with the size of the broadcast set. Unlike traditional broadcast encryption [FN93], there is no central authority that is responsible for issuing decryption keys to users. Instead, users generate their own public and secret keys, and the encryption algorithm takes in the list of public keys for the users in a broadcast set and outputs a single *short* ciphertext that can be decrypted by every member in the broadcast set. Note that this notion of broadcast encryption implies standard broadcast encryption with short ciphertexts and a long public key (by having a central broadcast authority generate the keys for each user and publishing all of them as part of the master public key). Flexible broadcast encryption also generalizes the notion of distributed broadcast encryption introduced by Boneh and Zhandry [BZ14]. The main difference is that in distributed broadcast encryption, users generate keys with respect to a specific index and one can broadcast to at most one key for each index. Flexible broadcast encryption imposes no requirement on the public keys to which one may broadcast. We now give the formal definition:

Definition 4.1 (Flexible Broadcast Encryption). Let $m \in \text{poly}(\lambda)$ and $\mathcal{M} = \{\mathcal{M}_\lambda\}_{\lambda \in \mathbb{N}}$ be a message space for messages of length m . A *flexible broadcast encryption scheme* FBE with message space $\mathcal{M}$ consists of polynomial-time algorithms (Setup, KeyGen, Enc, Dec) with the following syntax:

- $\text{Setup}(1^\lambda, n) \rightarrow \text{pp}$: A probabilistic algorithm that on input a security parameter λ and a bound on the size of the broadcast set n (in *binary*), outputs public parameters pp . We implicitly assume that pp contains 1^λ and n .
- $\text{KeyGen}(\text{pp}) \rightarrow (\text{pk}, \text{sk})$: A probabilistic algorithm that on input public parameters pp , outputs a public key pk and a secret key sk .
- $\text{Enc}(\text{pp}, \text{msg}, (\text{pk}_1, \dots, \text{pk}_k)) \rightarrow \text{ct}$: A probabilistic algorithm that on input public parameters pp , a message $\text{msg} \in \mathcal{M}_\lambda$, and an ordered sequence of k public keys $(\text{pk}_1, \dots, \text{pk}_k)$ for some $k \leq n$, outputs a ciphertext ct .⁷
- $\text{Dec}(\text{pp}, \text{ct}, (j, \text{sk}_j), (\text{pk}_1, \dots, \text{pk}_k)) \rightarrow \text{msg}$: A deterministic algorithm that on input public parameters pp , a ciphertext ct , an index and secret key pair (j, sk_j) , and public keys $(\text{pk}_1, \dots, \text{pk}_k)$ where $k \leq n$, outputs a message $\text{msg} \in \mathcal{M}_\lambda \cup \{\perp\}$.

⁷Here, we assume an ordered list of public keys for simplicity. However, we could have alternatively encrypted to an (unordered) set of public keys by first ordering the public keys in lexicographic order.

We require that $(\text{Setup}, \text{KeyGen}, \text{Enc}, \text{Dec})$ satisfy the following properties:

- **Succinct ciphertexts:** There exists a polynomial p such that for all $\lambda \in \mathbb{N}$, $k \in \mathbb{N}$, $n \in \mathbb{N}$ with $k \leq n$, public parameters $\text{pp} \in \text{Supp}(\text{Setup}(1^\lambda, n))$, public/secret keys $(\text{pk}_i, \text{sk}_i) \in \text{Supp}(\text{KeyGen}(\text{pp}))$ for $i \in [k]$, messages $\text{msg} \in \mathcal{M}_\lambda$, and ciphertexts $\text{ct} \in \text{Supp}(\text{Enc}(\text{pp}, \text{msg}, (\text{pk}_1, \dots, \text{pk}_k)))$, it holds that $|\text{ct}| \leq p(\lambda, m(\lambda), \log n)$.
- **Correctness:** There exists a negligible function negl such that for all $\lambda \in \mathbb{N}$, $n \in \text{poly}(\lambda)$, $k \leq n(\lambda)$, $j \in [k]$, and $\text{msg} \in \mathcal{M}_\lambda$, it holds that

$$\Pr \left[\begin{array}{l} \text{pp} \leftarrow \text{Setup}(1^\lambda, n(\lambda)) \\ \forall i \in [k], (\text{pk}_i, \text{sk}_i) \leftarrow \text{KeyGen}(\text{pp}) \\ \text{ct} \leftarrow \text{Enc}(\text{pp}, \text{msg}, (\text{pk}_1, \dots, \text{pk}_k)) \\ \text{msg}' = \text{Dec}(\text{pp}, \text{ct}, (j, \text{sk}_j), (\text{pk}_1, \dots, \text{pk}_k)) \end{array} : \text{msg}' = \text{msg} \right] \geq 1 - \text{negl}(\lambda).$$

We say that the scheme satisfies perfect correctness if the above probability is equal to 1.

- **Adaptive security:** For all stateful PPT adversaries A and all efficiently-computable functions $n \in \text{poly}(\lambda)$, there exists a negligible function negl such that for all $\lambda \in \mathbb{N}$,

$$\Pr \left[\text{Expt}_{A,n(\lambda)}^{\text{FBE}}(\lambda) = 1 \right] \leq 1/2 + \text{negl}(\lambda),$$

where $\text{Expt}_{A,n}^{\text{FBE}}(\lambda)$ is defined via the following security game between the adversary A and a challenger on common input 1^λ :

- **Setup phase:** The challenger samples $\text{pp} \leftarrow \text{Setup}(1^\lambda, n)$ and sends pp to A . The challenger also initializes a counter $\text{ctr} := 0$, a dictionary D , and a set of (corrupted) indices $C = \emptyset$.
- **Pre-challenge query phase:** The adversary A can now issue the following queries:
 - * **Key-generation query:** In a key-generation query, the challenger increments $\text{ctr} := \text{ctr} + 1$, samples a key-pair $(\text{pk}, \text{sk}) \leftarrow \text{KeyGen}(\text{pp})$, and replies to A with (ctr, pk) . The challenger adds the mapping $\text{ctr} \mapsto (\text{pk}, \text{sk})$ to the dictionary D .
 - * **Corruption query:** In a corruption query, the adversary specifies a counter value $c \in [\text{ctr}]$. In response, the challenger looks up $(\text{pk}, \text{sk}) := D[c]$, replies to A with sk , and adds ctr to C .
- **Challenge phase:** Algorithm A computes two messages $\text{msg}_0, \text{msg}_1 \in \mathcal{M}_\lambda$ and an ordered list $S^* = (i_1, \dots, i_{k^*}) \subseteq [\text{ctr}]$ that it sends to the challenger. If $S^* \cap C \neq \emptyset$, the experiment outputs 0. Otherwise, the challenger samples a bit $b \xleftarrow{\mathcal{R}} \{0, 1\}$ and computes an encryption of msg_b under the keys corresponding to S^* : $\text{ct}^* \leftarrow \text{Enc}(\text{pp}, \text{msg}_b, (\text{pk}_{i_1}, \dots, \text{pk}_{i_{k^*}}))$. The challenger sends ct^* to A .
- **Output phase:** Algorithm A outputs a bit $b' \in \{0, 1\}$. The experiment outputs 1 if $b' = b$ and 0 otherwise.

Semi-static security. In this work, we also consider a weaker notion of semi-static security introduced by Gentry and Waters [GW09] in the context of broadcast encryption. In the setting of flexible broadcast encryption, semi-static security corresponds to the setting where the adversary is allowed to make any number of honest key-generation queries, and then declares its challenge set to be some subset of these keys. The adversary cannot make any corruption queries in this model. In the context of broadcast encryption, Gentry and Waters showed that semi-static security implies adaptive security in the random oracle model. In [Appendix B](#), we show that the same technique also applies in the case of flexible broadcast encryption.

Definition 4.2 (Semi-Static Security [GW09, adapted]). We say that a flexible broadcast encryption scheme $(\text{Setup}, \text{KeyGen}, \text{Enc}, \text{Dec})$ satisfies semi-static security if instead of adaptive security, it satisfies the following:

- **Semi-static security:** For all stateful PPT adversaries A and all efficiently-computable functions $n \in \text{poly}(\lambda)$, there exists a negligible function negl such that for all $\lambda \in \mathbb{N}$,

$$\Pr \left[\text{Expt}_{A,n(\lambda)}^{\text{FBE,SS}}(\lambda) = 1 \right] \leq 1/2 + \text{negl}(\lambda),$$

where $\text{Expt}_{A,n}^{\text{FBE,SS}}(\lambda)$ is identical to the adaptive security game $\text{Expt}_{A,n}^{\text{FBE}}(\lambda)$, except the adversary A is not allowed to make any corruption queries.

4.1 Constructing Flexible Broadcast Encryption

We now show how to construct a flexible broadcast encryption scheme by combining a witness encryption scheme with a function-binding hash function and a vanilla public-key encryption scheme. Note that public-key encryption is implied by combining witness encryption and one-way functions [GGSW13]. Thus, the additional assumption of public-key encryption is technically unnecessary (since function-binding hash functions imply one-way functions). Our construction satisfies semi-static security. However, we show in [Appendix B](#) how to use this to construct an adaptively secure scheme via a transformation following the approach of [GW09].

Construction 4.3 (Flexible Broadcast Encryption). Let $s = s(\lambda)$, $\ell_{\text{blk}} = \ell_{\text{blk}}(\lambda)$, and $d = d(\lambda)$ be polynomials, and $m = m(\lambda)$ be any function. Let $\mathcal{M} = \{\mathcal{M}_\lambda\}_{\lambda \in \mathbb{N}}$ be a message space for messages of length m . Our construction of flexible broadcast encryption relies on the following primitives:

- Let $\text{PKE} = (\text{PKE.KeyGen}, \text{PKE.Enc}, \text{PKE.Dec})$ be a semantically-secure public-key bit encryption scheme where for $(\text{pk}, \text{sk}) \in \text{Supp}(\text{PKE.KeyGen}(1^\lambda))$, ciphertexts have length at most $\ell_{\text{blk}}(\lambda)$ and decryption can be computed by a circuit of size $s(\lambda)$ and depth $d(\lambda)$.
- Let $\text{FBH} = (\text{FBH.Setup}, \text{FBH.SetupBinding}, \text{FBH.Hash}, \text{FBH.ProveOpen}, \text{FBH.VerOpen})$ be a function-binding hash for the class $\mathcal{F}$ of disjunctions of block functions for input length ℓ_{blk} , size s , and depth d ([Definition 3.3](#)).
- Let $\text{WE} = (\text{WE.Enc}, \text{WE.Dec})$ be a witness encryption scheme for the language $\mathcal{L} = \{\mathcal{L}_\lambda\}_{\lambda \in \mathbb{N}}$ defined by the relation $\mathbf{R}_{\mathcal{L}}$ as follows. Instances of the language $\mathcal{L}_\lambda$ are of the form $(\text{hk}, \text{pk}_{\text{PKE}}, \text{dig})$ and the relation $\mathbf{R}_{\mathcal{L}}$ is given by

$$\begin{aligned} \mathbf{R}_{\mathcal{L}}(1^\lambda, (\text{hk}, \text{pk}_{\text{PKE}}, \text{dig}), (i, \text{hinp}_i, r, \pi)) &= 1 \\ \Leftrightarrow \text{hinp}_i &= \text{PKE.Enc}(\text{pk}_{\text{PKE}}, 1; r) \wedge \text{FBH.VerOpen}(\text{hk}, \text{dig}, \{i\}, \{(i, \text{hinp}_i)\}, \pi) = 1. \end{aligned}$$

We construct a flexible broadcast encryption scheme $\text{FBE} = (\text{Setup}, \text{KeyGen}, \text{Enc}, \text{Dec})$ with message space $\mathcal{M}$ as follows:

- $\text{Setup}(1^\lambda, n)$: On input the security parameter λ and a bound on the broadcast set size n , the setup algorithm samples $(\text{pk}_{\text{PKE}}, \text{sk}_{\text{PKE}}) \leftarrow \text{PKE.KeyGen}(1^\lambda)$, $\text{hk} \leftarrow \text{FBH.Setup}(1^\lambda, n)$, and outputs the public parameters $\text{pp} = (\text{pk}_{\text{PKE}}, \text{hk})$. Note that we implicitly assume that pp contain 1^λ and n .
- $\text{KeyGen}(\text{pp})$: On input the public parameters $\text{pp} = (\text{pk}_{\text{PKE}}, \text{hk})$, the key-generation algorithm samples $r \xleftarrow{\mathbb{R}} \{0, 1\}^\lambda$ and computes $\text{ct} = \text{PKE.Enc}(\text{pk}_{\text{PKE}}, 1; r)$. Output the public key $\text{pk} = \text{ct}$ and the secret key $\text{sk} = r$.
- $\text{Enc}(\text{pp}, \text{msg}, (\text{pk}_1, \dots, \text{pk}_k))$: On input the public parameters $\text{pp} = (\text{pk}_{\text{PKE}}, \text{hk})$, the message $\text{msg} \in \mathcal{M}_\lambda$, and public keys $\text{pk}_1, \dots, \text{pk}_k$, the encryption algorithm computes the digest $\text{dig} = \text{FBH.Hash}(\text{hk}, (\text{pk}_1, \dots, \text{pk}_k))$ and outputs the ciphertext $\text{ct} \leftarrow \text{WE.Enc}(1^\lambda, \text{msg}, (\text{hk}, \text{pk}_{\text{PKE}}, \text{dig}))$.
- $\text{Dec}(\text{pp}, \text{ct}, (j, \text{sk}_j), (\text{pk}_1, \dots, \text{pk}_k))$: On input the public parameters $\text{pp} = (\text{pk}_{\text{PKE}}, \text{hk})$, the ciphertext ct , an index $j \in [n]$, the associated secret key sk_j , and the public keys $\text{pk}_1, \dots, \text{pk}_k$, the decryption algorithm computes the digest $\text{dig} = \text{FBH.Hash}(\text{hk}, (\text{pk}_1, \dots, \text{pk}_k))$, the opening $\pi = \text{FBH.ProveOpen}(\text{hk}, (\text{pk}_1, \dots, \text{pk}_k), \{j\})$, and finally, outputs the message $\text{msg} = \text{WE.Dec}(1^\lambda, \text{ct}, (j, \text{pk}_j, \text{sk}_j, \pi))$.

Correctness and security analysis. We now show that FBE from [Construction 4.3](#) is a flexible broadcast encryption scheme.

Theorem 4.4 (Correctness and Succinctness). *Assuming PKE and WE are correct, and FBH is complete, then [Construction 4.3](#) has succinct ciphertexts and is correct.*

Proof. We consider each property separately:

- **Succinct ciphertexts:** This follows by efficiency of WE and FBH. Specifically, checking the relation $\mathbf{R}_{\mathcal{L}}$ requires time at most $\text{poly}(\lambda, s(\lambda), \log n)$ by the efficiency of FBH, which is $\text{poly}(\lambda, \log n)$ for polynomially-bounded s, ℓ_{blk} . Correspondingly, efficiency of WE implies that $|\text{ct}| \in \text{poly}(\lambda, m(\lambda), \log n)$, as required.

- **Perfect correctness:** We show perfect correctness of the scheme. Take any security parameter $\lambda \in \mathbb{N}$, bound on the number of participants $n \in \text{poly}(\lambda)$, $k \leq n(\lambda)$, index $j \in [k]$, and message $\text{msg} \in \mathcal{M}_\lambda$. Then, we have the following:

- Let $\text{pp} = (\text{pk}_{\text{PKE}}, \text{hk}) \leftarrow \text{Setup}(1^\lambda, n(\lambda))$, and for each $i \in [k]$, let $(\text{pk}_i, \text{sk}_i) \leftarrow \text{KeyGen}(\text{pp})$. By construction, this means $\text{pk}_i = \text{PKE.Enc}(\text{pk}_{\text{PKE}}, 1; r_i)$ and $\text{sk}_i = r_i$.
- Let $\text{ct} \leftarrow \text{Enc}(\text{pp}, \text{msg}, (\text{pk}_1, \dots, \text{pk}_k))$. Then, $\text{ct} \leftarrow \text{WE.Enc}(1^\lambda, \text{msg}, (\text{hk}, \text{pk}_{\text{PKE}}, \text{dig}))$ where the digest is $\text{dig} = \text{FBH.Hash}(\text{hk}, (\text{pk}_1, \dots, \text{pk}_k))$.
- We now argue that $\text{Dec}(\text{pp}, \text{ct}, (j, \text{sk}_j), (\text{pk}_1, \dots, \text{pk}_k)) = \text{msg}$. The decryption algorithm computes the digest dig as above and the proof $\pi = \text{FBH.ProveOpen}(\text{hk}, (\text{pk}_1, \dots, \text{pk}_k), \{j\})$. By completeness of FBH, we conclude $\text{FBH.VerOpen}(\text{hk}, \text{dig}, \{j\}, \{(j, \text{pk}_j)\}, \pi) = 1$. By construction $\text{pk}_j = \text{PKE.Enc}(\text{pk}_{\text{PKE}}, 1; \text{sk}_{\text{PKE}})$. This means that

$$\mathbf{R}_{\mathcal{L}}(1^\lambda, (\text{hk}, \text{pk}_{\text{PKE}}, \text{dig}), (j, \text{pk}_j, \text{sk}_j, \pi)) = 1,$$

and correctness now follows by correctness of WE. $\square$

Theorem 4.5 (Semi-Static Security). *Assuming PKE is semantically secure and satisfies perfect correctness, FBH satisfies computational function hiding and statistical function binding, and WE satisfies message indistinguishability, then Construction 4.3 satisfies semi-static security.*

Proof. Suppose by way of contradiction that FBE does not satisfy semi-static security. Namely, there exists a stateful PPT algorithm A , an efficiently-computable function $n \in \text{poly}(\lambda)$, a polynomial q , and an infinite set $\Lambda \subseteq \mathbb{N}$ such that for all $\lambda \in \Lambda$, it holds that

$$\Pr \left[\text{Expt}_{A, n(\lambda)}^{\text{FBE, SS}}(\lambda) = 1 \right] \geq 1/2 + 1/q(\lambda).$$

For fixed parameters as above, we define the following sequence of hybrid experiments for each $\lambda \in \mathbb{N}$.

- $H_{b,0}(\lambda)$: This hybrid is the experiment $\text{Expt}_{A, n(\lambda)}^{\text{FBE, SS}}(\lambda)$ where the challenge bit is fixed to $b \in \{0, 1\}$.
- $H_{b,1}(\lambda)$: This hybrid experiment is the same as $H_{b,0}$ except the challenger computes pk to be an encryption of 0 under pk_{PKE} for each honest key-generation query. Specifically, the challenger uses $(\text{pk}, \text{sk}) = (\text{ct}, r)$ where $r \leftarrow \{0, 1\}^\lambda$ and $\text{ct} = \text{PKE.Enc}(\text{pk}_{\text{PKE}}, 0; r)$.
- $H_{b,2}(\lambda)$: In this hybrid, the challenger instead computes $\text{hk} \leftarrow \text{FBH.SetupBinding}(1^\lambda, n(\lambda), f_g)$, where f_g is the function from Eq. (3.1) and g is the function computing $\text{PKE.Dec}(\text{sk}_{\text{PKE}}, \cdot)$.
- $H_{b,3}(\lambda)$: In this hybrid, instead of encrypting msg_b in the semi-static security game, the challenger encrypts the lexicographically-first message msg^* in $\mathcal{M}_\lambda$: $\text{ct}^* \leftarrow \text{Enc}(\text{pp}, \text{msg}^*, (\text{pk}_{i_1}, \dots, \text{pk}_{i_k}))$.

Since the challenge bit is sampled uniformly (i.e., $b \xleftarrow{\mathbb{R}} \{0, 1\}$), the success probability of A on any security parameter λ is equal to

$$\Pr[A \text{ wins}] = \frac{1}{2} \cdot (\Pr[H_{0,0}(\lambda) = 1] + \Pr[H_{1,0}(\lambda) = 1]).$$

For each $b \in \{0, 1\}$ and $i \in \{1, 2, 3\}$, we define

$$\delta_{b,i}(\lambda) = \Pr[H_{b,i-1}(\lambda) = 1] - \Pr[H_{b,i}(\lambda) = 1].$$

It follows by our assumption that for all $\lambda \in \Lambda$,

$$\begin{aligned} \frac{1}{2} + \frac{1}{q(\lambda)} &\leq \frac{1}{2} \cdot (\Pr[H_{0,0}(\lambda) = 1] + \Pr[H_{1,0}(\lambda) = 1]) \\ &= \frac{1}{2} \cdot (\Pr[H_{0,3}(\lambda) = 1] + \delta_{0,1}(\lambda) + \delta_{0,2}(\lambda) + \delta_{0,3}(\lambda) \\ &\quad + \Pr[H_{1,3}(\lambda) = 1] + \delta_{1,1}(\lambda) + \delta_{1,2}(\lambda) + \delta_{1,3}(\lambda)). \end{aligned}$$

As $H_{b,3}(\lambda)$ is independent of b for each $b \in \{0, 1\}$, it holds that $\Pr[H_{b,3}(\lambda) = 1] \leq 1/2$. It follows that for all $\lambda \in \Lambda$,

$$\frac{1}{q(\lambda)} \leq \frac{1}{2} \cdot (\delta_{0,1}(\lambda) + \delta_{0,2}(\lambda) + \delta_{0,3}(\lambda) + \delta_{1,1}(\lambda) + \delta_{1,2}(\lambda) + \delta_{1,3}(\lambda)).$$

As Λ is an infinitely large subset of $\mathbb{N}$, it must be the case that for some $b \in \{0, 1\}$ and $i \in \{1, 2, 3\}$, $\delta_{b,i}(\lambda) \geq 1/(3q(\lambda))$ for infinitely many $\lambda \in \mathbb{N}$. In the following three claims, we show that this must contradict one of our assumptions from the statement of [Theorem 4.5](#). We prove the following claims for $b = 0$ without loss of generality.

Claim 4.6. *If $\delta_{0,1}(\lambda) \geq 1/(3q(\lambda))$ for infinitely many $\lambda \in \mathbb{N}$, then PKE does not satisfy semantic security.*

Proof. Let $Q(\lambda)$ be a polynomial upper bounding the number of honest key-generation queries that A makes, which must exist as A is PPT. We construct an adversary B that uses A to break the semantic security of PKE. We define B as follows:

1. Algorithm B first takes as input a public key $\widehat{\text{pk}}$ for the PKE scheme and outputs $\text{msg}_0 = 0$ and $\text{msg}_1 = 1$ as its challenge message.
2. Algorithm B then receives a challenge ciphertext $\widehat{\text{ct}}$, corresponding to the message $\hat{b}$ which is either 0 or 1. It computes the guess $\hat{b}'$ for $\hat{b}$ as follows:
 - (a) Algorithm B computes $Q(\lambda)$ and samples a uniformly random index $i^* \leftarrow [n(\lambda)]$.
 - (b) Algorithm B then samples $\text{hk} \leftarrow \text{FBH.Setup}(1^\lambda, n(\lambda))$, sets $\text{pp} = (\widehat{\text{pk}}, \text{hk})$, and sends pp to A .
 - (c) For each counter value $\text{ctr} \in [Q(\lambda)]$, algorithm B computes the response pk to A 's honest key-generation for ctr as follows:
 - If $\text{ctr} = i^*$, B sets $\text{pk} = \widehat{\text{ct}}$.
 - If $\text{ctr} > i^*$, B samples $\text{sk} \xleftarrow{\mathbb{R}} \{0, 1\}^\lambda$ and sets $\text{pk} = \text{PKE.Enc}(\widehat{\text{pk}}, 1; \text{sk})$.
 - If $\text{ctr} < i^*$, B samples $\text{sk} \xleftarrow{\mathbb{R}} \{0, 1\}^\lambda$ and sets $\text{pk} = \text{PKE.Enc}(\widehat{\text{pk}}, 0; \text{sk})$.
 - (d) Algorithm B then receives the challenge messages $\text{msg}_0, \text{msg}_1$ and challenge set S^* from A , computes $\text{ct}^* \leftarrow \text{Enc}(\text{pp}, \text{msg}_0, (\text{pk}_{i_1}, \dots, \text{pk}_{i_{k^*}}))$ where $S^* = \{i_1, \dots, i_{k^*}\}$, and sends ct^* to A .
 - (e) The adversary A outputs a bit $b' \in \{0, 1\}$, which B outputs.

If A is PPT, then so is algorithm B by construction. It remains to analyze the success probability of B . To do so, we define a sequence of sub-hybrids. For each $i \in [0, Q(\lambda)]$, we define $H_{0,0,i}(\lambda)$ as follows:

- $H_{0,0,i}(\lambda)$: This experiment is the same as $H_{0,0}(\lambda)$ except that we sample the public keys for $j \leq k$ where $j \leq i$ to be encryptions of 0. Namely, we set $\text{pk}_j = \text{PKE.Enc}(\text{pk}_{\text{PKE}}, 0; r)$ for a random $r \leftarrow \{0, 1\}^\lambda$ as in $H_{0,1}(\lambda)$.

Note that $H_{0,0,0}(\lambda)$ is identical to $H_{0,0}(\lambda)$ and $H_{0,0,Q(\lambda)}(\lambda)$ is identical to $H_{0,1}(\lambda)$, so in particular, it holds that

$$\begin{aligned} \delta_{0,1}(\lambda) &= \Pr[H_{0,0}(\lambda) = 1] - \Pr[H_{0,1}(\lambda) = 1] \\ &= \sum_{i=1}^{Q(\lambda)} (\Pr[H_{0,0,i-1}(\lambda) = 1] - \Pr[H_{0,0,i}(\lambda) = 1]). \end{aligned}$$

We proceed to analyze the success probability of B conditioned on $i^* = i$. Note that when $\hat{b} = 1$, the view of A corresponds to the hybrid $H_{0,0,i}$ since pk_{i^*} is an encryption of 0. If $\hat{b} = 0$, this corresponds to the hybrid $H_{0,0,i-1}$. Since B outputs $\hat{b}' = b'$, it follows if $\hat{b} = 1$, then $\hat{b}' = \hat{b}$ whenever $b' = 1$ so $H_{0,0,i}(\lambda) = 0$ (i.e. A loses since b is fixed to 0). Similarly, if $\hat{b} = 0$, then $\hat{b}' = \hat{b}$ whenever $b' = 0$ so $H_{0,0,i-1}(\lambda) = 1$ (i.e. A wins). Thus, the success probability of B for any $i \in [Q(\lambda)]$ conditioned on $i^* = i$ is given by the following:

$$\begin{aligned} \Pr[\hat{b}' = \hat{b} \mid i^* = i] &= \frac{1}{2} \cdot (1 - \Pr[H_{0,0,i}(\lambda) = 1]) + \frac{1}{2} \cdot \Pr[H_{0,0,i-1}(\lambda) = 1] \\ &= \frac{1}{2} + \frac{1}{2} \cdot (\Pr[H_{0,0,i-1}(\lambda) = 1] - \Pr[H_{0,0,i}(\lambda) = 1]). \end{aligned}$$

It follows that the success probability of B , where B chooses a random i^* , is equal to

$$\begin{aligned}
\Pr[\hat{b}' = \hat{b}] &= \sum_{i=1}^{Q(\lambda)} \Pr[i^* = i] \cdot \Pr[\hat{b}' = \hat{b} \mid i^* = i] \\
&= \sum_{i=1}^{Q(\lambda)} \frac{1}{Q(\lambda)} \cdot \left(\frac{1}{2} + \frac{1}{2} \cdot (\Pr[H_{0,0,i-1}(\lambda) = 1] - \Pr[H_{0,0,i}(\lambda) = 1]) \right) \\
&= \frac{1}{2} + \frac{1}{2 \cdot Q(\lambda)} \cdot \sum_{i=1}^{Q(\lambda)} (\Pr[H_{0,0,i-1}(\lambda) = 1] - \Pr[H_{0,0,i}(\lambda) = 1]) \\
&= \frac{1}{2} + \frac{1}{2 \cdot Q(\lambda)} \cdot \delta_{0,1}(\lambda).
\end{aligned}$$

As $Q \in \text{poly}(\lambda)$ and $\delta_{0,1}(\lambda) \geq 1/(3q(\lambda))$ for infinitely many $\lambda \in \mathbb{N}$ by assumption, it follows that B succeeds with inverse polynomial probability for infinitely many $\lambda \in \mathbb{N}$. This violates semantic security of PKE, as required. $\square$

Claim 4.7. *If $\delta_{0,2}(\lambda) \geq 1/(3q(\lambda))$ for infinitely many $\lambda \in \mathbb{N}$, then FBH does not satisfy computational function hiding.*

Proof. We construct a stateful PPT adversary B that uses the adversary A to break the function hiding security of FBH. We define B as follows:

1. On input the security parameter λ , algorithm B samples $(\text{pk}_{\text{PKE}}, \text{sk}_{\text{PKE}}) \leftarrow \text{PKE.KeyGen}(1^\lambda)$ and outputs the function f_g from Eq. (3.1) where g is the function computing $\text{Dec}(\text{sk}_{\text{PKE}}, \cdot)$. Note that $f_g \in \mathcal{F}$ since g can be computed by a size $s(\lambda)$ circuit by assumption.
2. Algorithm B receives $\text{hk}_{\hat{b}}$ as input for some $\hat{b} \in \{0, 1\}$ and computes $\text{pp} = (\text{pk}_{\text{PKE}}, \text{hk})$. It then proceeds as in the experiment $H_{0,1}(\lambda)$, setting $\text{hk} = \text{hk}_{\hat{b}}$, until A outputs a bit b' , which B outputs.

Note that B only runs A and otherwise performs polynomial-time computation, so it remains to analyze the success probability of B at distinguishing $\text{hk}_0 \leftarrow \text{Setup}(1^\lambda, n(\lambda))$ versus $\text{hk}_1 \leftarrow \text{SetupBinding}(1^\lambda, n(\lambda), f_g)$. When hk is sampled from Setup (the case where $\hat{b} = 0$), algorithm A 's view is distributed identically to hybrid $H_{0,1}(\lambda)$, so $\hat{b} = b'$ whenever A outputs $b' = 0$, which corresponds to the event that A wins since b is fixed to 0. Therefore, B wins in this case with probability $\Pr[H_{0,1}(\lambda) = 1]$. Similarly, when hk is sampled from SetupBinding (the case where $\hat{b} = 1$), algorithm A 's view is identically distributed to hybrid $H_{0,2}(\lambda)$, so $\hat{b} = b'$ whenever A loses, which happens with probability $(1 - \Pr[H_{0,2}(\lambda) = 1])$. It follows that

$$\begin{aligned}
\Pr \left[\begin{array}{l} f_g \leftarrow B(1^\lambda) \\ \text{hk}_0 \leftarrow \text{Setup}(1^\lambda, n(\lambda)) \\ \text{hk}_1 \leftarrow \text{SetupBinding}(1^\lambda, n(\lambda), f_g) \\ b \xleftarrow{\mathcal{R}} \{0, 1\} \end{array} : f_g \in \mathcal{F}_\lambda \wedge B(\text{hk}_b) = b \right] \\
\geq \frac{1}{2} \cdot \Pr[H_{0,1}(\lambda) = 1] + \frac{1}{2} \cdot (1 - \Pr[H_{0,2}(\lambda) = 1]) \\
= 1/2 + \delta_{0,2}(\lambda)/2.
\end{aligned}$$

As $\delta_{0,2}(\lambda) \geq 1/(3q(\lambda))$ for infinitely many $\lambda \in \mathbb{N}$, this violates computational function hiding of FBH, as required. $\square$

Claim 4.8. *If $\delta_{0,3}(\lambda) \geq 1/(3q(\lambda))$ for infinitely many $\lambda \in \mathbb{N}$ and PKE satisfies (perfect) correctness, then either WE does not satisfy message indistinguishability or FBH does not satisfy statistical function binding.*

Proof. We construct a PPT adversary B that uses adversary A to either break the message indistinguishability security of WE, or the statistical function binding of FBH. We define B as follows:

1. Algorithm B takes 1^λ as input and uses A to compute $(x, \text{msg}_0, \text{msg}_1)$ to send to the WE challenger as follows:

- (a) Algorithm B samples $(pk_{\text{PKE}}, sk_{\text{PKE}}) \leftarrow \text{PKE.KeyGen}(1^\lambda)$ and $hk \leftarrow \text{FBH.SetupBinding}(1^\lambda, n(\lambda), f_g)$. It gives $pp = (pk_{\text{PKE}}, hk)$ to A .
 - (b) Whenever A makes an honest key-generation query for counter value $\text{ctr} \in \mathbb{N}$, algorithm B samples $sk \xleftarrow{\mathbb{R}} \{0, 1\}^\lambda$, sets $pk = \text{PKE.Enc}(pk_{\text{PKE}}, 0; sk)$, and returns pk .
 - (c) In the challenge phase, algorithm A then outputs two messages $\text{msg}'_0, \text{msg}'_1$ along with an ordered list $S^* = (i_1, \dots, i_{k^*})$. Algorithm B computes the digest $\text{dig} = \text{FBH.Hash}(hk, (pk_{i_1}, \dots, pk_{i_{k^*}}))$ as in Enc .
 - (d) Algorithm B sets $x = (hk, pk_{\text{PKE}}, \text{dig})$, $\text{msg}_0 = \text{msg}'_0$, and msg_1 is the lexicographically first message msg^* in $\mathcal{M}_\lambda$.
2. The WE challenger samples a random bit $\hat{b} \xleftarrow{\mathbb{R}} \{0, 1\}$, and algorithm B receives a challenge ciphertext ct^* corresponding to $\text{WE.Enc}(1^\lambda, \text{msg}_{\hat{b}}, x)$. Algorithm B uses ct^* as the challenge ciphertext for A . Let $b' \in \{0, 1\}$ be the final output of A in the experiment, which B outputs.

Note that B runs A and otherwise performs polynomial-time operations, so B is PPT. It remains to analyze the success probability of B for guessing $\hat{b}' = b'$ such that $\hat{b}' = \hat{b}$. For any $\lambda \in \mathbb{N}$, let $\text{Wit}(\lambda)$ be the event that there exists a valid witness $(j, \text{hinp}_j, r, \pi)$ for the instance $x = (hk, pk_{\text{PKE}}, \text{dig})$ such that

$$\text{hinp}_j = \text{PKE.Enc}(pk_{\text{PKE}}, 1; r) \wedge \text{FBH.VerOpen}(hk, \text{dig}, \{j\}, \{(j, \text{hinp}_j)\}, \pi) = 1. \quad (4.1)$$

Note that the view of A is identical to hybrid $H_{0,2}(\lambda)$ if $\hat{b} = 0$ and is identical to hybrid $H_{0,3}(\lambda)$ if $\hat{b} = 1$ since $\text{msg}_1 = \text{msg}^*$. Furthermore, when $\hat{b} = 0$, $\hat{b}' = \hat{b}$ whenever $b' = 0$, so $H_{0,2}(\lambda) = 1$ since $b = 0$. When $\hat{b} = 1$, $\hat{b}' = \hat{b}$ whenever $b' = 1$, so $H_{0,3}(\lambda) = 0$ since $b = 0$. Additionally, if $\text{Wit}(\lambda)$ holds, then B always loses the message indistinguishability game since it implies that $x \in \mathcal{L}_\lambda$. Thus, the success probability of B is at least

$$\begin{aligned} \Pr[B \text{ wins}] &\geq \Pr[\hat{b}' = \hat{b}] - \Pr[\text{Wit}(\lambda)] \\ &\geq \frac{1}{2} \cdot \Pr[H_{0,2}(\lambda) = 1] + \frac{1}{2} \cdot (1 - \Pr[H_{0,3}(\lambda) = 1]) - \Pr[\text{Wit}(\lambda)] \\ &= \frac{1}{2} + \frac{1}{2} \cdot \delta_{0,3}(\lambda) - \Pr[\text{Wit}(\lambda)]. \end{aligned}$$

We now consider two cases depending on the value of $\Pr[\text{Wit}(\lambda)]$:

- If $\Pr[\text{Wit}(\lambda)] \leq 1/(12q(\lambda))$ for infinitely many $\lambda \in \Lambda$, then this violates message indistinguishability of WE since $\delta_{0,3}(\lambda) \geq 1/(3q(\lambda))$.
- Suppose $\Pr[\text{Wit}(\lambda)] > 1/(12q(\lambda))$ for infinitely many $\lambda \in \mathbb{N}$. This means [Eq. \(4.1\)](#) occurs with probability at least $1/(12q(\lambda))$ for infinitely many $\lambda \in \mathbb{N}$. In the above construction, $\text{dig} = \text{FBH.Hash}(hk, (pk_{i_1}, \dots, pk_{i_{k^*}}))$ where

$$g(pk_{i_j}) = \text{PKE.Dec}(sk_{\text{PKE}}, pk_{i_j}) = 0,$$

by perfect correctness of PKE. Thus,

$$f_g(pk_{i_1}, \dots, pk_{i_{k^*}}) = \bigvee_{j \in [k^*]} g(pk_{i_j}) = 0.$$

Next, if $\text{hinp}_j = \text{PKE.Enc}(pk_{\text{PKE}}, 1; r)$ for some randomness $r \in \{0, 1\}^\lambda$, then $g(\text{hinp}_j) = 1$, and the output of f_g on *any* input that contains hinp_j is necessarily $1 \neq f_g(pk_{i_1}, \dots, pk_{i_{k^*}})$. As such, if such an input hinp_j exists such that [Eq. \(4.1\)](#) holds with probability at least $1/(12q(\lambda))$, then a computationally unbounded adversary would break statistical function binding of FBH with the same advantage.

Thus, algorithm B either breaks message indistinguishability of the witness encryption scheme or statistical function binding of the function-binding hash. $\square$

The proof now follows from [Claims 4.6 to 4.8](#). $\square$

Remark 4.9 (Transparent Setup). We note that the public parameters in our construction consist of a hash key for the function-binding hash and a public key for a public-key encryption scheme. If we instantiate the function-binding hash function using [Construction 3.5](#), then the hash key consists of a sequence of public keys and ciphertexts for a leveled homomorphic encryption scheme. Using a suitable encryption scheme (e.g., [\[GSW13\]](#)), these are all pseudorandom. In other words, the public parameters in [Construction 4.3](#) is pseudorandom and can be instantiated with a uniform random string. This yields a flexible broadcast encryption scheme with a *transparent* setup.

5 Optimal Broadcast Encryption in the Random Oracle Model

As noted in [Section 4](#), a flexible broadcast encryption scheme immediately implies a traditional broadcast encryption scheme with a central trusted authority. Namely, to construct a scheme for n users, the central authority would sample n different public/secret keys $(pk_1, sk_1), \dots, (pk_n, sk_n)$. The master public key for the broadcast encryption scheme is the concatenation of all n public keys while the secret key for user i is sk_i . While this yields a scheme with short secret keys and ciphertexts, the master public key is very long (scales linearly with the number of users). In this section, we show a simple adaptation of our approach yields an *optimal* broadcast encryption scheme in the *random oracle mode*. In an optimal broadcast encryption scheme [\[BWZ14\]](#), we require that all of the scheme parameters (i.e., the master public key, the user decryption keys, and the ciphertext) to be short (e.g., polylogarithmic with the number of users). We start by defining broadcast encryption:

Definition 5.1 (Broadcast Encryption). Let $m \in \text{poly}(\lambda)$ and $\mathcal{M} = \{\mathcal{M}_\lambda\}_{\lambda \in \mathbb{N}}$ be a message space for messages of length m . A *broadcast encryption* scheme with message space $\mathcal{M}$ consists of polynomial-time algorithms (Setup, KeyGen, Enc, Dec) with the following syntax:

- $\text{Setup}(1^\lambda, 1^n) \rightarrow (\text{pp}, \text{msk})$: A probabilistic algorithm that on input a security parameter λ and a bound on the number of users n , outputs public parameters pp and a master secret key msk . We implicitly assume that pp and msk contain 1^λ and 1^n .
- $\text{KeyGen}(\text{msk}, i) \rightarrow \text{sk}$: A probabilistic algorithm that on input a master secret key msk and an index $i \in [n]$, outputs a secret key sk .
- $\text{Enc}(\text{pp}, \text{msg}, S) \rightarrow \text{ct}$: A probabilistic algorithm that on input public parameters pp , a message $\text{msg} \in \mathcal{M}_\lambda$, and a set $S \subseteq [n]$, outputs a ciphertext ct .
- $\text{Dec}(\text{pp}, \text{ct}, (j, \text{sk}_j), S) \rightarrow \text{msg}$: A deterministic algorithm that on input public parameters pp , a ciphertext ct , an index and secret key pair (j, sk_j) , and a set S , outputs a message $\text{msg} \in \mathcal{M}_\lambda \cup \{\perp\}$.

We require that (Setup, KeyGen, Enc, Dec) satisfy the following properties:

- **Succinct ciphertexts:** There exists a polynomial p such that for all $\lambda \in \mathbb{N}$, $n \in \mathbb{N}$, $S \subseteq [n]$, $(\text{pp}, \text{msk}) \in \text{Supp}(\text{Setup}(1^\lambda, n))$, $\text{sk}_i \in \text{Supp}(\text{KeyGen}(\text{msk}, i))$ for all $i \in S$, $\text{msg} \in \mathcal{M}_\lambda$, and $\text{ct} \in \text{Supp}(\text{Enc}(\text{pp}, \text{msg}, S))$, it holds that

$$|\text{ct}| \leq p(\lambda, m(\lambda), \log n).$$

- **Correctness:** There exists a negligible function negl such that for all $\lambda \in \mathbb{N}$, $n \in \text{poly}(\lambda)$, $S \subseteq [n(\lambda)]$, $j \in S$, and $\text{msg} \in \mathcal{M}_\lambda$, it holds that

$$\Pr \left[\begin{array}{l} (\text{pp}, \text{msk}) \leftarrow \text{Setup}(1^\lambda, n(\lambda)) \\ \text{sk}_j \leftarrow \text{KeyGen}(\text{msk}, j) \\ \text{ct} \leftarrow \text{Enc}(\text{pp}, \text{msg}, S) \\ \text{msg}' = \text{Dec}(\text{pp}, \text{ct}, (j, \text{sk}_j), S) \end{array} : \text{msg}' = \text{msg} \right] \geq 1 - \text{negl}(\lambda).$$

We say that the scheme satisfies perfect correctness if the above probability is equal to 1.

- **Adaptive security:** For all stateful PPT adversaries A and efficiently-computable functions $n \in \text{poly}(\lambda)$, there exists a negligible function negl such that for all $\lambda \in \mathbb{N}$,

$$\Pr \left[\text{Expt}_{A,n(\lambda)}^{\text{BE}}(\lambda) = 1 \right] \leq 1/2 + \text{negl}(\lambda),$$

where $\text{Expt}_{A,n}^{\text{BE}}(\lambda)$ is defined via the following security game between the adversary A and a challenger on common input 1^λ :

- **Setup phase:** The challenger samples $(\text{pp}, \text{msk}) \leftarrow \text{Setup}(1^\lambda, n)$ and sends pp to A . The challenger also initializes a set of corrupted indices $C := \emptyset$.
- **Pre-challenge query phase:** The adversary A can now issue key-corruption queries for indices $i \in [n]$. On each query, the challenger responds to A with $\text{sk}_i \leftarrow \text{KeyGen}(\text{msk}, i)$ and adds i to C . Without loss of generality, we assume the adversary queries each index i at most once.
- **Challenge phase:** Algorithm A outputs two messages $\text{msg}_0, \text{msg}_1 \in \mathcal{M}_\lambda$ and a set $S^* \subseteq [n]$. If $S^* \cap C \neq \emptyset$, then the experiment halts with output 0. Otherwise, the challenger samples a bit $b \xleftarrow{\mathcal{R}} \{0, 1\}$ and gives the challenge ciphertext $\text{ct}^* \leftarrow \text{Enc}(\text{pp}, \text{msg}_b, S^*)$ to A .
- **Output phase:** Algorithm A outputs a bit $b' \in \{0, 1\}$. The output of the experiment is 1 if $b' = b$ and 0 otherwise.

Semi-static security. Similar to the case with flexible broadcast encryption (c.f., [Section 4](#) and [Definition 4.2](#)), we also consider the weaker notion of semi-static security from [\[GW09\]](#). A broadcast encryption scheme satisfies semi-static security if the adversary has to pre-commit to a set $S \subseteq [n]$ at the beginning of the security game. The adversary can then make adaptive key-corruption queries for indices $i \notin S$; for the challenge ciphertext, the adversary is allowed to choose any subset $S^* \subseteq S$. This is a *stronger* security model than selective security where the adversary is required to commit to its challenge set S^* at the beginning of the security game. In the semi-static model, the adversary only needs to commit to a *super-set* of its challenge set. Gentry and Waters previously showed that in the random oracle model, a broadcast encryption scheme satisfying semi-static security implies one that satisfies adaptive security. The transformation only incurs a constant factor overhead in the scheme parameters. We now recall the formal definition:

Definition 5.2 (Semi-Static Security [\[GW09\]](#)). We say that a broadcast encryption scheme $(\text{Setup}, \text{KeyGen}, \text{Enc}, \text{Dec})$ satisfies semi-static security if for all stateful PPT adversaries A and efficiently-computable functions $n \in \text{poly}(\lambda)$, there exists a negligible function negl such that for all $\lambda \in \mathbb{N}$,

$$\Pr \left[\text{Expt}_{A,n(\lambda)}^{\text{BE,SS}}(\lambda) = 1 \right] \leq 1/2 + \text{negl}(\lambda),$$

where $\text{Expt}_{A,n}^{\text{BE,SS}}(\lambda)$ is defined via the following security game between the adversary A and a challenger on common input 1^λ :

- **Commit phase:** Algorithm A begins by committing to a set of indices $S \subseteq [n]$.
- **Setup phase:** The challenger samples $(\text{pp}, \text{msk}) \leftarrow \text{Setup}(1^\lambda, n)$ and sends pp to A .
- **Pre-challenge query phase:** The adversary can now make adaptive key queries for indices $i \in [n] \setminus S$. On each query $i \in [n] \setminus S$, the challenger responds with the key $\text{sk}_i \leftarrow \text{KeyGen}(\text{msk}, i)$. Without loss of generality, we assume the adversary queries each index i at most once.
- **Challenge phase:** Algorithm A now outputs two messages $\text{msg}_0, \text{msg}_1 \in \mathcal{M}_\lambda$ and a set $S^* \subseteq S$. The challenger samples a bit $b \xleftarrow{\mathcal{R}} \{0, 1\}$ replies to A with the ciphertext $\text{ct}^* \leftarrow \text{Enc}(\text{pp}, \text{msg}_b, S^*)$.
- **Output phase:** Algorithm A outputs a bit $b' \in \{0, 1\}$. The experiment outputs 1 if $b' = b$ and outputs 0 otherwise.

Optimal broadcast encryption. In an optimal broadcast encryption scheme, we additionally require that all of the scheme parameters (i.e., the public parameters pp and the decryption keys sk) be short. We give the formal definition below:

Definition 5.3 (Optimal Broadcast Encryption). A broadcast encryption scheme $(\text{Setup}, \text{KeyGen}, \text{Enc}, \text{Dec})$ satisfies optimal succinctness if it satisfies the following property (in addition to the properties in [Definition 5.1](#)):

- **Succinct public key and secret keys:** There exists a polynomial p such that for all $\lambda \in \mathbb{N}$, $n \in \mathbb{N}$, $i \in [n]$, $(pp, msk) \in \text{Supp}(\text{Setup}(1^\lambda, n))$ and $sk_i \in \text{Supp}(\text{KeyGen}(msk, i))$, it holds that

$$|pp| \leq p(\lambda, \log n) \quad \text{and} \quad |sk_i| \leq p(\lambda, \log n).$$

5.1 Trapdoor Proof Generator

As outlined in [Section 1.1](#), the main building block we use to construct our optimal broadcast encryption scheme is a trapdoor proof generator. A trapdoor proof generator is defined over a family of sets $\mathcal{X} = \{\mathcal{X}_\lambda\}_{\lambda \in \mathbb{N}}$. There is a public verification algorithm that verifies proofs associated with elements $x \in \mathcal{X}$. The trapdoor proof generator consists of two setup algorithms: a “normal” setup and an “alternative” setup:

- **Normal mode:** The normal setup algorithm outputs a set of public parameters pp (used for verifying proofs) along with a trapdoor td (used for generating proofs). Specifically, using the trapdoor, one can generate proofs π for random instances $x \xleftarrow{R} \mathcal{X}_\lambda$.
- **Alternative mode:** The alternative setup algorithm outputs a set of public parameters pp^* that are computationally indistinguishable from that in the normal mode. However, pp^* effectively partitions $\mathcal{X}$ into two disjoint sets: (1) a dense subset $\mathcal{X}_T \subset \mathcal{X}$; and (2) a sparse pseudorandom subset $\mathcal{X}_F \subset \mathcal{X}$. There are two sampling algorithms: (1) a `SampleTrue` algorithm that jointly samples an instance $x \in \mathcal{X}_T$ together with an accepting proof π for x (whose distributions are computationally indistinguishable from sampling $x \xleftarrow{R} \mathcal{X}$ and using a normal-mode trapdoor to sample the proof); and (2) a `SampleFalse` algorithm that samples an instance $x \in \mathcal{X}_F$ for which there does *not* exist any proof π that verifies with respect to pp^* . Finally, in the alternative mode, there is a trapdoor that allows one to efficiently decide membership in $\mathcal{X}_T$ and $\mathcal{X}_F$.

We formalize this notion below:

Definition 5.4 (Trapdoor Proof Generator). Let $\mathcal{X} = \{\mathcal{X}_\lambda\}_{\lambda \in \mathbb{N}}$ be a sequence of efficiently samplable and recognizable sets. A *trapdoor proof generator (TPG)* for $\mathcal{X}$ consists of a tuple of polynomial-time algorithms $(\text{Setup}, \text{CreateProof}, \text{Verify}, \text{SetupAlt}, \text{SampleTrue}, \text{SampleFalse}, \text{TDDecide})$ with the following syntax:

- $\text{Setup}(1^\lambda) \rightarrow (pp, td)$: A probabilistic algorithm that on input a security parameter λ , outputs public parameters pp and a trapdoor td . We implicitly assume that pp includes 1^λ and td includes pp .
- $\text{CreateProof}(td, x) \rightarrow \pi$: A deterministic algorithm that on input a trapdoor td and an instance $x \in \mathcal{X}_\lambda$, outputs a proof π .
- $\text{Verify}(pp, x, \pi) \rightarrow b$: A deterministic algorithm that on input the public parameters pp , an instance x , and a proof π , outputs a bit $b \in \{0, 1\}$.
- $\text{SetupAlt}(1^\lambda) \rightarrow (pp, td)$: A probabilistic algorithm that on input a security parameter λ , outputs public parameters pp and a trapdoor td . We assume for simplicity that pp includes 1^λ and td includes pp .
- $\text{SampleTrue}(pp) \rightarrow (x, \pi)$: A probabilistic algorithm that on input the public parameters pp , outputs an instance $x \in \mathcal{X}_\lambda$ and a proof π .
- $\text{SampleFalse}(pp) \rightarrow x$: A probabilistic algorithm that on input the public parameters pp , outputs a instance $x \in \mathcal{X}_\lambda$.

- $\text{TDDecide}(\text{td}, x) \rightarrow b$: A deterministic algorithm that on input a trapdoor td and an instance $x \in \mathcal{X}_\lambda$, outputs a bit b .

We require that $(\text{Setup}, \text{CreateProof}, \text{Verify}, \text{SetupAlt}, \text{SampleTrue}, \text{SampleFalse}, \text{TDDecide})$ satisfy the following properties:

- **Correctness:** There exists a negligible function negl such that for all $\lambda \in \mathbb{N}$, it holds that

$$\Pr \left[\begin{array}{l} (\text{pp}, \text{td}) \leftarrow \text{Setup}(1^\lambda) \\ x \xleftarrow{\mathcal{R}} \mathcal{X}_\lambda \\ \pi \leftarrow \text{CreateProof}(\text{td}, x) \end{array} : \text{Verify}(\text{pp}, x, \pi) = 1 \right] \geq 1 - \text{negl}(\lambda).$$

- **Mode indistinguishability:** For all stateful PPT algorithms A , there exists a negligible function negl such that for all $\lambda \in \mathbb{N}$, it holds that

$$\Pr [\text{Expt}_A^{\text{MI}}(\lambda) = 1] \leq 1/2 + \text{negl}(\lambda),$$

where $\text{Expt}_A^{\text{MI}}(\lambda)$ is defined via the following security game between the adversary A and a challenger on common input 1^λ :

- **Setup phase:** The challenger samples a bit $b \xleftarrow{\mathcal{R}} \{0, 1\}$. If $b = 0$, the challenger samples $(\text{pp}, \text{td}) \leftarrow \text{Setup}(1^\lambda)$, and if $b = 1$, the challenger samples $(\text{pp}, \text{td}) \leftarrow \text{SetupAlt}(1^\lambda)$. The challenger sends pp to A .
- **Query phase:** Algorithm A makes adaptive queries for either true or false instances. The challenger responds to the queries as follows:
 - * **True-instance query:** If $b = 0$, the challenger samples $x \xleftarrow{\mathcal{R}} \mathcal{X}_\lambda$ and $\pi \leftarrow \text{CreateProof}(\text{td}, x)$ and replies with (x, π) . If $b = 1$, the challenge responds with $(x, \pi) \leftarrow \text{SampleTrue}(\text{pp})$.
 - * **False-instance query:** The challenger replies with $x \xleftarrow{\mathcal{R}} \mathcal{X}_\lambda$ if $b = 0$ and $x \leftarrow \text{SampleFalse}(\text{pp})$ if $b = 1$.
- **Output phase:** Algorithm A computes a guess $b' \in \{0, 1\}$ for b , and the experiment outputs 1 if $b = b'$ and 0 otherwise.

- **Trapdoor decidability:** The following hold regarding the algorithm TDDecide :

- **Accepting true instances:** For all (possibly unbounded) algorithms A , there exists a negligible function negl such that for all $\lambda \in \mathbb{N}$, it holds that

$$\Pr \left[\begin{array}{l} (\text{pp}, \text{td}) \leftarrow \text{SetupAlt}(1^\lambda) \\ (x, \pi) \leftarrow A(\text{pp}) \end{array} : \text{Verify}(\text{pp}, x, \pi) = 1 \wedge \text{TDDecide}(\text{td}, x) \neq 1 \right] \leq \text{negl}(\lambda).$$

- **Rejecting false instances:** There exists a negligible function negl such that for all $\lambda \in \mathbb{N}$, it holds that

$$\Pr \left[\begin{array}{l} (\text{pp}, \text{td}) \leftarrow \text{SetupAlt}(1^\lambda) \\ x \leftarrow \text{SampleFalse}(\text{pp}) \end{array} : \text{TDDecide}(\text{td}, x) = 0 \right] \geq 1 - \text{negl}(\lambda).$$

5.2 Trapdoor Proof Generator Building Blocks

In [Section 5.3](#), we show how to construct a trapdoor proof generator from a public-key encryption scheme with pseudorandom ciphertexts together with a computational non-interactive zero-knowledge (NIZK) proof system for NP. We start by recalling these notions below:

Definition 5.5 (PKE with Pseudorandom Ciphertexts). Let $\ell = \ell(\lambda)$ be a length parameter. We say that a public-key encryption scheme $(\text{KeyGen}, \text{Enc}, \text{Dec})$ has *pseudorandom ciphertexts of length ℓ* if for all stateful PPT algorithms A ,

there exists a negligible function negl such that for all $\lambda \in \mathbb{N}$, it holds that

$$\Pr \left[\begin{array}{l} (\text{pk}, \text{sk}) \leftarrow \text{KeyGen}(1^\lambda) \\ \text{msg} \leftarrow A(\text{pk}) \\ \text{ct}_0^* \leftarrow \text{Enc}(\text{pk}, \text{msg}) \\ \text{ct}_1^* \xleftarrow{\mathbb{R}} \{0, 1\}^{\ell(\lambda)} \\ b \xleftarrow{\mathbb{R}} \{0, 1\} \\ b' \leftarrow A(\text{ct}_b^*) \end{array} : b' = b \right] \leq 1/2 + \text{negl}(\lambda).$$

Definition 5.6 (Non-Interactive Zero Knowledge). A computational non-interactive zero knowledge (NIZK) proof system for an NP language $\mathcal{L} = \{\mathcal{L}_\lambda\}_{\lambda \in \mathbb{N}}$ with witness relation $\mathbf{R}_\mathcal{L}$ consists of polynomial-time algorithms (Setup, Prove, Verify) with the following syntax:

- $\text{Setup}(1^\lambda) \rightarrow \text{crs}$: A probabilistic algorithm that on input a security parameter λ , outputs a common reference string crs . We implicitly assume that crs includes 1^λ .
- $\text{Prove}(\text{crs}, x, w) \rightarrow \pi$: A deterministic algorithm that on input a common reference string crs , an instance x , and a witness w , outputs a proof π .
- $\text{Verify}(\text{crs}, x, \pi) \rightarrow b$: A deterministic algorithm that on input a common reference string crs , an instance x , and a proof π , outputs a bit $b \in \{0, 1\}$.

We require that (Setup, Prove, Verify) satisfy the following properties:

- **Completeness**: There exists a negligible function negl such that for all $\lambda \in \mathbb{N}$, tuple $(1^\lambda, x, w) \in \mathbf{R}_\mathcal{L}$, it holds that

$$\Pr \left[\begin{array}{l} \text{crs} \leftarrow \text{Setup}(1^\lambda) \\ \pi \leftarrow \text{Prove}(\text{crs}, x, w) \end{array} : \text{Verify}(\text{crs}, x, \pi) = 1 \right] \geq 1 - \text{negl}(\lambda).$$

- **Statistical soundness**: For all (possibly unbounded) algorithms A , there exists a negligible function negl such that for all $\lambda \in \mathbb{N}$, it holds that

$$\Pr \left[\begin{array}{l} \text{crs} \leftarrow \text{Setup}(1^\lambda) \\ (x, \pi) \leftarrow A(\text{crs}) \end{array} : x \notin \mathcal{L}_\lambda \wedge \text{Verify}(\text{crs}, x, \pi) = 1 \right] \leq \text{negl}(\lambda).$$

- **(Multi-theorem) computational zero knowledge**: There exist PPT algorithms (SimSetup, SimProve) such that for all stateful PPT adversaries A , there exists a negligible function negl such that for all $\lambda \in \mathbb{N}$, it holds that

$$\Pr \left[\begin{array}{l} b \xleftarrow{\mathbb{R}} \{0, 1\} \\ \text{crs}_0 \leftarrow \text{Setup}(1^\lambda) \\ (\text{crs}_1, \sigma) \leftarrow \text{SimSetup}(1^\lambda) \\ b' \leftarrow A^{\mathcal{O}_b(\cdot, \cdot)}(\text{crs}_b) \end{array} : b' = b \right] \leq 1/2 + \text{negl}(\lambda),$$

where $\mathcal{O}_0(\cdot, \cdot)$ and $\mathcal{O}_1(\cdot, \cdot)$ are defined as follows:

- $\mathcal{O}_0(x, w)$ outputs $\text{Prove}(\text{crs}_0, x, w)$ if $(1^\lambda, x, w) \in \mathbf{R}_\mathcal{L}$ and $\perp$ otherwise.
- $\mathcal{O}_1(x, w)$ outputs $\text{SimProve}(\text{crs}_1, x, \sigma)$ if $(1^\lambda, x, w) \in \mathbf{R}_\mathcal{L}$ and $\perp$ otherwise.

Instantiations. Computational NIZK proofs are known from standard pairing-based assumptions [GOS06, CHK03] and lattice-based assumptions [PS19]. Public-key encryption with pseudorandom ciphertexts can be constructed from standard assumptions over groups (with or without a pairing) [Gam84] and from lattice-based assumptions [Reg05].

5.3 Constructing a Trapdoor Proof Generator

We now show how to construct a trapdoor proof generator from a public-key encryption scheme with pseudorandom ciphertexts in conjunction with a computational NIZK proof system for NP. We remark that this is just one way to construct such a primitive. However, we believe that this primitive may be of more general interest, and there are likely other constructions from potentially simpler building blocks. We leave it as an interesting open question to explore such directions.

Construction 5.7 (Trapdoor Proof Generator). Let $\ell = \ell(\lambda)$ be a polynomial and $\mathcal{X} = \{\mathcal{X}_\lambda\}_{\lambda \in \mathbb{N}}$ be a sequence of sets where $\mathcal{X}_\lambda = \{0, 1\}^{\ell(\lambda)}$. Our construction relies on the following building blocks:

- Let $\text{PKE} = (\text{PKE.KeyGen}, \text{PKE.Enc}, \text{PKE.Dec})$ be a semantically-secure public-key bit encryption scheme with pseudorandom ciphertexts of length ℓ . Without loss of generality, we assume that the encryption randomness to PKE is λ -bits long.⁸
- Let $\text{NIZK} = (\text{NIZK.Setup}, \text{NIZK.Prove}, \text{NIZK.Verify}, \text{NIZK.SimSetup}, \text{NIZK.SimProve})$ be a computational non-interactive zero knowledge proof for the language $\mathcal{L} = \{\mathcal{L}_\lambda\}_{\lambda \in \mathbb{N}}$ defined by the relation $\mathbf{R}_{\mathcal{L}}$ as follows. Instances of the language $\mathcal{L}_\lambda$ are of the form (pk, ct) and the relation $\mathbf{R}_{\mathcal{L}}$ is given by

$$\mathbf{R}_{\mathcal{L}}(1^\lambda, (pk, ct), r) = 1 \Leftrightarrow \text{PKE.Enc}(pk, 1; r) = ct.$$

We construct a trapdoor proof generator $\text{TPG} = (\text{Setup}, \text{CreateProof}, \text{Verify}, \text{SetupAlt}, \text{SampleTrue}, \text{SampleFalse}, \text{TDDecide})$ for $\mathcal{X}$ as follows:

- $\text{Setup}(1^\lambda)$: On input the security parameter λ , the setup algorithm samples $(pk, sk) \leftarrow \text{PKE.KeyGen}(1^\lambda)$, $(\text{crs}_{\text{NIZK}}, \sigma) \leftarrow \text{NIZK.SimSetup}(1^\lambda)$, and output the public parameters $pp = (pk, \text{crs}_{\text{NIZK}})$ and a trapdoor $td = \sigma$. Note that we implicitly assume that pp contains 1^λ and n , and td contains pp .
- $\text{CreateProof}(td, x)$: On input a trapdoor $td = \sigma$ with associated public parameters $pp = (pk, \text{crs}_{\text{NIZK}})$ and an instance $x \in \mathcal{X}_\lambda$, the algorithm generates the associated proof $\pi_{\text{NIZK}} = \text{NIZK.SimProve}(\text{crs}_{\text{NIZK}}, (pk, x), \sigma)$.
- $\text{Verify}(pp, x, \pi)$: On input the public parameters $pp = (pk, \text{crs}_{\text{NIZK}})$, an instance $x \in \mathcal{X}_\lambda$, and a proof π , the verification algorithm outputs $\text{NIZK.Verify}(\text{crs}_{\text{NIZK}}, (pk, x), \pi)$.
- $\text{SetupAlt}(1^\lambda)$: On input the security parameter λ , the alternative setup algorithm starts by sampling $(pk, sk) \leftarrow \text{PKE.KeyGen}(1^\lambda)$, $\text{crs}_{\text{NIZK}} \leftarrow \text{NIZK.Setup}(1^\lambda)$, and outputs the public parameters $pp = (pk, \text{crs}_{\text{NIZK}})$ and trapdoor $td = sk$. Recall that we implicitly assume that pp contains 1^λ and n and td contains pp .
- $\text{SampleTrue}(pp)$: On input the public parameters $pp = (pk, \text{crs}_{\text{NIZK}})$, the true-instance sampling algorithm samples $r \xleftarrow{\mathcal{R}} \{0, 1\}^\lambda$, computes $ct = \text{PKE.Enc}(pk, 1; r)$, and outputs the instance $x = ct$ and proof $\pi = \text{NIZK.Prove}(\text{crs}_{\text{NIZK}}, (pk, ct), r)$.
- $\text{SampleFalse}(pp)$: On input the public parameters $pp = (pk, \text{crs}_{\text{NIZK}})$, the false-instance sampling algorithm computes $ct \leftarrow \text{PKE.Enc}(pk, 0)$, and outputs the instance $x = ct$.
- $\text{TDDecide}(td, x)$: On input the trapdoor $td = sk$ and an instance $x \in \mathcal{X}_\lambda$, the trapdoor decision algorithm outputs 1 if $\text{PKE.Dec}(sk, x) = 1$ and 0 otherwise.

Correctness and security analysis. We now show that TPG from [Construction 5.7](#) satisfies each of the correctness and security properties in [Definition 5.4](#).

Theorem 5.8 (Correctness). *Assuming PKE has pseudorandom ciphertexts and NIZK satisfies completeness and computational zero knowledge, then [Construction 5.7](#) satisfies correctness.*

⁸If PKE.Enc requires more than λ -bits of randomness, we can first stretch the randomness using a pseudorandom generator.

Proof. We consider the following sequence of hybrid experiments, defined as a function of the security parameter $\lambda \in \mathbb{N}$.

- $H_0(\lambda)$: This experiment corresponds to the correctness criteria. Namely, $(pp, td) \leftarrow \text{Setup}(1^\lambda)$, $x \xleftarrow{\mathcal{R}} \mathcal{X}_\lambda$, $\pi = \text{CreateProof}(td, x)$, and then the experiment outputs the results of $\text{Verify}(pp, x, \pi)$.
- $H_1(\lambda)$: This hybrid is the same as H_0 , except the challenger instead computes $x \leftarrow \text{PKE.Enc}(pk, 1; r)$, where pk is the public key specified in the public parameters pp , and $r \xleftarrow{\mathcal{R}} \{0, 1\}^\lambda$ is freshly-sampled randomness.
- $H_2(\lambda)$: In this hybrid, we now compute $\text{crs}_{\text{NIZK}} \leftarrow \text{NIZK.Setup}(1^\lambda)$ and $\pi_{\text{NIZK}} \leftarrow \text{NIZK.Prove}(\text{crs}_{\text{NIZK}}, (pk, x), r)$.

It remains to show that there exists a negligible function negl such that $\Pr[H_0(\lambda) = 1] \geq 1 - \text{negl}(\lambda)$. We now argue that there exist negligible functions μ, μ' such that $\Pr[H_2(\lambda) = 1] - \Pr[H_0(\lambda) = 1] \leq \mu(\lambda)$ and moreover, that $\Pr[H_2(\lambda) = 1] \geq 1 - \mu'(\lambda)$. This implies that $\Pr[H_0(\lambda) = 1] \geq 1 - \text{negl}(\lambda)$ for some negligible function negl , as required for correctness.

Claim 5.9. *Assuming PKE has pseudorandom ciphertexts, there exists a negligible function μ such that for all $\lambda \in \mathbb{N}$, $\Pr[H_1(\lambda) = 1] - \Pr[H_0(\lambda) = 1] \leq \mu(\lambda)$.*

Proof. Suppose by way of contradiction that there exists a polynomial q such that $\Pr[H_1(\lambda) = 1] - \Pr[H_0(\lambda) = 1] > 1/q(\lambda)$. We construct a PPT adversary B that breaks that pseudorandom ciphertext property of PKE:

1. On input a public key pk , B sends the message $\text{msg} = 1$.
2. The challenger samples a bit $b \xleftarrow{\mathcal{R}} \{0, 1\}$ and sends the challenge ciphertext ct_b^* , which is either equal to $\text{ct}_0^* \leftarrow \text{PKE.Enc}(pk, 1)$ or $\text{ct}_1^* \xleftarrow{\mathcal{R}} \{0, 1\}^{\ell(\lambda)}$, where ℓ is the length of ciphertexts for PKE.
3. Algorithm B samples $(\text{crs}_{\text{NIZK}}, \sigma) \leftarrow \text{NIZK.SimSetup}(1^\lambda)$ and $\pi \leftarrow \text{NIZK.SimProve}(\text{crs}_{\text{NIZK}}, (pk, \text{ct}_b^*), \sigma)$.
4. Algorithm B then computes $b' = \text{Verify}(pp, (pk, \text{ct}_b^*))$ and outputs $1 - b'$ as its guess for b .

Algorithm B runs in polynomial time by construction. To analyze the success probability of B , note that if the challenger chooses the bit $b = 0$, then the output of B corresponds to $1 - H_1(\lambda)$, so B wins if $H_1(\lambda)$ outputs 1. If $b = 1$, then the output of B corresponds to $1 - H_0(\lambda)$, so B wins if $H_0(\lambda)$ outputs 0. Therefore, B succeeds with probability

$$\frac{1}{2} \cdot \Pr[H_1(\lambda) = 1] + \frac{1}{2} \cdot \Pr[H_0(\lambda) = 0] = \frac{1}{2} + \frac{1}{2} \cdot (\Pr[H_0(\lambda) = 1] - \Pr[H_1(\lambda) = 1]).$$

As $\Pr[H_1(\lambda) = 1] - \Pr[H_0(\lambda) = 1] > 1/q(\lambda)$ by assumption, this violates the pseudorandom ciphertext property of PKE, as required. $\square$

Claim 5.10. *Assuming NIZK satisfies computational zero knowledge, there exists a negligible function μ such that for all $\lambda \in \mathbb{N}$, $\Pr[H_2(\lambda) = 1] - \Pr[H_1(\lambda) = 1] \leq \mu(\lambda)$.*

Proof. Suppose by way of contradiction, there exists a polynomial q such that $\Pr[H_2(\lambda) = 1] - \Pr[H_1(\lambda) = 1] > q(\lambda)$. We construct a PPT adversary B that breaks the computational zero knowledge property of NIZK:

1. The NIZK challenger starts by sampling a random bit $b \xleftarrow{\mathcal{R}} \{0, 1\}$ and sends crs_b , which is either equal to $\text{crs}_0 \leftarrow \text{NIZK.Setup}(1^\lambda)$ or $(\text{crs}_1, \sigma) \leftarrow \text{NIZK.SimSetup}(1^\lambda)$.
2. On input crs_b , algorithm B samples $(pk, sk) \leftarrow \text{PKE.KeyGen}(1^\lambda)$ and sets $pp = (pk, \text{crs}_b)$. Algorithm B computes $\text{ct} \leftarrow \text{PKE.Enc}(pk, 1; r)$ for randomness $r \xleftarrow{\mathcal{R}} \{0, 1\}^\lambda$ and queries its oracle on the instance (pk, ct) with witness r .
 - If $b = 0$, the oracle returns $\pi_0 \leftarrow \text{NIZK.Prove}(\text{crs}_0, (pk, \text{ct}), r)$.
 - If $b = 1$, the oracle returns $\pi_1 \leftarrow \text{SimProve}(\text{crs}_1, (pk, \text{ct}), \sigma)$.
3. Algorithm B computes $b' = \text{Verify}(pp, (pk, \text{ct}), \pi_b)$ and outputs $1 - b'$ as its guess for b .

Algorithm B runs in polynomial time by construction. To analyze the success probability of B , note that if the challenger chooses the bit $b = 0$, then the output of B corresponds to $1 - H_2(\lambda)$, so B wins if $H_2(\lambda)$ outputs 1. If $b = 1$, the output of B corresponds to $1 - H_1(\lambda)$, so B wins if $H_1(\lambda)$ outputs 0. Therefore, the success probability of B is given by

$$\frac{1}{2} \cdot \Pr[H_2(\lambda) = 1] + \frac{1}{2} \cdot \Pr[H_1(\lambda) = 0] = \frac{1}{2} + \frac{1}{2} \cdot (\Pr[H_2(\lambda) = 1] - \Pr[H_1(\lambda) = 1]).$$

As $\Pr[H_2(\lambda) = 1] - \Pr[H_1(\lambda) = 1] > 1/q(\lambda)$ by assumption, this violates the computational zero knowledge property of NIZK, as required. $\square$

Claim 5.11. *Assuming NIZK is complete, there exists a negligible function μ such that for all $\lambda \in \mathbb{N}$, it holds that $\Pr[H_2(\lambda) = 1] \geq 1 - \mu(\lambda)$.*

Proof. By construction, $H_2(\lambda)$ corresponds to the completeness experiment for NIZK for the language $\mathcal{L}_\lambda$ with instance (pk, ct) for $ct \leftarrow \text{PKE.Enc}(pk, 1)$ and witness r . Thus, $(1^\lambda, (pk, ct), r) \in \mathbf{R}_{\mathcal{L}}$, so by completeness of NIZK, there exists a negligible function μ such that $\Pr[H_2(\lambda) = 1] \geq 1 - \mu(\lambda)$. $\square$

This proof now follows from [Claims 5.9 to 5.11](#). $\square$

Theorem 5.12 (Mode Indistinguishability). *Assuming PKE has pseudorandom ciphertexts and NIZK satisfies computational zero knowledge, then [Construction 5.7](#) satisfies mode indistinguishability.*

Proof. Suppose by way of contradiction that mode indistinguishability does not hold. Then there exists a stateful PPT algorithm A and a polynomial q such that for infinitely many $\lambda \in \mathbb{N}$, it holds that

$$\Pr[\text{Expt}_A^{\text{MI}}(\lambda) = 1] > 1/2 + 1/q(\lambda).$$

We now define the following sequence of hybrid experiments, each parameterized by the security parameter $\lambda \in \mathbb{N}$:

- $H_0(\lambda)$: This experiment corresponds to the mode indistinguishability experiment where b is fixed to 0, and the hybrid outputs the final value b' output by A .
- $H_1(\lambda)$: This hybrid is the same as H_0 except that the challenger samples $x \leftarrow \text{PKE.Enc}(pk, 1)$ for true-instance queries, and $x \leftarrow \text{PKE.Enc}(pk, 0)$ for false-instance queries.
- $H_2(\lambda)$: This hybrid corresponds to the mode indistinguishability experiment where b is fixed to 1. Specifically, compared to hybrid $H_1(\lambda)$, the challenger now samples $(x, \pi) \leftarrow \text{SampleTrue}(pp)$ for true-instance queries.

For $i \in \{1, 2\}$, we define $\delta_i(\lambda) = \Pr[H_i(\lambda) = 1] - \Pr[H_{i-1}(\lambda) = 1]$. Then, the success probability of A can be written as

$$\begin{aligned} \Pr[A \text{ wins}] &= \frac{1}{2} \cdot \Pr[H_0(\lambda) = 0] + \frac{1}{2} \cdot \Pr[H_2(\lambda) = 1] \\ &= \frac{1}{2} \cdot \Pr[H_0(\lambda) = 0] + \frac{1}{2} \cdot (\Pr[H_0(\lambda) = 1] + \delta_1(\lambda) + \delta_2(\lambda)) \\ &= \frac{1}{2} + \frac{1}{2} \cdot (\delta_1(\lambda) + \delta_2(\lambda)). \end{aligned}$$

It follows from our assumption that, for infinitely many $\lambda \in \mathbb{N}$, $\delta_1(\lambda) + \delta_2(\lambda) > 2/q(\lambda)$. Thus, either $\delta_1(\lambda) > 1/q(\lambda)$ or $\delta_2(\lambda) > 1/q(\lambda)$. We show in the subsequent claims that if either case holds, it must contradict one of the assumptions in the theorem statement.

Claim 5.13. *If $\delta_1(\lambda) > 1/q(\lambda)$ for infinitely many $\lambda \in \mathbb{N}$, then PKE does not have pseudorandom ciphertexts.*

Proof. Let $Q = Q(\lambda)$ be polynomial upper bounding the number of queries that A makes in the mode indistinguishability experiment. We construct an adversary B that breaks the pseudorandom ciphertext property of PKE:

1. At the beginning of the security game, algorithm B receives a public key pk as input from the PKE challenger.

2. Algorithm B samples $(\text{crs}_{\text{NIZK}}, \sigma) \leftarrow \text{NIZK.SimSetup}(1^\lambda)$ and sets $\text{pp} = (\text{pk}, \text{crs}_{\text{NIZK}})$, $\text{td} = \sigma$. It gives pp to A .
3. Algorithm B samples an index $i^* \xleftarrow{\mathbb{R}} [0, Q(\lambda) - 1]$. Whenever A makes its i th query, algorithm B responds as follows.
 - If $i < Q(\lambda) - i^*$, algorithm B responds as in hybrid $H_0(\lambda)$ by sampling a random $x \xleftarrow{\mathbb{R}} \mathcal{X}_\lambda$ and additionally providing a proof $\pi = \text{CreateProof}(\text{td}, x)$ on queries for true instances.
 - If $i = Q(\lambda) - i^*$, algorithm B sends the message $\text{msg} = 1$ to the PKE challenger if A made a query for a true instance and the message $\text{msg} = 0$ for a false query. The challenger replies with a challenge ciphertext ct^* which algorithm B forwards to A . In addition, if the query was for a true instance, algorithm B also gives a proof $\pi = \text{CreateProof}(\text{td}, x)$ to A . Here, the challenge ciphertext is computed as follows. The challenger samples a bit $\hat{b} \xleftarrow{\mathbb{R}} \{0, 1\}$ and sets the challenge ciphertext $\text{ct}^* \leftarrow \text{Enc}(\text{pk}, \text{msg})$ if $\hat{b} = 0$ and $\text{ct}^* \xleftarrow{\mathbb{R}} \{0, 1\}^{\ell(\lambda)}$ if $\hat{b} = 1$.
 - If $i > Q(\lambda) - i^*$, B sets $x \leftarrow \text{Enc}(\text{pk}, 0)$ on false-instance queries and $x \leftarrow \text{Enc}(\text{pk}, 1)$ on true-instance queries. On true-instance queries, it also provides a proof $\pi = \text{CreateProof}(\text{td}, x)$ for true-instance queries.
4. A computes a guess b' for b and B outputs the guess $\hat{b}' = 1 - b'$.

It is clear that A runs in polynomial-time if B does. To analyze the success probability of B , we define the following sub-hybrid experiments.

- $H_{0,i}(\lambda)$: In this hybrid, B responds to the first $Q(\lambda) - i$ queries according to H_0 and the final i queries according to H_1 .

Note that $H_{0,0}(\lambda)$ corresponds to $H_0(\lambda)$ and $H_{0,Q(\lambda)}(\lambda)$ corresponds to $H_1(\lambda)$. For a fixed index $i^* = i$, we observe that if $\hat{b} = 1$, then A 's response is distributed as in hybrid $H_{0,i}$, and if $\hat{b} = 0$, then A 's response is distributed as in hybrid $H_{0,i+1}$. It follows that B succeeds with probability

$$\begin{aligned}
& \sum_{i=0}^{Q(\lambda)-1} \Pr[i^* = i] \cdot \Pr[\hat{b}' = b' \mid i^* = i] \\
&= \frac{1}{Q(\lambda)} \cdot \sum_{i=0}^{Q(\lambda)-1} \left(\frac{1}{2} \cdot \Pr[H_{0,i}(\lambda) = 0] + \frac{1}{2} \cdot \Pr[H_{0,i+1}(\lambda) = 1] \right) \\
&= \frac{1}{2} + \frac{1}{2Q(\lambda)} \cdot \sum_{i=0}^{Q(\lambda)-1} (\Pr[H_{0,i+1}(\lambda) = 1] - \Pr[H_{0,i}(\lambda) = 1]) \\
&= \frac{1}{2} + \frac{1}{2Q(\lambda)} \cdot (\Pr[H_1(\lambda) = 1] - \Pr[H_0(\lambda) = 1]).
\end{aligned}$$

By our assumption, this is at least $1/2 + 1/(2 \cdot Q(\lambda) \cdot q(\lambda))$. Since Q and q are both polynomially bounded, this contradicts the pseudorandom ciphertext property of PKE, as required. $\square$

Claim 5.14. *If $\delta_2(\lambda) > 1/q(\lambda)$ for infinitely many $\lambda \in \mathbb{N}$, then NIZK does not satisfy multi-theorem computational zero knowledge.*

Proof. We construct an adversary B that breaks the computational zero knowledge property of NIZK:

1. The NIZK challenger samples a bit $\hat{b} \xleftarrow{\mathbb{R}} \{0, 1\}$. If $\hat{b} = 0$, it sends $\text{crs} \leftarrow \text{NIZK.Setup}(1^\lambda)$, and if $\hat{b} = 1$, it sends crs where $(\text{crs}, \sigma) \leftarrow \text{NIZK.SimSetup}(1^\lambda)$.
2. Algorithm B computes $(\text{pk}, \text{sk}) \leftarrow \text{PKE.KeyGen}(1^\lambda)$, sets $\text{pp} = (\text{pk}, \text{crs})$, and sends pp to A .
3. Whenever A makes a true-instance query, algorithm B samples $\text{ct} \leftarrow \text{Enc}(\text{pk}, 1; r)$ for $r \xleftarrow{\mathbb{R}} \{0, 1\}^\lambda$ and queries the NIZK proof oracle on $((\text{pk}, \text{ct}), r)$ to get a proof π . Algorithm B responds to A with (ct, π) . Whenever A makes a false-instance query, algorithm B samples $\text{ct} \leftarrow \text{Enc}(\text{pk}, 0)$ and returns the instance ct .

4. A outputs a guess b' for b , and B outputs the same guess $\hat{b}' = b'$ for $\hat{b}$.

Algorithm B clearly runs in polynomial-time if A does. For correctness, observe that if $\hat{b} = 0$, then the view of A is identically distributed as in hybrid $H_1(\lambda)$, and if $\hat{b} = 1$, then the view of A is identically distributed as in hybrid $H_2(\lambda)$. So, B succeeds with probability

$$\Pr[B \text{ wins}] = \frac{1}{2} \cdot \Pr[H_1(\lambda) = 0] + \frac{1}{2} \cdot \Pr[H_2(\lambda) = 1] = \frac{1}{2} + \frac{1}{2} \cdot (\Pr[H_2(\lambda) = 1] - \Pr[H_1(\lambda) = 1]).$$

As $\Pr[H_2(\lambda) = 1] - \Pr[H_1(\lambda) = 1] = \delta_2(\lambda) > 1/q(\lambda)$ by assumption, this violates the computational zero knowledge property of NIZK, as required. $\square$

The proof now follows via [Claims 5.13](#) and [5.14](#). $\square$

Theorem 5.15 (Decidability in Trapdoor Mode). *Assuming PKE is correct and NIZK satisfies completeness and is statistically sound, then [Construction 5.7](#) satisfies decidability in trapdoor mode.*

Proof. For any $\lambda \in \mathbb{N}$, let $(pp, td) \leftarrow \text{SetupAlt}(1^\lambda)$. Recall that $pp = (pk, \text{crs}_{\text{NIZK}})$ and $td = sk$, where $\text{crs}_{\text{NIZK}} \leftarrow \text{NIZK.Setup}(1^\lambda)$. We show the two properties separately:

- **Accepting true instances:** Suppose that there exists a proof π such that $\text{Verify}(pp, x, \pi) = 1$. By construction, this implies that $\text{NIZK.Verify}(\text{crs}_{\text{NIZK}}, (pk, x), \pi) = 1$. By statistical soundness of NIZK, there exists a negligible function negl such that that $(pk, x) \in \mathcal{L}_\lambda$ with probability at least $1 - \text{negl}(\lambda)$. If $(pk, x) \in \mathcal{L}_\lambda$, then there exists $r \in \{0, 1\}^\lambda$ such that $\text{PKE.Enc}(pk, 1; r) = x$. By correctness of PKE, this means that $\text{PKE.Dec}(sk, x) = 1$. In this case, $\text{TDDecide}(pp, x)$ outputs 1 with probability at least $1 - \text{negl}(\lambda)$, and the claim follows.
- **Rejecting false instances:** Suppose $x \leftarrow \text{SampleFalse}(pp)$. This implies that $x \in \text{Supp}(\text{PKE.Enc}(pk, 0))$, so $\text{PKE.Dec}(sk, x) = 0$. That immediately implies that $\text{TDDecide}(td, x)$ outputs 0, as required. $\square$

5.4 Constructing Optimal Broadcast Encryption

We are now ready to construct an optimal broadcast encryption scheme from witness encryption and function-binding hash functions. Our construction follows the following template:

- We first construct a (non-optimal) broadcast encryption scheme with short ciphertexts and secret keys, but *long* public parameters. In particular, the public parameters contains a *uniform* random string of length n , where n is the number of users.
- To obtain an optimal broadcast encryption scheme in the random oracle model, we instantiate the above construction, but now derive the long random string in the public parameters as the output of a random oracle. The rest of the scheme is unchanged. The random oracle is only used to “compress” the long public parameters.

We now describe our broadcast encryption scheme:

Construction 5.16 (Broadcast Encryption with Long Public Key). Let $s = s(\lambda)$, $\ell_{\text{blk}} = \ell_{\text{blk}}(\lambda)$, $d = d(\lambda)$, and $r = r(\lambda)$ be polynomials, and let $m = m(\lambda)$ be any function. Let $\mathcal{M} = \{\mathcal{M}_\lambda\}_{\lambda \in \mathbb{N}}$ be a message space of length m . Let $\mathcal{X} = \{\mathcal{X}_\lambda\}_{\lambda \in \mathbb{N}}$ be a set of strings of size ℓ_{blk} that is efficiently-recognizable. Our broadcast encryption scheme relies on the following primitives:

- First, let $\text{TPG} = (\text{TPG.Setup}, \text{TPG.CreateProof}, \text{TPG.Verify}, \text{TPG.SetupAlt}, \text{TPG.SampleTrue}, \text{TPG.SampleFalse}, \text{TPG.TDDecide})$ be a trapdoor proof generator for $\mathcal{X}$. For all security parameters $\lambda \in \mathbb{N}$ and parameters $(pp_{\text{TPG}}, td_{\text{TPG}}) \in \text{Supp}(\text{TPG.SetupAlt}(1^\lambda))$, we require that $\text{TPG.TDDecide}(td_{\text{TPG}}, \cdot)$ can be computed by a circuit of size $s(\lambda)$ and depth $d(\lambda)$.
- Let $\text{FBH} = (\text{FBH.Setup}, \text{FBH.SetupBinding}, \text{FBH.Hash}, \text{FBH.ProveOpen}, \text{FBH.VerOpen})$ be a function-binding hash for the class $\mathcal{F}$ of disjunctions of block functions for input length ℓ_{blk} , size s , and depth d ([Definition 3.3](#)).

- Let WE be a witness encryption scheme for the language $\mathcal{L} = \{\mathcal{L}_\lambda\}_{\lambda \in \mathbb{N}}$ defined by the relation $\mathbf{R}_\mathcal{L}$ as follows. An instance of the language $\mathcal{L}_\lambda$ has the form $(\text{hk}, \text{pp}_{\text{TPG}}, \text{dig})$ and the relation $\mathbf{R}_\mathcal{L}$ is given by

$$\begin{aligned} \mathbf{R}_\mathcal{L}(1^\lambda, (\text{hk}, \text{pp}_{\text{TPG}}, \text{dig}), (i, \text{hinp}_i, \pi_{\text{TPG}}, \pi_{\text{FBH}})) &= 1 \\ \Leftrightarrow \text{TPG.Verify}(\text{pp}_{\text{TPG}}, \text{hinp}_i, \pi_{\text{TPG}}) &= 1 \wedge \text{FBH.VerOpen}(\text{hk}, \text{dig}, \{i\}, \{(i, \text{hinp}_i)\}, \pi_{\text{FBH}}) = 1. \end{aligned}$$

We construct an optimal broadcast encryption scheme $\text{BE} = (\text{Setup}, \text{KeyGen}, \text{Enc}, \text{Dec})$ as follows:

- $\text{Setup}(1^\lambda, 1^n)$: On input the security parameter λ and a bound on the number of users n , the setup algorithm first samples $(\text{pp}_{\text{TPG}}, \text{td}_{\text{TPG}}) \leftarrow \text{TPG.Setup}(1^\lambda)$ and $\text{hk} \leftarrow \text{FBH.Setup}(1^\lambda, n)$. Then, for each $i \in [n]$, it samples $x_i \xleftarrow{\mathcal{R}} \mathcal{X}_\lambda$. It now outputs the public parameters $\text{pp} = (\text{pp}_{\text{TPG}}, \text{hk}, (x_1, \dots, x_n))$ and master secret key $\text{msk} = (\text{td}_{\text{TPG}}, (x_1, \dots, x_n))$.
- $\text{KeyGen}(\text{msk}, i)$: On input the master secret key $\text{msk} = (\text{td}_{\text{TPG}}, (x_1, \dots, x_n))$, the key-generation algorithm computes the proof $\pi_{\text{TPG}} = \text{TPG.CreateProof}(\text{td}_{\text{TPG}}, x_i)$, and outputs the secret key $\text{sk}_i = \pi_{\text{TPG}}$.
- $\text{Enc}(\text{pp}, \text{msg}, S)$: On input the public parameters $\text{pp} = (\text{pp}_{\text{TPG}}, \text{hk}, (x_1, \dots, x_n))$, a message $\text{msg} \in \mathcal{M}_\lambda$, and a set $S = \{i_1, \dots, i_k\} \subseteq [n]$ for some $k \leq n$ where $i_1 < \dots < i_k$, the encryption algorithm computes the digest $\text{dig} = \text{FBH.Hash}(\text{hk}, (x_{i_1}, \dots, x_{i_k}))$ and outputs $\text{ct} \leftarrow \text{WE.Enc}(1^\lambda, \text{msg}, (\text{hk}, \text{pp}_{\text{TPG}}, \text{dig}))$.
- $\text{Dec}(\text{pp}, \text{ct}, (j, \text{sk}_j), S)$: On input the public parameters $\text{pp} = (\text{pp}_{\text{TPG}}, \text{hk}, (x_1, \dots, x_n))$, a ciphertext ct , an index/secret-key pair (j, sk_j) , and a set $S = \{i_1, \dots, i_k\} \subseteq [n]$ for some $k \leq n$ where $i_1 < \dots < i_k$, the decryption algorithm computes $\pi_{\text{FBH}} = \text{FBH.ProveOpen}(\text{hk}, (x_{i_1}, \dots, x_{i_k}), \{j'\})$, where $j' \in [k]$ is the index such that $j = i_{j'}$. It then outputs $\text{msg} = \text{WE.Dec}(1^\lambda, \text{ct}, (j', x_j, \text{sk}_j, \pi_{\text{FBH}}))$.

Correctness and efficiency. We first that the broadcast encryption scheme in [Construction 5.16](#) satisfies correctness and has succinct ciphertexts and secret keys.

Theorem 5.17 (Correctness and Efficiency). *Assuming TPG and WE are correct and efficient, and FBH satisfies completeness and efficiency, then [Construction 5.16](#) is correct and has succinct ciphertexts and secret keys.*

Proof. We consider each property separately:

- **Succinct ciphertexts and secret keys:** Take any $\lambda \in \mathbb{N}$, $n \in \mathbb{N}$, $\text{msg} \in \mathcal{M}_\lambda$, $S \subseteq [n]$, $(\text{pp}, \text{msk}) \in \text{Supp}(\text{Setup}(1^\lambda, n))$ and $\text{ct} \in \text{Supp}(\text{Enc}(\text{pp}, \text{msg}, S))$. To show that the scheme has succinct ciphertexts, we need to show that $|\text{ct}| \in \text{poly}(\lambda, m(\lambda), \log n)$. First, since WE is efficient, $|\text{ct}| \in \text{poly}(\lambda, m(\lambda), T)$ where T is the time required to check $\mathbf{R}_\mathcal{L}$ on instances in $\mathcal{L}_\lambda$. Checking instances of $\mathbf{R}_\mathcal{L}$ requires running $\text{TPG.Verify}(\text{pp}_{\text{TPG}}, \text{hinp}_i, \pi_{\text{TPG}})$ and $\text{FBH.VerOpen}(\text{hk}, \text{dig}, \{i\}, \{(i, \text{hinp}_i)\}, \pi_{\text{FBH}})$. It suffices to bound the length of the inputs to TPG.Verify and FBH.VerOpen :
 - For the inputs TPG.Verify , note that $|\text{pp}_{\text{TPG}}| \in \text{poly}(\lambda)$, $|\text{hinp}_i| = \ell_{\text{blk}}(\lambda)$, and $|\pi_{\text{TPG}}| \in \text{poly}(\lambda, \ell_{\text{blk}}(\lambda))$, which are all of size $\text{poly}(\lambda)$ by assumption.
 - For the inputs to FBH.VerOpen , first observe that $|\text{hk}| \in \text{poly}(\lambda, s(\lambda), \log n)$ by the efficiency of FBH, which is $\text{poly}(\lambda, \log n)$ as s is polynomially-bounded. Also by the efficiency of FBH, $|\text{dig}|, |\pi_{\text{FBH}}| \in \text{poly}(\lambda, \log |S|)$. Lastly, $|i| \leq \log n$ and $|\text{hinp}_i| = \ell_{\text{blk}}(\lambda) \in \text{poly}(\lambda)$. Thus the inputs to FBH.VerOpen have size at most $\text{poly}(\lambda, \log n)$, as required.

To argue that the scheme has short secret keys, take any index $i \in [n]$ and consider any secret key $\text{sk}_i \in \text{Supp}(\text{KeyGen}(\text{msk}, i))$. By construction, $\text{sk}_i = \pi_{\text{TPG}}$. By our above analysis, $|\pi_{\text{TPG}}| = \text{poly}(\lambda, \ell_{\text{blk}}) = \text{poly}(\lambda)$.

- **Correctness:** Let $n = n(\lambda)$ be a bound on the number of participants. Take any $\lambda \in \mathbb{N}$, subset of users $S = \{i_1, \dots, i_k\} \subseteq [n]$, index $j \in S$, and message $\text{msg} \in \mathcal{M}_\lambda$. Then, we have the following:
 - Let $(\text{pp}, \text{msk}) \leftarrow \text{Setup}(1^\lambda, n(\lambda))$, where $\text{pp} = (\text{pp}_{\text{TPG}}, \text{hk}, (x_1, \dots, x_n))$, and $\text{msk} = \text{td}_{\text{TPG}}$. Let $\text{sk}_j \leftarrow \text{KeyGen}(\text{msk}, j)$. By construction, $\text{sk}_j = \pi_{\text{TPG}} = \text{TPG.CreateProof}(\text{td}_{\text{TPG}}, x_j)$.

- By correctness of TPG, $\text{TPG.Verify}(\text{pp}_{\text{TPG}}, x_j, \text{sk}_j) = 1$ with probability at least $1 - \mu(\lambda)$ for some negligible function $\mu = \mu(\lambda)$.
- Let $\text{ct} \leftarrow \text{Enc}(\text{pp}, \text{msg}, S)$. By construction, Enc computes $\text{dig} = \text{FBH.Hash}(\text{hk}, (x_{i_1}, \dots, x_{i_k}))$. The ciphertext ct is a witness encryption of msg for the instance $(\text{hk}, \text{pp}_{\text{TPG}}, \text{dig})$.
- Consider the output of $\text{Dec}(\text{pp}, \text{ct}, (j, \text{sk}_j), S)$. Let $j' \in [k]$ be the index such that $j = i_{j'}$. Now, the decryption algorithm computes $\pi_{\text{FBH}} = \text{FBH.ProveOpen}(\text{hk}, (x_{i_1}, \dots, x_{i_k}), \{j'\})$. By completeness of FBH, $\text{FBH.VerOpen}(\text{hk}, \text{dig}, S, \{(j', x_j)\}, \pi_{\text{FBH}}) = 1$. Thus, $(j', x_j, \text{sk}_j, \pi_{\text{FBH}}) = (j', x_i, \pi_{\text{TPG}}, \pi_{\text{FBH}})$ is a valid witness for the instance $(\text{hk}, \text{pp}_{\text{TPG}}, \text{dig})$ with probability at least $1 - \mu(\lambda)$. By correctness of WE, Dec outputs msg with the same probability. Since μ is negligible, correctness follows. $\square$

Security. Next, we show that [Construction 5.16](#) satisfies semi-static security. The structure of the proof is similar to that of [Theorem 4.5](#), but we provide the details for completeness and to provide another example for how to use function-binding hash functions in conjunction with witness encryption.

Theorem 5.18 (Semi-Static Security). *Assuming TPG satisfies mode indistinguishability and trapdoor decidability, FBH satisfies computational function hiding and statistical function binding, and WE satisfies message indistinguishability, then [Construction 5.16](#) satisfies semi-static security.*

Proof. Suppose by way of contradiction that semi-static security does not hold. Namely, there exists a stateful PPT algorithm A , an efficiently computable function $n \in \text{poly}(\lambda)$, a polynomial q , and an infinite set $\Lambda \subseteq \mathbb{N}$ such that for all $\lambda \in \Lambda$, it holds that

$$\Pr \left[\text{Expt}_{A, n(\lambda)}^{\text{BE, SS}}(\lambda) = 1 \right] \geq 1/2 + 1/q(\lambda).$$

For fixed parameters as above, we define the following sequence of hybrid experiments for each $\lambda \in \mathbb{N}$.

- $H_{b,0}(\lambda)$: This hybrid is identical to the experiment $\text{Expt}_{A, n(\lambda)}^{\text{BE, SS}}(\lambda)$ for the fixed value of b .
- $H_{b,1}(\lambda)$: This hybrid is the same as $H_{b,0}$ except the challenger samples the public parameters pp using the following modified algorithm:
 - First, it samples $(\text{pp}_{\text{TPG}}, \text{td}_{\text{TPG}}) \leftarrow \text{TPG.SetupAlt}(1^\lambda)$.
 - The hash key $\text{hk} \leftarrow \text{FBH.Setup}(1^\lambda, n)$ is sampled as in $H_{b,0}$.
 - Let $S \subseteq [n]$ be the set the adversary commits to at the beginning of the security game. For each $i \in [n] \setminus S$, the challenger samples $(x_i, \pi_i) \leftarrow \text{TPG.SampleTrue}(\text{pp}_{\text{TPG}})$. For each $i \in S$, the challenger samples $x_i \leftarrow \text{TPG.SampleFalse}(\text{pp}_{\text{TPG}})$.

The public parameters pp is still defined to be $\text{pp} = (\text{pp}_{\text{TPG}}, \text{hk}, (x_1, \dots, x_n))$. Then, when answering the key-generation queries for an index $i \in [n] \setminus S$, the challenger responds with π_i .

- $H_{b,2}(\lambda)$: In this hybrid, the challenger instead computes $\text{hk} \leftarrow \text{FBH.SetupBinding}(1^\lambda, n(\lambda), f_g)$ where f_g is the function from [Eq. \(3.1\)](#) and g is the function $\text{TPG.TDDecide}(\text{td}_{\text{TPG}}, \cdot)$.
- $H_{b,3}(\lambda)$: In this hybrid, instead of encrypting msg_b in the semi-static security game, the challenger encrypts the lexicographically first message $\text{msg}^* \in \mathcal{M}_\lambda$: $\text{ct}^* \leftarrow \text{Enc}(\text{pp}, \text{msg}^*, S^*)$.

Since the challenge bit is sampled uniformly (i.e., $b \xleftarrow{\mathbb{R}} \{0, 1\}$), the success probability of A on any security parameter λ is given by

$$\Pr[A \text{ wins}] = \frac{1}{2} \cdot (\Pr[H_{0,0}(\lambda) = 1] + \Pr[H_{1,0}(\lambda) = 1]).$$

For each $b \in \{0, 1\}$ and $i \in \{1, 2, 3\}$, we define

$$\delta_{b,i}(\lambda) = \Pr[H_{b,i-1}(\lambda) = 1] - \Pr[H_{b,i}(\lambda) = 1].$$

It follows by our assumption that for all $\lambda \in \Lambda$,

$$\begin{aligned} \frac{1}{2} + \frac{1}{q(\lambda)} &\leq \frac{1}{2} \cdot (\Pr[H_{0,3}(\lambda) = 1] + \delta_{0,1}(\lambda) + \delta_{0,2}(\lambda) + \delta_{0,3}(\lambda) \\ &\quad + \Pr[H_{1,3}(\lambda) = 1] + \delta_{1,1}(\lambda) + \delta_{1,2}(\lambda) + \delta_{1,3}(\lambda)). \end{aligned}$$

Since $\Pr[H_{b,3}(\lambda) = 1] \leq 1/2$ for each $b \in \{0, 1\}$, it follows that for all $\lambda \in \Lambda$ that

$$\frac{1}{q(\lambda)} \leq \frac{1}{2} \cdot (\delta_{0,1}(\lambda) + \delta_{0,2}(\lambda) + \delta_{0,3}(\lambda) + \delta_{1,1}(\lambda) + \delta_{1,2}(\lambda) + \delta_{1,3}(\lambda)).$$

As Λ is an infinitely large subset of $\mathbb{N}$, it must be the case that for some $b \in \{0, 1\}$ and $i \in \{1, 2, 3\}$, $\delta_{b,i}(\lambda) \geq 1/(3q(\lambda))$ for infinitely many $\lambda \in \mathbb{N}$. In the following three claims, we show that this must contradict one of our assumptions from the statement of [Theorem 5.18](#). We prove the following claims for $b = 0$ without loss of generality.

Claim 5.19. *If $\delta_{0,1}(\lambda) \geq 1/(3q(\lambda))$ for infinitely many $\lambda \in \mathbb{N}$, then TPG does not satisfy mode indistinguishability.*

Proof. We construct a stateful PPT adversary B that uses the adversary A to break the mode indistinguishability of TPG. We define B as follows:

1. At the beginning of the game, algorithm B receives the public parameters pp_{TPG} for TPG as input from the challenger. Specifically, the mode-indistinguishability challenger samples a bit $\hat{b} \xleftarrow{\mathcal{R}} \{0, 1\}$ and computes $(\text{pp}_{\text{TPG}}, \text{td}) \leftarrow \text{TPG.Setup}(1^\lambda)$ if $\hat{b} = 0$ and $(\text{pp}_{\text{TPG}}, \text{td}) \leftarrow \text{TPG.SetupAlt}(1^\lambda)$ if $\hat{b} = 1$.
2. Algorithm B computes $\text{hk} \leftarrow \text{FBH.Setup}(1^\lambda, n)$ as in $H_{0,0}$ and $H_{0,1}$.
3. Algorithm B starts running adversary A who starts by committing to a set $S \subseteq [n]$. For each $i \in S$, algorithm B makes a false-instance query to its challenger to obtain x_i . For each $i \in [n] \setminus S$, algorithm B makes a true-instance query to its challenger to obtain (x_i, π_i) .
 - If $\hat{b} = 0$, then in both cases, $x_i \xleftarrow{\mathcal{R}} \mathcal{X}_\lambda$ and for the true-instance queries, $\pi_i \leftarrow \text{CreateProof}(\text{td}, x_i)$.
 - If $\hat{b} = 1$, then for the true-instance queries, the challenger samples $(x_i, \pi_i) \leftarrow \text{SampleTrue}(\text{pp}_{\text{TPG}})$ and for the false-instance queries, $x_i \leftarrow \text{SampleFalse}(\text{pp}_{\text{TPG}})$.

Algorithm B gives $\text{pp} = (\text{pp}_{\text{TPG}}, \text{hk}, (x_1, \dots, x_n))$ to A .

4. Whenever algorithm A makes a key-generation query for an index $i \in [n] \setminus S$, algorithm B replies with the secret key $\text{sk}_i = \pi_i$.
5. When algorithm A makes its challenge query on messages $\text{msg}_0, \text{msg}_1$ and a set $S^* \subseteq S$, algorithm B computes $\text{ct}^* \leftarrow \text{Enc}(\text{pp}, \text{msg}_0, S^*)$ as in $H_{0,0}$ and $H_{0,1}$ (using the pp constructed above).
6. Algorithm A responds with a bit $b' \in \{0, 1\}$, which B also outputs.

If A is efficient, then so is B , so it is sufficient to analyze the advantage of B . By construction, if the mode indistinguishability challenger samples $\hat{b} = 0$, then the view of A is distributed exactly as in $H_{0,0}$, and if $\hat{b} = 1$ is sampled, then the view of A is distributed exactly as in $H_{0,1}$. Therefore, when $\hat{b} = 0$, algorithm B succeeds if $b' = 0$ which means that $H_{0,0}(\lambda) = 1$, and when $\hat{b} = 1$, algorithm B succeeds if $b' = 1$ which means that $H_{0,1}(\lambda) = 0$. Thus, the success probability of B is given by

$$\Pr[\hat{b}' = \hat{b}] = \frac{1}{2} \cdot \Pr[H_{0,0}(\lambda) = 1] + \frac{1}{2} \cdot (1 - \Pr[H_{0,1}(\lambda) = 1]) = \frac{1}{2} + \delta_{0,1}(\lambda).$$

If $\delta_{0,1}(\lambda) \geq 1/(3q(\lambda))$ for infinitely many $\lambda \in \mathbb{N}$, then this violates the mode indistinguishability property of TPG. $\square$

Claim 5.20. *If $\delta_{0,2}(\lambda) \geq 1/(3q(\lambda))$ for infinitely many $\lambda \in \mathbb{N}$, then FBH does not satisfy computational function hiding.*

Proof. The proof of this claim follows nearly identically to the proof of [Claim 4.7](#). $\square$

Claim 5.21. *If $\delta_{0,3}(\lambda) \geq 1/(3q(\lambda))$ for infinitely many $\lambda \in \mathbb{N}$, then either (1) WE does not satisfy message indistinguishability; (2) FBH does not satisfy statistical function binding; or (3) TPG does not satisfy trapdoor decidability.*

Proof. We construct a PPT adversary B that uses the adversary A to break the message indistinguishability security of WE or we show that either A breaks the statistical function binding of FBH or trapdoor decidability of TPG. We construct algorithm B as follows:

1. Algorithm B takes 1^λ as input and starts running algorithm A . After algorithm A commits to its set $S \subseteq [n]$, it constructs the public parameters pp exactly as described in $H_{0,3}$. Similarly, it answers key-generation queries according to the specification in $H_{0,3}$.
2. When algorithm A makes its challenge query on messages $\text{msg}_0, \text{msg}_1$ and a set $S^* = \{i_1, \dots, i_k\} \subseteq S$, algorithm B computes $\text{dig} = \text{FBH.Hash}(\text{hk}, (x_{i_1}, \dots, x_{i_k}))$. It then outputs the statement $x = (\text{hk}, \text{pp}_{\text{TPG}}, \text{dig})$ and challenge messages $(\text{msg}_0, \text{msg}^*)$, where msg^* is the lexicographically-first message in $\mathcal{M}_\lambda$.
3. The FBH challenger replies with a challenge ciphertext ct^* which algorithm B forwards to A . Specifically, the witness encryption challenger samples a random bit $\hat{b} \xleftarrow{\mathcal{R}} \{0, 1\}$ and sets $\text{ct}^* \leftarrow \text{WE.Enc}(1^\lambda, \text{msg}_0, x)$ if $\hat{b} = 0$ and $\text{ct}^* \leftarrow \text{WE.Enc}(1^\lambda, \text{msg}^*, x)$ if $\hat{b} = 1$.
4. At the end of the game, algorithm A outputs a bit $b' \in \{0, 1\}$, which B also outputs.

If A is efficient, then so is B , so it suffices to compute the advantage of algorithm B . For any $\lambda \in \mathbb{N}$, let $\text{Wit}(\lambda)$ be the event that there exists a valid witness $(j, \text{hinp}_j, \pi_{\text{TPG}}, \pi_{\text{FBH}})$ for the instance $x = (\text{hk}, \text{pp}_{\text{TPG}}, \text{dig})$ such that

$$\text{TPG.Verify}(\text{pp}_{\text{TPG}}, \text{hinp}_j, \pi_{\text{TPG}}) = 1 \wedge \text{FBH.VerOpen}(\text{hk}, \text{dig}, \{j\}, \{(j, \text{hinp}_j)\}, \pi_{\text{FBH}}) = 1. \quad (5.1)$$

By construction, the view of A is identical to hybrid $H_{0,2}(\lambda)$ if $\hat{b} = 0$ and is identical to hybrid $H_{0,3}(\lambda)$ if $\hat{b} = 1$. When $\hat{b} = 0$, algorithm A outputs $b' = 0$ if $H_{0,2}(\lambda) = 1$ (since $b = 0$). Similarly, when $\hat{b} = 1$, algorithm A outputs $b' = 1$ if $H_{0,3}(\lambda) = 0$. Additionally, if $\text{Wit}(\lambda)$ holds, then B always loses the message indistinguishability game since it implies that $x \in \mathcal{L}_\lambda$. Thus, the success probability of B is at least

$$\begin{aligned} \Pr[B \text{ wins}] &\geq \Pr[\hat{b}' = \hat{b}] - \Pr[\text{Wit}(\lambda)] \\ &\geq \frac{1}{2} \cdot \Pr[H_{0,2}(\lambda) = 1] + \frac{1}{2} \cdot (1 - \Pr[H_{0,3}(\lambda) = 1]) - \Pr[\text{Wit}(\lambda)] \\ &\geq \frac{1}{2} + \frac{1}{2} \cdot \delta_{0,3}(\lambda) - \Pr[\text{Wit}(\lambda)]. \end{aligned}$$

If $\Pr[\text{Wit}(\lambda)] \leq 1/(12q(\lambda))$ for infinitely many $\lambda \in \mathbb{N}$, then this violates message indistinguishability of WE since $\delta_{0,3}(\lambda) \geq 1/(3q(\lambda))$. To complete the proof, we show that if $\Pr[\text{Wit}(\lambda)] > 1/(12q(\lambda))$ for infinitely many $\lambda \in \mathbb{N}$, then we can either break the statistical function binding of FBH or the statistical binding for false instances and trapdoor decidability of TPG. Suppose

$$\Pr[\text{Wit}(\lambda)] > 1/(12q(\lambda)) \quad (5.2)$$

for infinitely many $\lambda \in \mathbb{N}$. This means that there exists a witness $(j, \text{hinp}_j, \pi_{\text{TPG}}, \pi_{\text{FBH}})$ that satisfies [Eq. \(5.1\)](#) with probability at least $1/(12q(\lambda))$ for infinitely many $\lambda \in \mathbb{N}$. In the above construction, $\text{dig} = \text{FBH.Hash}(\text{hk}, (x_{i_1}, \dots, x_{i_k}))$. Recall also that in $H_{0,3}$, the hash key hk is sampled to be function binding for the function f_g where

$$g(x) := \text{TPG.TDDecide}(\text{td}_{\text{TPG}}, x).$$

Let $E(\lambda)$ be the event (taken over the random coins of SetupAlt and SampleFalse) that there exists an index $\ell \in [k]$ such that $g(x_{i_\ell}) = 1$. We consider two cases:

- Suppose for infinitely many $\lambda \in \mathbb{N}$, $\Pr[E(\lambda)] \leq 1/(24q(\lambda))$. Under the assumption that [Eq. \(5.2\)](#), this means that $\Pr[\text{Wit}(\lambda) \wedge \neg E(\lambda)] > 1/(24q(\lambda))$. In this case, for all $\ell \in [k]$, $g(x_{i_\ell}) = 0$, so

$$f_g(x_{i_1}, \dots, x_{i_k}) = \bigvee_{\ell \in [k]} g(x_{i_\ell}) = 0.$$

Since $\text{Wit}(\lambda)$ holds, there exists a witness $(j, \text{hinp}_j, \pi_{\text{TPG}}, \pi_{\text{FBH}})$ that satisfies [Eq. \(5.1\)](#) for $x = (\text{hk}, \text{pp}_{\text{TPG}}, \text{dig})$ from the experiment. This means that $\text{TPG.Verify}(\text{pp}_{\text{TPG}}, \text{hinp}_j, \pi_{\text{TPG}}) = 1$. Then there are two possibilities:

- Suppose $\text{TPG.TDDecide}(\text{td}_{\text{TPG}}, \text{hinp}_j) = 0$. This violates the trapdoor decidability property of TPG.
- Suppose $g(\text{hinp}_j) = \text{TPG.TDDecide}(\text{td}_{\text{TPG}}, \text{hinp}_j) = 1$. Then the output of f_g on any input that contains hinp_j is $1 \neq f_g(x_{i_1}, \dots, x_{i_k}) = 0$. Since $\text{dig} = \text{FBH.Hash}(\text{hk}, (x_{i_1}, \dots, x_{i_k}))$, the existence of an opening π_{FBH} for (j, hinp_j) with respect to dig breaks statistical function binding of FBH.

If $\Pr[\text{Wit}(\lambda) \wedge \neg E(\lambda)] > 1/(24q(\lambda))$, then at least one of the two events above happens with probability at least $1/(48q(\lambda))$. This breaks either statistical function binding of FBH or trapdoor decidability of TPG.

- Suppose for infinitely many $\lambda \in \mathbb{N}$, $\Pr[E(\lambda)] > 1/(24q(\lambda))$. This means that with probability $1/(24q(\lambda))$, there exists an index $\ell \in [k]$ such that $g(x_{i_\ell}) = \text{TPG.TDDecide}(\text{td}_{\text{TPG}}, x_{i_\ell}) = 1$. Since in $H_{0,2}$ and $H_{0,3}$, the challenger samples $x_{i_\ell} \leftarrow \text{SampleFalse}(\text{pp}_{\text{TPG}})$, this translates to an attack on the trapdoor decidability property of TPG with advantage $1/(24q(\lambda) \cdot k(\lambda))$, which is non-negligible since $k \leq n$ is polynomially-bounded.

We conclude that assuming statistical function binding of FBH and trapdoor decidability of TPG, it cannot be the case that $\Pr[\text{Wit}(\lambda)] > 1/(12q(\lambda))$, and the claim follows. $\square$

The proof follows from [Claims 5.19](#) to [5.21](#). $\square$

Remark 5.22 (Optimal Broadcast Encryption in the Random Oracle Model). [Construction 5.16](#) gives a broadcast encryption scheme with short ciphertexts and secret keys. However, the public parameters are long (i.e., scales linearly with the number of users n). Specifically, the public parameters consist of the public parameters pp_{TPG} for the trapdoor proof system, a hash key hk for a function-binding hash function, and n random values $x_1, \dots, x_n \xleftarrow{\mathcal{R}} \mathcal{X}$ from the domain of the trapdoor proof system. By definition, the size of the public parameters pp_{TPG} is $\text{poly}(\lambda)$ and the size of the hash key hk is $\text{poly}(\lambda, \log n)$; see [Theorem 5.17](#) for a more detailed calculation. Since the points $x_1, \dots, x_n$ are uniformly random over $\mathcal{X}$ (and independent of all other parameters), we can “compress” them in the random oracle model. Namely, suppose $\mathcal{O}: [n] \rightarrow \mathcal{X}$ is a hash function modeled as a random oracle. Then, in [Construction 5.16](#), we can define $x_i \leftarrow \mathcal{O}(i)$. In this way, the public parameters only needs to contain pp_{TPG} and hk , both of which are short. This yields an optimal broadcast encryption scheme from witness encryption, function-binding hash functions, and trapdoor proof generators in the random oracle model. The resulting construction satisfies semi-static security, but can be generically boosted to full adaptive security via the Gentry-Waters [\[GW09\]](#) transformation (which also relies on the random oracle model).

6 Registered Attribute-Based Encryption

In this section, we recall the notion of a *slotted* registered attribute-based encryption (ABE) scheme [\[HLWW23\]](#) and then show how to construct it from a witness encryption scheme together with a function binding hash function. To obtain a standard registered ABE scheme (without slots), we can then apply the generic transformation from [\[HLWW23\]](#) (restated in [Theorem A.4](#)). We recall the formal definition of a registered ABE scheme from [\[HLWW23\]](#) in [Appendix A](#). Our (slotted) registered ABE supports general policies over an attribute universe of super-polynomial size, an unbounded number of users, and has a transparent setup. In contrast, the pairing-based construction from [\[HLWW23\]](#) could only support monotone Boolean formula policies over a polynomial-size attribute universe, an a priori bounded number of users, and relied on a *structured* reference string whose size scales quadratically with the number of users. On the flip side, the pairing-based construction from [\[HLWW23\]](#) is adaptively secure whereas our construction is only selectively secure.

6.1 Slotted Registered ABE Definition

In this section, we recall the notion of a *slotted* registered ABE scheme from [HLWW23]. In our setting, each user is associated with an attribute of length $\ell = \ell(\lambda)$ and each ciphertext is associated with a policy $P: \{0, 1\}^{\ell(\lambda)} \rightarrow \{0, 1\}$ taken from some policy space $\mathcal{P} = \{\mathcal{P}_\lambda\}_{\lambda \in \mathbb{N}}$. We say that an attribute x satisfies the policy P if $P(x) = 1$. We now recall the formal definition. We present the definition for the setting with a large (i.e., super-polynomial size) attribute universe and for supporting an arbitrary number of users (c.f., [HLWW23, Remark 6.10]):

Definition 6.1 (Slotted Registered ABE [HLWW23, adapted]). Let $\ell = \ell(\lambda)$ be an attribute length and $\mathcal{P} = \{\mathcal{P}_\lambda\}_{\lambda \in \mathbb{N}}$ be a policy space on ℓ -bit inputs (i.e., $\mathcal{P}_\lambda$ is a set of functions $P: \{0, 1\}^{\ell(\lambda)} \rightarrow \{0, 1\}$). Let $m = m(\lambda)$ and $\mathcal{M} = \{\mathcal{M}_\lambda\}_{\lambda \in \mathbb{N}}$ be a message space with message length m . A *slotted registered attribute-based encryption scheme* for attributes of length ℓ , policy space $\mathcal{P}$, and message space $\mathcal{M}$ consists of polynomial-time algorithms (Setup, KeyGen, Aggregate, Enc, Dec)⁹ with the following syntax:

- $\text{Setup}(1^\lambda, L) \rightarrow \text{pp}$: A probabilistic algorithm that on input a security parameter λ and a number of slots L , outputs public parameters pp . We implicitly assume that pp contains 1^λ and L .
- $\text{KeyGen}(\text{pp}, i) \rightarrow (\text{pk}, \text{sk})$: A probabilistic algorithm that on input the public parameters pp and a slot index $i \in [L]$, outputs a public key pk and a secret key sk .
- $\text{Aggregate}(\text{pp}, (\text{pk}_1, x_1), \dots, (\text{pk}_L, x_L)) \rightarrow (\text{mpk}, \text{hsk}_1, \dots, \text{hsk}_L)$: A deterministic algorithm that on input public parameters pp and a list of public keys with associated attributes $(\text{pk}_1, x_1), \dots, (\text{pk}_L, x_L)$ outputs a master public key mpk and a sequence of helper decryption keys $\text{hsk}_1, \dots, \text{hsk}_L$.
- $\text{Enc}(\text{mpk}, P, \text{msg}) \rightarrow \text{ct}$: A probabilistic algorithm that on input a master public key mpk , an access policy $P \in \mathcal{P}_\lambda$, and a message $\text{msg} \in \mathcal{M}_\lambda$, outputs a ciphertext ct .
- $\text{Dec}(\text{mpk}, \text{hsk}, \text{sk}, \text{ct}) \rightarrow \text{msg}$: A deterministic algorithm that on input a master public key mpk , a helper decryption key hsk , a secret key sk , and a ciphertext ct , outputs a message $\text{msg} \in \mathcal{M}_\lambda \cup \{\perp\}$.

We require that (Setup, KeyGen, Aggregate, Enc, Dec) satisfy the following properties:

- **Compactness:** There exists a polynomial p such that for any $\lambda \in \mathbb{N}$, $L \in \mathbb{N}$, public parameters $\text{pp} \in \text{Supp}(\text{Setup}(1^\lambda, L))$, public/secret key-pairs $(\text{pk}_i, \text{sk}_i) \in \text{Supp}(\text{KeyGen}(\text{pp}, i))$, and attributes $x_i \in \{0, 1\}^{\ell(\lambda)}$ for each $i \in [L]$, and setting $(\text{mpk}, \text{hsk}_1, \dots, \text{hsk}_L) = \text{Aggregate}(\text{pp}, (\text{pk}_1, x_1), \dots, (\text{pk}_L, x_L))$, the following holds:
 - $|\text{mpk}| \leq p(\lambda, \ell, \log L)$, and
 - $|\text{hsk}_i| \leq p(\lambda, \ell, \log L)$ for each $i \in [L]$.

This compactness notion requires that the size of the master public key and the helper decryption keys scale polylogarithmically with the size of the attribute universe ($|\{0, 1\}^\ell| = 2^\ell$). We can define a weaker notion of compactness where the size of the master public key and the helper decryption keys scale polynomially with the size of the attribute universe. This weaker notion is the default in [HLWW23], as the parameters of their pairing-based construction scale polynomially with the size of the attribute universe.

- **Correctness:** For any $\lambda \in \mathbb{N}$, $L \in \mathbb{N}$, $i \in [L]$, message $\text{msg} \in \mathcal{M}_\lambda$, attributes $x_1, \dots, x_L \in \{0, 1\}^{\ell(\lambda)}$, policy $P \in \mathcal{P}_\lambda$ where $P(x_i) = 1$, $\text{pp} \in \text{Supp}(\text{Setup}(1^\lambda, L))$, $(\text{pk}_i, \text{sk}_i) \in \text{Supp}(\text{KeyGen}(\text{pp}, i))$, and any public keys $\{\text{pk}_j\}_{j \neq i \in [L]}$ (which may be correlated with pk_i), let $(\text{mpk}, \text{hsk}_1, \dots, \text{hsk}_L) = \text{Aggregate}(\text{pp}, (\text{pk}_1, x_1), \dots, (\text{pk}_L, x_L))$. Then, it holds that

$$\Pr \left[\begin{array}{l} \text{ct} \leftarrow \text{Enc}(\text{mpk}, P, \text{msg}) \\ \text{msg}' = \text{Dec}(\text{mpk}, \text{hsk}_i, \text{sk}_i, \text{ct}) \end{array} : \text{msg}' = \text{msg} \right] = 1.$$

⁹The definition from [HLWW23] also includes an algorithm $\text{IsValid}(\text{pp}, i, \text{pk}_i) \rightarrow \{0, 1\}$ that checks if a given public key is valid. Our construction does not require this check. However, to match their syntax, we could define it to simply output 1 on any public key of the correct length with respect to the public parameters pp .

- **Adaptive security:** For all stateful PPT adversaries A and efficiently-computable function $L \in \text{poly}(\lambda)$, there exists a negligible function negl such that for all $\lambda \in \mathbb{N}$,

$$\Pr \left[\text{Expt}_{A,L}^{\text{srABE}}(\lambda) = 1 \right] \leq 1/2 + \text{negl}(\lambda),$$

where $\text{Expt}_{A,L}^{\text{srABE}}(\lambda)$ is defined via the following security game between the adversary A and a challenger on common input 1^λ :

- **Setup phase:** The challenger samples $\text{pp} \leftarrow \text{Setup}(1^\lambda, L)$ and sends pp to A . The challenger initializes a counter $\text{ctr} := 0$, a dictionary D , and a set of (corrupted) slot indices $C := \emptyset$.
- **Pre-challenge query phase:** The adversary A can now issue the following queries:
 - * **Key-generation query:** In a key-generation query, the adversary specifies a slot index $i \in [L]$. The challenger increments the counter value $\text{ctr} := \text{ctr} + 1$, samples $(\text{pk}, \text{sk}) \leftarrow \text{KeyGen}(\text{pp}, i)$ and replies with (ctr, pk) to A . The challenger adds the mapping $\text{ctr} \mapsto (i, \text{pk}, \text{sk})$ to the dictionary D .
 - * **Corruption query:** In a corruption query, the adversary specifies a counter value $c \in [\text{ctr}]$. In response, the challenger looks up the tuple $(i, \text{pk}, \text{sk}) := D[c]$ and replies to A with sk .
- **Challenge phase:** For each slot $i \in [L]$, the adversary A specifies a tuple $(c_i, x_i, \text{pk}_i^*)$ where either $c_i \in \{1, \dots, \text{ctr}\}$ to reference a challenger-generated key or $c_i = \perp$ to reference a key outside this set. The adversary also sends a challenge policy $P^* \in \mathcal{P}_\lambda$ and two messages $\text{msg}_0, \text{msg}_1 \in \mathcal{M}_\lambda$. The challenger responds by first constructing pk_i as follows for each $i \in [L]$:
 - * If $c_i \in \{1, \dots, \text{ctr}\}$, then the challenger sets $(i', \text{pk}, \text{sk}) := D[c_i]$. If $i = i'$, then the challenger sets $\text{pk}_i := \text{pk}$. Moreover, if the adversary previously issued a corruption query on counter c_i , then the challenger adds the slot index i to C . Otherwise, if $i \neq i'$, then the experiment halts and outputs 0.
 - * If $c_i = \perp$, the challenger sets $\text{pk}_i := \text{pk}_i^*$ and adds the slot index i to C .

The challenger computes $(\text{mpk}, \text{hsk}_1, \dots, \text{hsk}_L) = \text{Aggregate}(\text{pp}, (\text{pk}_1, x_1), \dots, (\text{pk}_L, x_L))$, samples a random bit $b \leftarrow \{0, 1\}$, known as the *challenge bit*, and replies with the challenge ciphertext $\text{ct}^* \leftarrow \text{Enc}(\text{mpk}, P^*, \text{msg}_b)$.¹⁰
- **Output phase:** At the end of the experiment, the adversary A outputs a bit $b' \in \{0, 1\}$. We say that algorithm A is *admissible* if $P^*(x_i) = 0$ for all $i \in C$ (i.e., the attributes associated with corrupted slots do not satisfy the challenge policy). The experiment outputs 1 if $b' = b$ and A is admissible. Otherwise, the experiment outputs 0.

Selective security. We now consider two versions of selective security that relax the notion of adaptive security defined above. We consider (1) policy-selective security, where the adversary has to declare its challenge policy at the beginning of the security game; and (2) policy-selective security without corruptions where the adversary has to declare its challenge policy at the beginning of the security game, and moreover, it cannot make any corruption queries. Our construction of slotted registered ABE will satisfy the weakest notion of policy-selective security without corruptions. In [Appendix C](#), we show how to adapt the two-key approach from [\[GW09\]](#) to transform a slotted registered ABE scheme that satisfies policy-selective security without corruptions into one that satisfies policy-selective security with corruptions in the random oracle model. We now give a formal definition of our relaxed security definitions:

Definition 6.2 (Policy-Selective Security). We say that a slotted registered ABE scheme ($\text{Setup}, \text{KeyGen}, \text{Aggregate}, \text{Enc}, \text{Dec}$) satisfies policy-selective security if instead of adaptive security, it satisfies the following:

- **Policy-selective security:** For all stateful PPT adversaries A and efficiently-computable function $L \in \text{poly}(\lambda)$, there exists a negligible function negl such that for all $\lambda \in \mathbb{N}$,

$$\Pr \left[\text{Expt}_{A,L}^{\text{srABE,PS}}(\lambda) = 1 \right] \leq 1/2 + \text{negl}(\lambda),$$

¹⁰Note that because Aggregate is deterministic and can be run by A itself, there is no need to additionally provide $(\text{mpk}, \text{hsk}_1, \dots, \text{hsk}_L)$ to A . Similarly, there is no advantage to allowing the adversary to select the challenge policy and messages after seeing the aggregated key.

where $\text{Expt}_{A,L}^{\text{srABE,PS}}(\lambda)$ is the adaptive security game $\text{Expt}_{A,L}^{\text{srABE}}(\lambda)$ with the following modification:

- **Setup phase:** At the beginning of the setup phase, the adversary sends the challenge policy P^* to the challenger. The rest of the setup phase proceeds as in $\text{Expt}_{A,L}^{\text{srABE}}(\lambda)$.

The remainder of $\text{Expt}_{A,L}^{\text{srABE,PS}}(\lambda)$ proceeds exactly as defined in $\text{Expt}_{A,L}^{\text{srABE}}(\lambda)$.

Definition 6.3 (Policy-Selective Security without Corruptions). We say that a slotted registered ABE scheme (Setup, KeyGen, Aggregate, Enc, Dec) satisfies policy-selective security without corruptions if instead of adaptive security, it satisfies the following:

- **Policy-selective security without corruptions:** For all stateful PPT adversaries A and efficiently-computable function $L \in \text{poly}(\lambda)$, there exists a negligible function negl such that for all $\lambda \in \mathbb{N}$,

$$\Pr \left[\text{Expt}_{A,L(\lambda)}^{\text{srABE,PSWC}}(\lambda) = 1 \right] \leq 1/2 + \text{negl}(\lambda),$$

where $\text{Expt}_{A,L}^{\text{srABE,PSWC}}(\lambda)$ is the policy-selective security game $\text{Expt}_{A,L}^{\text{srABE,PS}}(\lambda)$ with the following modifications:

- **Pre-challenger query phase:** Same as $\text{Expt}_{A,L}^{\text{srABE}}(\lambda)$ except the adversary A is not allowed to make any corruption queries.

The remainder of $\text{Expt}_{A,L}^{\text{srABE,PSWC}}(\lambda)$ proceeds exactly as defined in $\text{Expt}_{A,L}^{\text{srABE}}(\lambda)$.

6.2 Constructing Slotted Registered ABE

We now show how to construct a slotted registered ABE scheme from witness encryption, function binding hash functions, and public-key encryption. Our scheme satisfies policy-selective security without corruptions. We show in [Appendix C](#) how to use this to generically transform this scheme into one that satisfies policy-selective security with corruptions in the random oracle model. As was the case for our flexible broadcast encryption scheme ([Construction 4.3](#)), the public parameters for our registered ABE scheme is a uniform random string, and thus, our scheme supports a transparent setup (see [Remark 6.10](#)).

Construction 6.4 (Slotted Registered ABE). Let $\ell = \ell(\lambda)$ be the attribute length and $m = m(\lambda)$ be the message length. Let $s = s(\lambda, \ell)$, $\ell_{\text{blk}} = \ell_{\text{blk}}(\lambda)$, and $d = d(\lambda, \ell)$ be polynomials. Let $\mathcal{M} = \{\mathcal{M}_\lambda\}_{\lambda \in \mathbb{N}}$ be a message space for message of length m . Let $\mathcal{P} = \{\mathcal{P}_\lambda\}_{\lambda \in \mathbb{N}}$ be a set of policies on ℓ -bit attributes (i.e., $\mathcal{P}_\lambda$ consists of functions $P: \{0, 1\}^{\ell(\lambda)} \rightarrow \{0, 1\}$) where each policy $P \in \mathcal{P}_\lambda$ can be implemented by a Boolean circuit of size s and depth d . Our construction of slotted registered ABE relies on the following primitives:

- Let $\text{PKE} = (\text{PKE.KeyGen}, \text{PKE.Enc}, \text{PKE.Dec})$ be a semantically-secure public-key bit encryption scheme where, for $(\text{pk}, \text{sk}) \in \text{Supp}(\text{PKE.KeyGen}(1^\lambda))$, ciphertexts have length at most $\ell_{\text{blk}}(\lambda)$ and decryption can be computed by a circuit of size s and depth d .
- Let $\text{FBH} = (\text{FBH.Setup}, \text{FBH.SetupBinding}, \text{FBH.Hash}, \text{FBH.ProveOpen}, \text{FBH.VerOpen})$ be a function-binding hash for the class $\mathcal{F}$ of disjunctions of block functions for input length $\ell_{\text{blk}}(\lambda) + \ell(\lambda)$, size $2s + 1$, and depth d ([Definition 3.3](#)).
- Let $\text{WE} = (\text{WE.Enc}, \text{WE.Dec})$ be a witness encryption scheme for the language $\mathcal{L} = \{\mathcal{L}_\lambda\}_{\lambda \in \mathbb{N}}$ defined by the relation $\mathbf{R}_{\mathcal{L}}$ as follows. Instances of the language $\mathcal{L}_\lambda$ are of the form $(\text{hk}, \text{pk}_{\text{PKE}}, \text{dig}, P)$ and the relation $\mathbf{R}_{\mathcal{L}}$ is given by

$$\begin{aligned} \mathbf{R}_{\mathcal{L}}((\text{hk}, \text{pk}_{\text{PKE}}, \text{dig}, P), (i, \text{ct}, x, r, \pi)) &= 1 \\ \Leftrightarrow \text{ct} &= \text{PKE.Enc}(\text{pk}_{\text{PKE}}, 1; r) \wedge P(x) = 1 \wedge \text{FBH.VerOpen}(\text{hk}, \text{dig}, \{i\}, \{(i, (\text{ct}, x))\}, \pi) = 1. \end{aligned}$$

We construct a slotted registered ABE scheme $\text{srABE} = (\text{Setup}, \text{KeyGen}, \text{Aggregate}, \text{Enc}, \text{Dec})$ with attribute length ℓ , policy space $\mathcal{P}$, and message space $\mathcal{M}$ as follows:

- **Setup**($1^\lambda, L$): On input the security parameter λ and the number of slots L , the setup algorithm samples $(pk_{PKE}, sk_{PKE}) \leftarrow PKE.KeyGen(1^\lambda)$, $hk \leftarrow FBH.Setup(1^\lambda, L)$, and outputs public parameters $pp = (pk_{PKE}, hk)$.
- **KeyGen**(pp, i): On input the public parameters $pp = (pk_{PKE}, hk)$ and an index $i \in [L]$, the key-generation algorithm samples $r \xleftarrow{\mathcal{R}} \{0, 1\}^\lambda$ and computes $ct = PKE.Enc(pk_{PKE}, 1; r)$. It then outputs the public key $pk = ct$ and secret key $sk = r$.
- **Aggregate**($pp, (pk_1, x_1), \dots, (pk_L, x_L)$): On input the public parameters $pp = (pk_{PKE}, hk)$ and a ordered list of public keys pk_i with associated attributes $x_i \in \{0, 1\}^\ell$, the aggregation algorithm first computes the digest $dig = FBH.Hash(hk, ((pk_1, x_1), \dots, (pk_L, x_L)))$. Then, for each $i \in [L]$, it computes the proof $\pi_i = FBH.ProveOpen(hk, ((pk_1, x_1), \dots, (pk_L, x_L)), \{i\})$. Finally, it outputs the master public key $mpk = (pp, dig)$ and the helper decryption key $hsk_i = (i, \pi_i, pk_i, x_i)$ for each $i \in [L]$.
- **Enc**(mpk, P, msg): On input the master public key $mpk = ((pk_{PKE}, hk), dig)$, a policy $P \in \mathcal{P}_\lambda$, and a message $msg \in \mathcal{M}_\lambda$, the encryption algorithm outputs $ct \leftarrow WE.Enc(1^\lambda, msg, (hk, pk_{PKE}, dig, P))$.
- **Dec**(mpk, hsk, sk, ct): On input the master public key mpk , the helper decryption key $hsk = (i, \pi, pk, S)$, a secret key sk , and a ciphertext ct , the decryption algorithm outputs $msg = WE.Dec(1^\lambda, ct, (i, pk, S, sk, \pi))$.

Correctness and efficiency. We start by showing that [Construction 6.4](#) satisfies (perfect) correctness and compactness.

Theorem 6.5 (Correctness and Efficiency). *Assuming PKE and WE are correct and FBH is complete, then [Construction 6.4](#) is compact and correct.*

Proof. We show each property separately:

- **Compactness:** Compactness follows by efficiency of WE and FBH. Specifically, $|mpk| = |pp| + |dig|$. First, $|pp| \in \text{poly}(\lambda, \log L)$ by construction of Setup. Similarly, $|dig| \in \text{poly}(\lambda, \log L)$ by efficiency of FBH, so the master public key is short. Next, consider the helper decryption keys. For any slot $i \in [L]$, we have $|hsk_i| = |i| + |\pi_i| + |pk_i| + |x_i|$. By construction, $|i| \leq \log L$, $|\pi_i| \in \text{poly}(\lambda, \log L)$ by efficiency of FBH and $|pk_i| \in \text{poly}(\lambda)$ by efficiency of PKE, and $|x_i| = \ell$. Thus, $|hsk_i| \in \text{poly}(\lambda, \ell, \log L)$, as required.
- **Correctness:** Take any security parameter $\lambda \in \mathbb{N}$, number of slots $L \in \mathbb{N}$, index $i \in [L]$, message $msg \in \mathcal{M}_\lambda$, attributes $x_1, \dots, x_L \in \{0, 1\}^\ell$, policy $P \in \mathcal{P}_\lambda$ such that $P(x_i) = 1$. Then, we have the following:
 - Let $pp = (pk_{PKE}, hk) \leftarrow \text{Setup}(1^\lambda, L)$, and for each $i \in [L]$, let $(pk_i, sk_i) \leftarrow \text{KeyGen}(pp, i)$. By construction, this means that $pk_i = PKE.Enc(pk_{PKE}, 1; sk_i)$.
 - Let $(mpk, hsk_1, \dots, hsk_L) = \text{Aggregate}(pp, (pk_1, x_1), \dots, (pk_L, x_L))$ and $ct \leftarrow \text{Enc}(mpk, P, msg)$. By construction, $ct \leftarrow WE.Enc(1^\lambda, msg, (hk, pk_{PKE}, dig, P))$. The encryption algorithm computes the digest as $dig = FBH.Hash(hk, ((pk_1, x_1), \dots, (pk_L, x_L)))$.
 - We argue that $\text{Dec}(mpk, hsk_i, sk_i, ct) = msg$. Specifically, we show that using $hsk_i = (i, \pi_i, pk_i, x_i)$ and sk_i , we can construct a valid witness for the statement $(hk, pk_{PKE}, dig, P) \in \mathcal{L}_\lambda$. Specifically, $(i, pk_i, x_i, sk_i, \pi_i)$ is a valid witness. By construction $\pi_i = FBH.ProveOpen(hk, ((pk_1, x_1), \dots, (pk_L, x_L)), \{i\})$. By completeness of FBH, it follows that $FBH.VerOpen(hk, dig, \{i\}, \{(i, (pk_i, x_i))\}, \pi_i) = 1$. Moreover, $pk_i = PKE.Enc(pk_{PKE}, 1; sk_i)$ and $P(x_i) = 1$ so we conclude that $(i, pk_i, x_i, sk_i, \pi_i)$ is a valid witness. The claim now follows by correctness of WE. $\square$

Selective security. We now show that [Construction 6.4](#) satisfies policy-selective security without corruptions. In [Appendix C](#), we show how to generically transform this scheme into one that satisfies policy-selective security with corruptions in the random oracle mode. The analysis follows a similar structure as the security analysis for our flexible broadcast encryption scheme ([Theorem 4.5](#)).

Theorem 6.6 (Policy-Selective Security without Corruptions). *Assuming PKE is semantically secure and satisfies perfect correctness, FBH satisfies computational function hiding and statistical function binding, and WE satisfies message indistinguishability, then Construction 6.4 satisfies policy-selective security without corruptions.*

Proof. Suppose by way of contradiction that srABE does not satisfy policy-selective security without corruptions. Namely, there exists a stateful PPT adversary A , an efficiently computable function $L \in \text{poly}(\lambda)$, a polynomial q , and an infinite set $\Lambda \subseteq \mathbb{N}$ such that for all $\lambda \in \Lambda$, it holds that

$$\Pr \left[\text{Expt}_{A,L(\lambda)}^{\text{srABE,PSWC}}(\lambda) = 1 \right] \geq 1/2 + 1/q(\lambda).$$

For fixed parameters as above, we define the following sequence of hybrid experiments for each $\lambda \in \mathbb{N}$.

- $H_{b,0}(\lambda)$: This is the experiment $\text{Expt}_{A,L(\lambda)}^{\text{srABE,PSWC}}(\lambda)$ where the challenge bit is fixed to the value b .
- $H_{b,1}(\lambda)$: This hybrid is identical to the previous experiment, except in key generation queries, the challenger samples (pk, sk) such that pk is an encryption of 0. Specifically, the challenger responds to key-generation queries by sampling $(pk, sk) = (ct, r)$ where $r \leftarrow \{0, 1\}^\lambda$ and $ct = \text{PKE.Enc}(pk_{\text{PKE}}, 0; r)$ and replying with pk .
- $H_{b,2}(\lambda)$: In this hybrid, the challenger instead computes $hk \leftarrow \text{FBH.SetupBinding}(1^\lambda, L(\lambda), f_g)$ where f_g is the function from Eq. (3.1) and g is defined as follows

$$g(ct, x) := \begin{cases} 1 & \text{PKE.Dec}(sk_{\text{PKE}}, ct) = 1 \wedge P^*(x) = 1 \\ 0 & \text{otherwise.} \end{cases}$$

Note that the secret key sk_{PKE} and the challenge policy P^* is hard-coded into the description of g .

- $H_{b,3}(\lambda)$: In this hybrid, instead of encrypting msg_b in the policy-selective security game without corruptions, the challenger encrypts the lexicographically-first message $\text{msg}^* \in \mathcal{M}_\lambda$: $ct^* \leftarrow \text{Enc}(\text{mpk}, P^*, \text{msg}^*)$.

Since the challenge bit is sampled uniformly (i.e., $b \xleftarrow{\mathcal{R}} \{0, 1\}$), the success probability of A on any security parameter λ is equal to

$$\Pr[A \text{ wins}] = \frac{1}{2} \cdot (\Pr[H_{0,0}(\lambda) = 1] + \Pr[H_{1,0}(\lambda) = 1]).$$

For each $b \in \{0, 1\}$ and $i \in \{1, 2, 3\}$, we define

$$\delta_{b,i}(\lambda) = \Pr[H_{b,i-1}(\lambda) = 1] - \Pr[H_{b,i}(\lambda) = 1].$$

It follows by our assumption that for all $\lambda \in \Lambda$,

$$\begin{aligned} \frac{1}{2} + \frac{1}{q(\lambda)} &\leq \frac{1}{2} \cdot (\Pr[H_{0,0}(\lambda) = 1] + \Pr[H_{1,0}(\lambda) = 1]) \\ &= \frac{1}{2} \cdot (\Pr[H_{0,3}(\lambda) = 1] + \delta_{0,1}(\lambda) + \delta_{0,2}(\lambda) + \delta_{0,3}(\lambda) \\ &\quad + \Pr[H_{1,3}(\lambda) = 1] + \delta_{1,1}(\lambda) + \delta_{1,2}(\lambda) + \delta_{1,3}(\lambda)). \end{aligned}$$

As $H_{b,3}$ is independent of b , it must be the case that $\Pr[H_{b,3}(\lambda) = 1] \leq 1/2$ for each $b \in \{0, 1\}$. Therefore, for all $\lambda \in \Lambda$,

$$\frac{1}{q(\lambda)} \leq \frac{1}{2} \cdot (\delta_{0,1}(\lambda) + \delta_{0,2}(\lambda) + \delta_{0,3}(\lambda) + \delta_{1,1}(\lambda) + \delta_{1,2}(\lambda) + \delta_{1,3}(\lambda)).$$

As Λ is an infinitely large subset of $\mathbb{N}$, it must be the case that, for some $b \in \{0, 1\}$ and $i \in \{1, 2, 3\}$, $\delta_{b,i}(\lambda) \geq 1/(3q(\lambda))$ for infinitely many $\lambda \in \mathbb{N}$. We show in the following claims that this must contradict one of our assumptions from the statement of Theorem 6.6. We prove the following claims for $b = 0$ without loss of generality.

Claim 6.7. *If $\delta_{0,1}(\lambda) \geq 1/(3q(\lambda))$ for infinitely many $\lambda \in \mathbb{N}$, then PKE does not satisfy semantic security.*

Proof. Let $Q(\lambda)$ be an upper bound on the number of key-generation queries that A makes in the pre-challenge query phase of the policy-selective security game without corruptions. Note that Q is polynomially-bounded since A is efficient. We construct a stateful PPT adversary B that uses A to break the semantic security of PKE:

1. Algorithm B receives a public key $\widehat{\text{pk}}$ for the PKE scheme. As PKE is a bit encryption scheme, B outputs challenge messages $\text{msg}_0 = 0$ and $\text{msg}_1 = 1$.
2. Algorithm B receives a challenge ciphertext $\widehat{\text{ct}}$ as input. The challenger samples $\hat{b} \xleftarrow{\mathcal{R}} \{0, 1\}$ and sets $\widehat{\text{ct}} \leftarrow \text{PKE.Enc}(\widehat{\text{pk}}, \hat{b})$.
 - (a) Algorithm B computes $Q(\lambda)$ and samples a uniformly random index $i^* \leftarrow [Q(\lambda)]$.
 - (b) Algorithm B starts running A , which starts by committing to a challenge policy P^* . Algorithm B then computes the public parameters $\text{pp} = (\text{pk}_{\text{PKE}}, \text{hk})$ as in Setup except using $\text{pk}_{\text{PKE}} := \widehat{\text{pk}}$. Algorithm B sends the public parameters pp to A .
 - (c) For each counter value $\text{ctr} \in [Q(\lambda)]$, algorithm B computes the response pk to A 's key-generation queries for ctr as follows:
 - If $\text{ctr} = i^*$ for an key-generation query, B sets $\text{pk} = \widehat{\text{ct}}$.
 - If $\text{ctr} < i^*$ for an key-generation query, B samples $\text{sk} \xleftarrow{\mathcal{R}} \{0, 1\}^\lambda$ and sets $\text{pk} = \text{PKE.Enc}(\widehat{\text{pk}}, 0; \text{sk})$.
 - If $\text{ctr} > i^*$ for an key-generation query, B samples $\text{sk} \xleftarrow{\mathcal{R}} \{0, 1\}^\lambda$ and sets $\text{pk} = \text{PKE.Enc}(\widehat{\text{pk}}, 1; \text{sk})$.
 - (d) In the challenge phase, algorithm B receives tuples $(c_i, x_i, \text{pk}_i^*)$ from A and constructs the challenge ciphertext ct^* using the procedure described in hybrid $H_{0,0}(\lambda)$. Algorithm B gives ct^* to A .
 - (e) The adversary A outputs a bit $b' \in \{0, 1\}$, which B also outputs.

If A is efficient, then so is B by construction. To analyze the success probability, we define a sequence of sub-hybrids. For each $i \in [0, Q(\lambda)]$, we define $H_{0,0,i}(\lambda)$ as follows:

- $H_{0,0,i}(\lambda)$: Same as $H_{0,0}(\lambda)$ except the challenger answers the first $i - 1$ key-generation queries using the procedure from $H_{0,1}(\lambda)$. More precisely, when $\text{ctr} < i$, the challenger sets $\text{pk} = \text{PKE.Enc}(\text{pk}_{\text{PKE}}, 0; \text{sk})$ for a random $\text{sk} \xleftarrow{\mathcal{R}} \{0, 1\}^\lambda$ (i.e., using the same procedure as in $H_{0,1}(\lambda)$).

Note that hybrid $H_{0,0,0}$ is identical to $H_{0,0}$, and hybrid $H_{0,0,Q(\lambda)}$ is identical to $H_{0,1}$. Therefore, it holds that

$$\begin{aligned} \delta_{0,1}(\lambda) &= \Pr[H_{0,0}(\lambda) = 1] - \Pr[H_{0,1}(\lambda) = 1] \\ &= \sum_{i=1}^{Q(\lambda)} (\Pr[H_{0,0,i-1}(\lambda) = 1] - \Pr[H_{0,0,i}(\lambda) = 1]) \end{aligned}$$

We proceed to analyze the success probability of B conditioned on i^* being equal to a fixed values of i . Note that when $\hat{b} = 1$, the view of A corresponds to hybrid $H_{0,0,i}$ since the public key pk for counter value i is an encryption of 0, and when $\hat{b} = 0$, the view of A corresponds to hybrid $H_{0,0,i-1}$. Since B outputs b' , it follows that if $\hat{b} = 1$, then $b' = \hat{b}$ whenever $b' = 1$, or equivalently, when $H_{0,0,i}(\lambda) = 0$ (i.e., algorithm A loses since b is fixed to 0). Similarly, if $\hat{b} = 0$, then $b' = \hat{b}$ whenever $b' = 0$, or equivalently, when $H_{0,0,i-1}(\lambda) = 1$ (i.e., A wins). The success probability of B for any $i \in [L(\lambda)]$ conditioned on $i^* = i$ is given by the following:

$$\begin{aligned} \Pr[b' = \hat{b} \mid i^* = i] &= \frac{1}{2} \cdot (1 - \Pr[H_{0,0,i}(\lambda) = 1]) + \frac{1}{2} \cdot \Pr[H_{0,0,i-1}(\lambda) = 1] \\ &= \frac{1}{2} + \frac{1}{2} \cdot (\Pr[H_{0,0,i-1}(\lambda) = 1] - \Pr[H_{0,0,i}(\lambda) = 1]). \end{aligned}$$

It follows that the success probability of B , where B chooses a random i^* , is equal to

$$\begin{aligned}
\Pr[b' = \hat{b}] &= \sum_{i=1}^{Q(\lambda)} \Pr[i^* = i] \cdot \Pr[b' = \hat{b} \mid i^* = i] \\
&= \frac{1}{Q(\lambda)} \cdot \sum_{i=1}^{Q(\lambda)} \left(\frac{1}{2} + \frac{1}{2} \cdot (\Pr[H_{0,0,i-1}(\lambda) = 1] - \Pr[H_{0,0,i}(\lambda) = 1]) \right) \\
&= \frac{1}{2} + \frac{1}{2 \cdot Q(\lambda)} \cdot \sum_{i=1}^{Q(\lambda)} (\Pr[H_{0,0,i-1}(\lambda) = 1] - \Pr[H_{0,0,i}(\lambda) = 1]) \\
&= \frac{1}{2} + \frac{1}{2 \cdot Q(\lambda)} \cdot \delta_{0,1}(\lambda).
\end{aligned}$$

As $Q \in \text{poly}(\lambda)$ and $\delta_{0,1}(\lambda) \geq 1/(3q(\lambda))$ for infinitely many $\lambda \in \mathbb{N}$, it follows that B succeeds with inverse polynomial probability for infinitely many $\lambda \in \mathbb{N}$, which violates the semantic security of PKE, as required. $\square$

Claim 6.8. *If $\delta_{0,2}(\lambda) \geq 1/(3q(\lambda))$ for infinitely many $\lambda \in \mathbb{N}$, then FBH does not satisfy computational function hiding.*

Proof. The proof of this claim follows nearly identically to the proof of Claim 4.7. $\square$

Claim 6.9. *If $\delta_{0,3}(\lambda) \geq 1/(3q(\lambda))$ for infinitely many $\lambda \in \mathbb{N}$ and PKE satisfies (perfect) correctness, then either WE does not satisfy message indistinguishability or FBH does not satisfy statistical function binding.*

Proof. We construct a PPT adversary B that uses adversary A to either break the message indistinguishability security of WE, or we show that A can be used to break statistical function binding of FBH. We define B as follows:

1. Algorithm B starts running A . The adversary A sends the challenger policy P^* to B . Algorithm B computes $\text{pp} = (\text{pk}_{\text{PKE}}, \text{hk})$ according to the specification in hybrid $H_{0,2}(\lambda)$ and sends pp to A .
2. Whenever A makes a key-generation query, algorithm B responds as in hybrid $H_{0,2}(\lambda)$.
3. In the challenge phase, algorithm A outputs two messages $\text{msg}'_0, \text{msg}'_1$ and tuples $(c_i, x_i, \text{pk}_i^*)$ for each slot $i \in [L(\lambda)]$. Algorithm B computes public key pk_i for each slot as in hybrid $H_{0,2}(\lambda)$ and computes $\text{dig} = \text{FBH.Hash}(\text{hk}, ((\text{pk}_1, x_1), \dots, (\text{pk}_L, x_L)))$ as done in Aggregate.
4. Algorithm B outputs the statement $(\text{hk}, \text{pk}_{\text{PKE}}, \text{dig}, P^*)$ and a pair of messages $(\text{msg}_0, \text{msg}_1)$ where $\text{msg}_0 = \text{msg}'_0$ and $\text{msg}_1 = \text{msg}'_1$ is the lexicographically first message in $\mathcal{M}_\lambda$.
5. Algorithm B receives a challenge ciphertext ct^* corresponding to $\text{WE.Enc}(1^\lambda, \text{msg}_{\hat{b}}, x)$, where $\hat{b} \xleftarrow{\mathcal{R}} \{0, 1\}$ is sampled by the challenger. Algorithm B gives ct^* to A .
6. At the end of the experiment, algorithm A outputs a bit $b' \in \{0, 1\}$, which B also outputs.

If A is efficient, then so is B . It suffices to analyze the success probability of B . Let $\text{Wit}(\lambda)$ be the event that there exists a valid witness $(j, \text{ct}, x, r, \pi)$ for the instance $(\text{hk}, \text{pk}_{\text{PKE}}, \text{dig}, P^*)$ such that

$$\text{ct} = \text{PKE.Enc}(\text{pk}_{\text{PKE}}, 1; r) \wedge P^*(x) = 1 \wedge \text{FBH.VerOpen}(\text{hk}, \text{dig}, \{j\}, \{(j, (\text{ct}, x))\}, \pi) = 1. \quad (6.1)$$

Note that the view of A is identical to hybrid $H_{0,2}(\lambda)$ if $\hat{b} = 0$ and is identical to $H_{0,3}(\lambda)$ if $\hat{b} = 1$ since $\text{msg}_1 = \text{msg}'_1$. Furthermore, when $\hat{b} = 0$, B wins whenever $b' = 0$, so $H_{0,2}(\lambda) = 1$ since b is fixed to 0. When $\hat{b} = 1$, B wins whenever $b' = 1$, so $H_{0,3}(\lambda) = 0$ since b is fixed to 0. Additionally, if $\text{Wit}(\lambda)$ holds, then B always loses the message indistinguishability game since it implies that $x \in \mathcal{L}_\lambda$. Thus, the success probability of B is at least

$$\begin{aligned}
\Pr[B \text{ wins}] &\geq \Pr[\hat{b}' = \hat{b}] - \Pr[\text{Wit}(\lambda)] \\
&\geq \frac{1}{2} \cdot \Pr[H_{0,2}(\lambda) = 1] + \frac{1}{2} \cdot (1 - \Pr[H_{0,3}(\lambda) = 1]) - \Pr[\text{Wit}(\lambda)] \\
&= \frac{1}{2} + \frac{1}{2} \cdot \delta_{0,3}(\lambda) - \Pr[\text{Wit}(\lambda)]
\end{aligned}$$

If $\Pr[\text{Wit}(\lambda)] \leq 1/(12q(\lambda))$ for infinitely many $\lambda \in \Lambda$, then this violates the message indistinguishability of WE since $\delta_{0,3}(\lambda) \geq 1/(3q(\lambda))$. We next argue below that if $\Pr[\text{Wit}(\lambda)] > 1/(12q(\lambda))$ for infinitely many $\lambda \in \mathbb{N}$, then this violates the statistical function binding of FBH, as required for the statement of the claim. Suppose $\Pr[\text{Wit}(\lambda)] > 1/(12q(\lambda))$ for infinitely many $\lambda \in \mathbb{N}$ (i.e., Eq. (6.1) occurs with probability at least $1/(12q(\lambda))$). In the above construction, $\text{dig} = \text{FBH.Hash}(\text{hk}, ((\text{pk}_1, x_1), \dots, (\text{pk}_{L(\lambda)}, x_{L(\lambda)})))$. For each $i \in [L]$, we have that

$$g(\text{pk}_i, x_i) = 1 \quad \text{if and only if} \quad \text{PKE.Dec}(\text{sk}_{\text{PKE}}, \text{pk}_i) = 1 \wedge P^*(x_i) = 1.$$

By construction of hybrid $H_{0,2}(\lambda)$, it must be the case that either:

- $i \in C$, so $P^*(x_i) = 0$ for admissible A ; or
- $i \notin C$ and corresponds to a key-generation query where $\text{pk}_i \in \text{Supp}(\text{PKE.Enc}(\text{pk}_{\text{PKE}}, 0))$. For this case, perfect correctness of PKE implies that $\text{PKE.Dec}(\text{sk}_{\text{PKE}}, \text{pk}_i) = 0$.

Thus, for all $i \in [L]$, we have that $g(\text{pk}_i, x_i) = 0$, and correspondingly,

$$f_g((\text{pk}_1, x_1), \dots, (\text{pk}_L, x_L)) = \bigvee_{j \in [L]} g(\text{pk}_j, x_j) = 0.$$

Therefore, if Eq. (6.1) is satisfied for some witness $(j, \text{ct}, x, r, \pi)$ corresponding to instance $(\text{hk}, \text{pk}_{\text{PKE}}, \text{dig}, P^*)$, then it must be the case that $g(\text{ct}, x) = 1$. But then the output of f_g on *any* input that contains (ct, x) is necessarily $1 \neq f_g((\text{pk}_1, x_1), \dots, (\text{pk}_L, x_L))$. As such, if $\text{Wit}(\lambda)$ holds with probability at least $1/(12q(\lambda))$ for infinitely many $\lambda \in \mathbb{N}$, then there exists an adversary that break statistical function binding of FBH with the same (non-negligible) advantage. $\square$

This proof follows by combining Claims 6.7 to 6.9. $\square$

Remark 6.10 (Transparent Setup). As as the case with the flexible broadcast encryption scheme from Section 4, the public parameters in Construction 6.4 is pseudorandom with a suitable instantiation of the function-binding hash function and public-key encryption scheme (see Remark 4.9). Correspondingly, we obtain slotted registered ABE scheme for general policies with a *transparent* setup.

Instantiation. Combining our slotted registered ABE scheme from Construction 6.4 with the generic transformation from [HLWW23] (see also Appendix A), we obtain a registered ABE scheme for general policies. We summarize this construction below:

Corollary 6.11 (Registered ABE for General Policies). *Let λ be a security parameter and $\ell = \ell(\lambda)$ be an attribute length parameter. Let $\mathcal{P} = \{\mathcal{P}_\lambda\}_{\lambda \in \mathbb{N}}$ be a set of policies on ℓ -bit attributes where each policy $P \in \mathcal{P}_\lambda$ can be implemented by a Boolean circuit of size at most $s = s(\lambda)$. Let $\mathcal{M} = \{\mathcal{M}_\lambda\}_{\lambda \in \mathbb{N}}$ be a message space for messages of length m . Then, assuming witness encryption for NP and the plain learning with errors (LWE) assumption, there exists a registered ABE scheme for attributes of length ℓ , policy family $\mathcal{P}$, and message space $\mathcal{M}$ with the following properties:*

- The public parameters consists of a uniform random string of length $\text{poly}(\lambda, s)$.
- The running time of key generation is $\text{poly}(\lambda)$.
- The size of the master public key and the size of the helper decryption keys is $\text{poly}(\lambda, s, \log L)$ where L is the number of registered users.
- Both the encryption and decryption algorithms run in time $\text{poly}(\lambda, s, m, \log L)$.

The scheme satisfies policy-selective security without corruptions. In the random oracle model, we can further amplify the scheme to satisfy policy-selective security with corruptions (see Appendix C).

Remark 6.12 (Comparison with [HLWW23]). [Corollary 6.11](#) is the first registered ABE scheme that supports general policies that does not rely on indistinguishability obfuscation. The parameter sizes of [Corollary 6.11](#) are consistent with those obtained from the obfuscation-based construction from [HLWW23]. However, the obfuscation-based construction satisfies *adaptive* security whereas [Corollary 6.11](#) only provides policy-selective security without corruptions. To conclude, we also compare [Corollary 6.11](#) with the pairing-based construction from [HLWW23]:

- **Setup assumptions:** The pairing-based construction relies on a *structured* public parameters whose size scales *quadratically* with the maximum number of registered users. Thus, [HLWW23] requires a trusted setup to generate the public parameters. The public parameters in [Corollary 6.11](#) consist of a uniform random string and thus, the scheme supports a *transparent* setup.
- **Unbounded users:** The pairing-based construction imposes an a priori polynomial bound on the maximum number of registered users L in the system, and moreover, the size of the public parameters, the running time of key generation, and the running time of registration scales with the bound L . [Corollary 6.11](#) supports an arbitrary number of users (e.g., an implicit bound of 2^λ users).
- **Policy family:** The pairing-based construction supports monotone Boolean formulas whereas [Corollary 6.11](#) supports arbitrary circuit policies.
- **Security:** The pairing-based construction follows a dual-system methodology and satisfies adaptive security. In contrast, [Corollary 6.11](#) satisfies the weaker notion of policy-selective security without corruptions (and with corruptions if we work in the random oracle model).

7 Flexible and Distributed Broadcast Encryption from Registered ABE

In this section, we show that it is also possible to *generically* obtain either a flexible broadcast encryption scheme or a distributed broadcast encryption scheme from a registered ABE scheme (defined in [Appendix A](#)). For instance, an alternative approach to obtaining flexible broadcast encryption is to apply our transformation to the registered ABE scheme from [Section 6](#). We do note however that the scheme obtained via our generic transformation satisfies a weaker notion of *static* security ([Definition 7.3](#)) as opposed to the notion of semi-static security satisfied by [Construction 4.3](#). We can boost semi-static security to adaptive security (see [Appendix B](#)) in the random oracle model, but an analogous notion is not known starting from only a statically-secure scheme.

We also note that we can apply our transformation to the previous pairing-based registered ABE scheme from [HLWW23]. For reasons that we discuss below (see [Remark 7.6](#)), our transformation in the pairing-based setting yields the more restricted notion of *distributed* broadcast encryption where each user’s public key must be associated with a unique index.

Transformation overview. As mentioned in [Section 1.1](#), our transformation only requires the registered ABE scheme to support a single attribute and a single “always-accept” policy. The idea is simple:

- The public parameters for the flexible broadcast encryption scheme is simple the public parameters for the registered ABE scheme.
- To join the system, a user generates a public key for the underlying registered ABE scheme.
- To encrypt to a collection of public keys $(pk_1, \dots, pk_k)$, the encrypter simulates the role of the key curator for the underlying registered ABE scheme and derives the master public key mpk associated with registering public keys $pk_1, \dots, pk_k$ (with the dummy attribute). Then, the user encrypts the message with respect to mpk and the “always-accept” policy.
- To decrypt a ciphertext encrypted to $(pk_1, \dots, pk_k)$ using a secret key sk_i for some $i \in [k]$, the decrypter runs the same registration algorithm on the keys $(pk_1, \dots, pk_k)$ and uses it to compute the helper decryption key hsk_i for user i . Using its own secret key sk_i together with hsk_i , the user can now run the decryption algorithm for the registered ABE scheme to recover the message.

Critically, the above transformation requires that users can generate their public keys in the registered ABE scheme *independently* and without coordination. This is the notion satisfied by our registered ABE scheme from [Section 6](#). In some registered ABE schemes such as the pairing-based construction of [\[HLWW23\]](#), user public keys are associated with an index, and functionality requires that different users register public keys to *distinct* indices. In this case, the above transformation is still applicable, except when users join the system, they must generate their key to a specific index and one can only broadcast to a set of public keys where each public key occupies a different slot. This corresponds to the notion of *distributed* broadcast encryption [\[BZ14\]](#). We provide additional discussion in [Remark 7.6](#).

Construction 7.1 (Flexible Broadcast Encryption from Registered ABE). Let $m = m(\lambda)$ and $\mathcal{M} = \{\mathcal{M}_\lambda\}_{\lambda \in \mathbb{N}}$ be a message space with messages of length m . Let $\text{rABE} = (\text{rABE.Setup}, \text{rABE.KeyGen}, \text{rABE.RegPK}, \text{rABE.Enc}, \text{rABE.Update}, \text{rABE.Dec})$ be a registered ABE scheme for 1-bit attributes and the policy family $\mathcal{P} = \{\mathcal{P}_\lambda\}_{\lambda \in \mathbb{N}}$ where $\mathcal{P}_\lambda = \{P_{\text{on}}\}$ and $P_{\text{on}}: \{0, 1\} \rightarrow \{1\}$ is the “always-accept” policy. We additionally require that rABE support *stateless* key generation ([Remark A.5](#)). We construct a flexible broadcast encryption scheme $\text{FBE} = (\text{Setup}, \text{KeyGen}, \text{Enc}, \text{Dec})$ with message space $\mathcal{M}$ as follows:

- $\text{Setup}(1^\lambda, n)$: On input the security parameter λ and a bound on the number of recipients n , the setup algorithm samples and outputs $\text{pp} \leftarrow \text{rABE.Setup}(1^\lambda)$.
- $\text{KeyGen}(\text{pp})$: On input the public parameters pp , the key-generation algorithm samples and outputs $(\text{pk}, \text{sk}) \leftarrow \text{rABE.KeyGen}(\text{pp})$. Here, we assume that rABE supports stateless key generation ([Remark A.5](#)) so rABE.KeyGen only takes pp as input.
- $\text{Enc}(\text{pp}, \text{msg}, (\text{pk}_1, \dots, \text{pk}_k))$: On input the public parameters pp , a message $\text{msg} \in \mathcal{M}_\lambda$, and a list of public keys $(\text{pk}_1, \dots, \text{pk}_k)$, the encryption algorithm does the following:
 - Initializes an empty master public key and auxiliary input $(\text{mpk}, \text{aux}) := (\perp, \perp)$.
 - It now registers each public key pk_i with the dummy attribute 1. Specifically, for each $i \in [k]$, it computes $(\text{mpk}, \text{aux}) := (\text{mpk}', \text{aux}') = \text{rABE.RegPK}(\text{pp}, \text{aux}, \text{pk}_i, 1)$.
 - Finally, it encrypts the message with respect to the aggregated master public key and the “always-accept” policy: $\text{ct} \leftarrow \text{rABE.Enc}(\text{mpk}, P_{\text{on}}, \text{msg})$.
- $\text{Dec}(\text{pp}, \text{ct}, (j, \text{sk}_j), (\text{pk}_1, \dots, \text{pk}_k))$: On input the public parameters pp , a ciphertext ct , an index/secret-key pair (j, sk_j) , and a list of public keys $(\text{pk}_1, \dots, \text{pk}_k)$, the decryption algorithm does the following:
 - Initializes an empty master public key and auxiliary input $(\text{mpk}, \text{aux}) := (\perp, \perp)$. Like Enc , the decryption algorithm now registers each public key pk_i with the dummy attribute 1. Specifically, for each $i \in [k]$, it computes $(\text{mpk}, \text{aux}) := (\text{mpk}', \text{aux}') = \text{rABE.RegPK}(\text{pp}, \text{aux}, \text{pk}_i, 1)$.
 - Compute the helper decryption key $\text{hsk} = \text{rABE.Update}(\text{pp}, \text{aux}, \text{pk}_j)$ for public key pk_j .
 - Output $\text{msg} = \text{rABE.Dec}(\text{mpk}, \text{hsk}, \text{sk}_j, \text{ct})$.

Correctness and efficiency. We now show [Construction 7.1](#) is correct and satisfies the efficiency requirements on a flexible broadcast encryption scheme.

Theorem 7.2 (Correctness and Efficiency). *Assuming rABE is efficient, compact, and correct, then [Construction 7.1](#) has succinct ciphertexts and is correct.*

Proof. We consider each property separately:

- **Succinct ciphertexts:** Consider a master public key mpk and ciphertext ct constructed by the encryption algorithm Enc . By compactness of rABE , we have that $|\text{mpk}| \in \text{poly}(\lambda, \ell, \log k)$. Since $\ell = 1$ and $|P_{\text{on}}| \in O(1)$, it follows that $|\text{ct}| \in \text{poly}(\lambda, m(\lambda), \log k)$, as required.
- **Correctness:** Take any $\lambda \in \mathbb{N}$, $n \in \text{poly}(\lambda)$, $k \leq n(\lambda)$, $j \in [k]$, and $\text{msg} \in \mathcal{M}_\lambda$. Then, we have the following:

- Let $pp \leftarrow \text{Setup}(1^\lambda, n(\lambda))$, and for $i \in [k]$, sample $(pk_i, sk_i) \leftarrow \text{KeyGen}(pp)$. By construction, pp and (pk_i, sk_i) are sampled from $\text{rABE.Setup}(1^\lambda)$ and $\text{rABE.KeyGen}(pp)$, respectively.
- Let $ct \leftarrow \text{Enc}(pp, \text{msg}, (pk_1, \dots, pk_k))$. By construction, it follows that $ct \leftarrow \text{rABE.Enc}(mpk, P_{\text{on}}, \text{msg})$ where mpk is derived from a series of calls to RegPK with each public key pk_i with the dummy attribute 1.
- We now argue that $\text{Dec}(pp, ct, (j, sk_j), (pk_1, \dots, pk_k)) = \text{msg}$. As rABE.RegPK is deterministic, Enc and Dec compute the same master public key mpk . As a result, if Dec does not output msg , that violates the correctness of rABE . So, there exists a negligible function negl such that Dec output msg with probability at least $1 - \text{negl}(\lambda)$. Moreover, if rABE satisfies perfect correctness, then so does FBE . $\square$

Static security. The flexible broadcast scheme in [Construction 7.1](#) satisfies a weaker notion of *static* security, where the adversary has to commit to the exact set of keys in the challenge ciphertext; this is the analog of selective security in the context of traditional broadcast encryption. Static security is a weaker property than semi-static security ([Definition 4.2](#)) where the adversary is able to choose which subset of honestly-generated keys it can use in the challenge ciphertext. We formally define static security for flexible broadcast encryption and then show that [Construction 7.1](#) is statically secure in [Theorem 7.4](#).

Definition 7.3 (Static Security). We say that a flexible broadcast encryption scheme $(\text{Setup}, \text{KeyGen}, \text{Enc}, \text{Dec})$ satisfies static security if instead of adaptive security, it satisfies the following:

- **Static security:** For all stateful PPT adversaries A and all efficiently-computable functions $n \in \text{poly}(\lambda)$, there exists a negligible function negl such that for all $\lambda \in \mathbb{N}$,

$$\Pr \left[\text{Expt}_{A,n}^{\text{FBE,Static}}(\lambda) = 1 \right] \leq 1/2 + \text{negl}(\lambda),$$

where $\text{Expt}_{A,n}^{\text{FBE,Static}}(\lambda)$ is defined via the following security game between the adversary A and a challenger on common input 1^λ :

- **Setup phase:** The adversary starts by choosing an integer $k \leq n$. The challenger samples $pp \leftarrow \text{Setup}(1^\lambda, n)$ and samples key-pairs $(pk_i, sk_i) \leftarrow \text{KeyGen}(pp)$ for each $i \in [k]$. It gives pp and $(pk_1, \dots, pk_k)$ to A .
- **Challenge phase:** Algorithm A outputs two messages $\text{msg}_0, \text{msg}_1 \in \mathcal{M}_\lambda$. The challenger samples a bit $b \xleftarrow{\mathcal{R}} \{0, 1\}$ and computes an encryption of msg_b under the set $S^* = (pk_1, \dots, pk_k)$: $ct^* \leftarrow \text{Enc}(pp, \text{msg}_b, (pk_1, \dots, pk_k))$. The challenger gives ct^* to A .
- **Output phase:** Algorithm A outputs a bit $b' \in \{0, 1\}$. The experiment outputs 1 if $b' = b$ and 0 otherwise.

Theorem 7.4 (Static Security). Assuming rABE satisfies policy-selective security without corruption queries, then [Construction 7.1](#) satisfies static security.

Proof. Suppose by way of contradiction that FBE does not satisfy static security. Namely, there exists a stateful PPT adversary A , an efficiently computable function $n \in \text{poly}(\lambda)$, and a polynomial q such that for infinitely many $\lambda \in \mathbb{N}$, it holds that

$$\Pr \left[\text{Expt}_{A,n(\lambda)}^{\text{FBE,Static}}(\lambda) = 1 \right] > 1/2 + 1/q(\lambda).$$

We construct an adversary B that breaks policy-selective security without corruption queries of rABE :

- **Setup phase:** Algorithm B sends the unique challenge policy $P^* = P_{\text{on}} \in \mathcal{P}_\lambda$ to the rABE challenger. Algorithm B then receives the public parameters pp from the challenger, which B then forwards to A . The adversary A specifies a number of keys $k \leq n(\lambda)$ that it will generate.
- **Pre-challenge query phase:** The adversary A makes k key-generation queries. On the i^{th} query, algorithm B makes a corresponding register-honest-key query to the rABE challenger for attribute set 1 to obtain a public key pk_i . Algorithm B sends pk_i to A .

- **Challenge phase:** In the challenge phase, A sends two messages $\text{msg}_0, \text{msg}_1$. Algorithm B sends the same messages $\text{msg}_0, \text{msg}_1$ to its challenger. The challenger samples a bit $b \xleftarrow{R} \{0, 1\}$ and replies with the challenge ciphertext $\text{ct}^* \leftarrow \text{Enc}(\text{mpk}, P^*, \text{msg}_b)$, which B forwards to A .
- **Output phase:** The adversary A outputs a guess b' for b , which B outputs.

If A is efficient, then so is B . To analyze the success probability of B , we observe that by construction of B and the definition of Enc , the challenge ciphertext ct^* defined above is distributed exactly as the challenge ciphertext in the flexible broadcast encryption static security game. Specifically, this is because mpk is the result of registering keys $\text{pk}_1, \dots, \text{pk}_k$ in order, where each public key is registered with the attribute 1, and $P^* = P_{\text{on}}$ is the only allowable policy. Therefore, if A succeeds in guessing $b' = b$ with probability $1/2 + 1/q(\lambda)$, then so does B . $\square$

Remark 7.5 (On Adaptively-Secure Flexible Broadcast from Registered ABE). While we only know how to prove static security for [Construction 7.1](#), we believe this is mostly a caveat of the registered ABE definition from [\[HLWW23\]](#). Namely, the [\[HLWW23\]](#) security definition does not allow for more adaptive capabilities where the adversary can request that the challenger generate (honest) public keys and then *adaptively* choose which public keys it would like to register. It is interesting to properly define such a stronger notion, and it seems plausible that existing constructions can be adapted to satisfy this stronger property. We leave further exploration of the security notions underlying registered ABE to future work.

Remark 7.6 (Distributed Broadcast Encryption from Pairings). Correctness of [Construction 7.1](#) critically relies on the underlying registered ABE scheme supporting *stateless* key-generation ([Remark A.5](#)). This is because the subset of keys that are registered is determined at *encryption* time (rather than key-generation time); moreover, the scheme needs to support registering to any subset of the user-generated keys. Thus, if the scheme requires that users generate keys in a stateful manner, where each key can depend arbitrarily on the set of previously-registered keys, then [Construction 7.1](#) is no longer correct. While [Construction 6.4](#) (in conjunction with [Theorem A.4](#)) yields a registered ABE with stateless key generation, the previous pairing-based constructions of registered ABE [\[HLWW23\]](#) have a *stateful* key-generation procedure, and thus [Construction 7.1](#) cannot be directly applied to [\[HLWW23\]](#) to obtain a pairing-based flexible broadcast encryption scheme.

Nonetheless, we observe that the previous pairing-based registered ABE scheme of [\[HLWW23\]](#) has the property that key generation does not depend *arbitrarily* on the auxiliary input. Instead, the key-generation algorithm only demands knowledge of an index, with the stipulation that every user has a distinct index. For instance, the index could be the current count on the number of registered users in the system. This means we can apply [Construction 7.1](#) to the pairing-based registered ABE scheme to obtain a *distributed* broadcast encryption scheme [\[BZ14\]](#). In a distributed broadcast encryption scheme, each user is associated with a unique index and when users generate their key, they generate it with respect to their particular index. This coincides with the semantics of the pairing-based registered ABE scheme from [\[HLWW23\]](#). Thus, combining [Construction 7.1](#) with the construction of [\[HLWW23\]](#), we obtain the first distributed broadcast encryption scheme using composite-order bilinear groups. The resulting construction supports an a priori bounded number of users and requires a structured reference string of size $n^2 \cdot \text{polylog}(n)$, where n is the bound on the number of users. The scheme satisfies static security (analogous to [Definition 7.3](#) and [Theorem 7.4](#)).

Acknowledgments

We thank Dan Boneh and Hoeteck Wee for helpful pointers on broadcast encryption. We thank George Lu and Shafik Nassar for helpful comments on the exposition. Cody Freitag’s work was done while at Cornell Tech, and he is supported by the National Science Foundation Graduate Research Fellowship under Grant No. DGE-2139899, DARPA Award HR00110C0086, AFOSR Award FA9550-18-1-0267, NSF CNS-2128519, and DARPA under Agreement No. HR00112020023. Brent Waters is supported by NSF CNS-1908611, a Simons Investigator award, and the Packard Foundation Fellowship. David J. Wu is supported by NSF CNS-2151131, CNS-2140975, a Microsoft Research Faculty Fellowship, and a Google Research Scholar award.

References

- [AL18] Prabhanjan Ananth and Alex Lombardi. Succinct garbling schemes from functional encryption through a local simulation paradigm. In *TCC*, 2018.
- [AWY20] Shweta Agrawal, Daniel Wichs, and Shota Yamada. Optimal broadcast encryption from LWE and pairings in the standard model. In *TCC*, 2020.
- [AY20] Shweta Agrawal and Shota Yamada. Optimal broadcast encryption from pairings and LWE. In *EUROCRYPT*, 2020.
- [BBK⁺23] Zvika Brakerski, Maya Farber Brodsky, Yael Tauman Kalai, Alex Lombardi, and Omer Paneth. SNARGs for monotone policy batch NP. In *CRYPTO*, 2023.
- [BGI⁺01] Boaz Barak, Oded Goldreich, Russell Impagliazzo, Steven Rudich, Amit Sahai, Salil P. Vadhan, and Ke Yang. On the (im)possibility of obfuscating programs. In *CRYPTO*, 2001.
- [BGI⁺17] Saikrishna Badrinarayanan, Sanjam Garg, Yuval Ishai, Amit Sahai, and Akshay Wadia. Two-message witness indistinguishability and secure computation in the plain model from new assumptions. In *ASIACRYPT*, 2017.
- [BGV12] Zvika Brakerski, Craig Gentry, and Vinod Vaikuntanathan. (leveled) fully homomorphic encryption without bootstrapping. In *ITCS*, 2012.
- [BGW05] Dan Boneh, Craig Gentry, and Brent Waters. Collusion resistant broadcast encryption with short ciphertexts and private keys. In *CRYPTO*, 2005.
- [BIOW20] Ohad Barta, Yuval Ishai, Rafail Ostrovsky, and David J. Wu. On succinct arguments and witness encryption from groups. In *CRYPTO*, 2020.
- [BISW18] Dan Boneh, Yuval Ishai, Amit Sahai, and David J. Wu. Quasi-optimal SNARGs via linear multi-prover interactive proofs. In *EUROCRYPT*, 2018.
- [BJK⁺18] Zvika Brakerski, Aayush Jain, Ilan Komargodski, Alain Passelègue, and Daniel Wichs. Non-trivial witness encryption and null-iO from standard assumptions. In *SCN*, 2018.
- [BPR15] Nir Bitansky, Omer Paneth, and Alon Rosen. On the cryptographic hardness of finding a Nash equilibrium. In *FOCS*, 2015.
- [Bra12] Zvika Brakerski. Fully homomorphic encryption without modulus switching from classical GapSVP. In *CRYPTO*, 2012.
- [BSW11] Dan Boneh, Amit Sahai, and Brent Waters. Functional encryption: Definitions and challenges. In *TCC*, pages 253–273, 2011.
- [BV11] Zvika Brakerski and Vinod Vaikuntanathan. Efficient fully homomorphic encryption from (standard) LWE. In *FOCS*, 2011.
- [BV22] Zvika Brakerski and Vinod Vaikuntanathan. Lattice-inspired broadcast encryption and succinct ciphertext-policy ABE. In *ITCS*, 2022.
- [BW06] Dan Boneh and Brent Waters. A fully collusion resistant broadcast, trace, and revoke system. In *ACM CCS*, 2006.
- [BW07] Dan Boneh and Brent Waters. Conjunctive, subset, and range queries on encrypted data. In *TCC*, pages 535–554, 2007.

- [BW13] Dan Boneh and Brent Waters. Constrained pseudorandom functions and their applications. In *ASIACRYPT*, 2013.
- [BWZ14] Dan Boneh, Brent Waters, and Mark Zhandry. Low overhead broadcast encryption from multilinear maps. In *CRYPTO*, 2014.
- [BZ14] Dan Boneh and Mark Zhandry. Multiparty key exchange, efficient traitor tracing, and more from indistinguishability obfuscation. In *CRYPTO*, 2014.
- [CES21] Kelong Cong, Karim Eldefrawy, and Nigel P. Smart. Optimizing registration based encryption. In *IMACC*, 2021.
- [CGW15] Jie Chen, Romain Gay, and Hoeteck Wee. Improved dual system ABE in prime-order groups via predicate encodings. In *EUROCRYPT*, 2015.
- [CH85] Stephen A. Cook and H. James Hoover. A depth-universal circuit. *SIAM J. Comput.*, 14(4), 1985.
- [CHK03] Ran Canetti, Shai Halevi, and Jonathan Katz. A forward-secure public-key encryption scheme. In *EUROCRYPT*, 2003.
- [CLP15] Kai-Min Chung, Huijia Lin, and Rafael Pass. Constant-round concurrent zero-knowledge from indistinguishability obfuscation. In *CRYPTO*, 2015.
- [CPP20] Ran Canetti, Sunoo Park, and Oxana Poburinnaya. Fully deniable interactive encryption. In *CRYPTO*, 2020.
- [CVW18] Yilei Chen, Vinod Vaikuntanathan, and Hoeteck Wee. GGH15 beyond permutation branching programs: Proofs, attacks, and candidates. In *CRYPTO*, 2018.
- [DF02] Yevgeniy Dodis and Nelly Fazio. Public key broadcast encryption for stateless receivers. In *ACM CCS*, 2002.
- [EFPK20] Naomi Ephraim, Cody Freitag, Ilan Komargodski, and Rafael Pass. Sparks: Succinct parallelizable arguments of knowledge. In *EUROCRYPT*, 2020.
- [FN93] Amos Fiat and Moni Naor. Broadcast encryption. In *CRYPTO*, 1993.
- [FNV17] Antonio Faonio, Jesper Buus Nielsen, and Daniele Venturi. Predictable arguments of knowledge. In *PKC*, 2017.
- [Gam84] Taher El Gamal. A public key cryptosystem and a signature scheme based on discrete logarithms. In *CRYPTO*, 1984.
- [Gen09] Craig Gentry. *A fully homomorphic encryption scheme*. PhD thesis, Stanford University, USA, 2009.
- [GGH⁺13] Sanjam Garg, Craig Gentry, Shai Halevi, Mariana Raykova, Amit Sahai, and Brent Waters. Candidate indistinguishability obfuscation and functional encryption for all circuits. In *FOCS*, 2013.
- [GGHR14] Sanjam Garg, Craig Gentry, Shai Halevi, and Mariana Raykova. Two-round secure MPC from indistinguishability obfuscation. In *TCC*, 2014.
- [GGSW13] Sanjam Garg, Craig Gentry, Amit Sahai, and Brent Waters. Witness encryption and its applications. In *STOC*, 2013.
- [GHM⁺19] Sanjam Garg, Mohammad Hajiabadi, Mohammad Mahmoody, Ahmadreza Rahimi, and Sruthi Sekar. Registration-based encryption from standard assumptions. In *PKC*, 2019.
- [GHMR18] Sanjam Garg, Mohammad Hajiabadi, Mohammad Mahmoody, and Ahmadreza Rahimi. Registration-based encryption: Removing private-key generator from IBE. In *TCC*, 2018.

- [GKMR23] Noemi Glaeser, Dimitris Kolonelos, Giulio Malavolta, and Ahmadreza Rahimi. Efficient registration-based encryption. In *ACM CCS*, 2023.
- [GKP⁺13] Shafi Goldwasser, Yael Tauman Kalai, Raluca A. Popa, Vinod Vaikuntanathan, and Nickolai Zeldovich. How to run turing machines on encrypted data. In *CRYPTO*, 2013.
- [GKW17] Rishab Goyal, Venkata Koppula, and Brent Waters. Lockable obfuscation. In *FOCS*, 2017.
- [GLW14] Craig Gentry, Allison B. Lewko, and Brent Waters. Witness encryption from instance independent assumptions. In *CRYPTO*, 2014.
- [GMM17] Sanjam Garg, Mohammad Mahmoody, and Ameer Mohammed. Lower bounds on obfuscation from all-or-nothing encryption primitives. In *CRYPTO*, 2017.
- [GOS06] Jens Groth, Rafail Ostrovsky, and Amit Sahai. Perfect non-interactive zero knowledge for NP. In *EUROCRYPT*, 2006.
- [GPSW06] Vipul Goyal, Omkant Pandey, Amit Sahai, and Brent Waters. Attribute-based encryption for fine-grained access control of encrypted data. In *ACM CCS*, 2006.
- [GSW13] Craig Gentry, Amit Sahai, and Brent Waters. Homomorphic encryption from learning with errors: Conceptually-simpler, asymptotically-faster, attribute-based. In *CRYPTO*, 2013.
- [GV20] Rishab Goyal and Satyanarayana Vusirikala. Verifiable registration-based encryption. In *CRYPTO*, 2020.
- [GVW19] Rishab Goyal, Satyanarayana Vusirikala, and Brent Waters. Collusion resistant broadcast and trace from positional witness encryption. In *PKC*, 2019.
- [GW09] Craig Gentry and Brent Waters. Adaptive security in broadcast encryption systems (with short ciphertexts). In *EUROCRYPT*, 2009.
- [HLWW23] Susan Hohenberger, George Lu, Brent Waters, and David J. Wu. Registered attribute-based encryption. In *EUROCRYPT*, 2023.
- [HS02] Dani Halevy and Adi Shamir. The LSD broadcast encryption scheme. In *CRYPTO*, 2002.
- [HW15] Pavel Hubáček and Daniel Wichs. On the communication complexity of secure function evaluation with long output. In *ITCS*, 2015.
- [JLS21] Aayush Jain, Huijia Lin, and Amit Sahai. Indistinguishability obfuscation from well-founded assumptions. In *STOC*, 2021.
- [KRR17] Yael Tauman Kalai, Guy N. Rothblum, and Ron D. Rothblum. From obfuscation to the security of fiat-shamir for proofs. In *CRYPTO*, 2017.
- [KSW08] Jonathan Katz, Amit Sahai, and Brent Waters. Predicate encryption supporting disjunctions, polynomial equations, and inner products. In *EUROCRYPT*, pages 146–162, 2008.
- [Mer89] Ralph C. Merkle. A certified digital signature. In *CRYPTO*, 1989.
- [NNL01] Dalit Naor, Moni Naor, and Jeffery Lotspiech. Revocation and tracing schemes for stateless receivers. In *CRYPTO*, 2001.
- [PS19] Chris Peikert and Sina Shiehian. Noninteractive zero knowledge for NP from (plain) learning with errors. In *CRYPTO*, 2019.
- [Reg05] Oded Regev. On lattices, learning with errors, random linear codes, and cryptography. In *STOC*, 2005.
- [SW05] Amit Sahai and Brent Waters. Fuzzy identity-based encryption. In *EUROCRYPT*, 2005.

- [SW14] Amit Sahai and Brent Waters. How to use indistinguishability obfuscation: deniable encryption, and more. In *STOC*, 2014.
- [Tsa22] Rotem Tsabary. Candidate witness encryption from lattice techniques. In *CRYPTO*, 2022.
- [VWW22] Vinod Vaikuntanathan, Hoeteck Wee, and Daniel Wichs. Witness encryption and null-io from evasive LWE. In *ASIACRYPT*, 2022.
- [Wee21] Hoeteck Wee. Broadcast encryption with size $n^{1/3}$ and more from k-lin. In *CRYPTO*, 2021.
- [Wee22] Hoeteck Wee. Optimal broadcast encryption and CP-ABE from evasive lattice assumptions. In *EUROCRYPT*, 2022.
- [WZ17] Daniel Wichs and Giorgos Zirdelis. Obfuscating compute-and-compare programs under LWE. In *FOCS*, 2017.

A Registered Attributed-Based Encryption

In this section, we recall the full notion of registered attribute-based encryption (ABE) from [HLWW23]. For simplicity of notation, we describe our notion for the setting where the attribute is an arbitrary bit-string.

Definition A.1 (Registered Attributed-Based Encryption Syntax [HLWW23, adapted]). Let $\ell = \ell(\lambda)$ be an attribute length and $\mathcal{P} = \{\mathcal{P}_\lambda\}_{\lambda \in \mathbb{N}}$ be a set of policies on ℓ -bit inputs (i.e., $\mathcal{P}_\lambda$ is a set of functions $P: \{0, 1\}^{\ell(\lambda)} \rightarrow \{0, 1\}$). Let $m = m(\lambda)$ and $\mathcal{M} = \{\mathcal{M}_\lambda\}_{\lambda \in \mathbb{N}}$ be a message space with message length m . A *registered attributed-based encryption scheme* for attributes of length ℓ , policy space $\mathcal{P}$, and message space $\mathcal{M}$ consists of polynomial-time algorithms (Setup, KeyGen, RegPK, Enc, Update, Dec) with the following syntax:

- $\text{Setup}(1^\lambda) \rightarrow \text{pp}$: A probabilistic algorithm that on input the security parameter λ , outputs the public parameters pp. We implicitly assume that pp contains 1^λ .
- $\text{KeyGen}(\text{pp}, \text{aux}) \rightarrow (\text{pk}, \text{sk})$: A probabilistic algorithm that on input the public parameters pp and a (possibly empty) auxiliary state aux, outputs a public key pk and a secret key sk.
- $\text{RegPK}(\text{pp}, \text{aux}, \text{pk}, x) \rightarrow (\text{mpk}, \text{aux}')$: A deterministic algorithm that on input the public parameters pp, a (possibly empty) auxiliary state aux, a public key pk, and an attribute $x \in \{0, 1\}^{\ell(\lambda)}$, outputs a master public key mpk and an updated state aux'.
- $\text{Enc}(\text{mpk}, P, \text{msg}) \rightarrow \text{ct}$: A probabilistic algorithm that on input a master public key mpk, a policy $P \in \mathcal{P}_\lambda$, and a message $\text{msg} \in \mathcal{M}_\lambda$, outputs a ciphertext ct.
- $\text{Update}(\text{pp}, \text{aux}, \text{pk}) \rightarrow \text{hsk}$: A deterministic algorithm that on input the public parameters pp, a (possibly empty) auxiliary state aux, and a public key pk, outputs a helper decryption key hsk.
- $\text{Dec}(\text{mpk}, \text{hsk}, \text{sk}, \text{ct}) \rightarrow \text{msg}$: A deterministic algorithm that on input a master public key mpk, a helper decryption key hsk, a secret key sk, and a ciphertext ct, outputs either (1) a message $\text{msg} \in \mathcal{M}_\lambda$; (2) a special symbol $\perp$ indicating decryption failure; or (3) a special flag GetUpdate that indicates an updated helper decryption key is needed to decrypt.

Definition A.2 (Registered ABE Correctness and Efficiency). Let $\text{rABE} = (\text{Setup}, \text{KeyGen}, \text{RegPK}, \text{Enc}, \text{Update}, \text{Dec})$ be a registered ABE scheme for attributes of length $\ell = \ell(\lambda)$, policy space $\mathcal{P} = \{\mathcal{P}_\lambda\}_{\lambda \in \mathbb{N}}$, and message space $\mathcal{M} = \{\mathcal{M}_\lambda\}_{\lambda \in \mathbb{N}}$. For a security parameter λ and a stateful adversary A , we define the following experiment between A and a challenger on common input 1^λ :

- **Setup phase:** The challenger samples public parameters $\text{pp} \leftarrow \text{Setup}(1^\lambda)$. It initializes its auxiliary state $\text{aux} := \perp$ and a master public key $\text{mpk} := \perp$. It also initializes a counter $\text{ctr}[\text{reg}] := 0$ to keep track of the number of registration queries and another counter $\text{ctr}[\text{enc}] := 0$ to keep track of the number of encryption queries. Finally, it initializes $\text{ctr}[\text{reg}]^* := \infty$ as the index for the target key (which will be updated during the course of the experiment). The challenger sends pp to A .

- **Query phase:** During the query phase, the adversary A is able to make the following queries:
 - **Register non-target key query:** In a non-target key-registration query, A specifies a public key pk and an attribute $x \in \{0, 1\}^\ell$. The challenger increments $\text{ctr}[\text{reg}] := \text{ctr}[\text{reg}] + 1$ and registers the key by computing $(\text{mpk}_{\text{ctr}[\text{reg}]}, \text{aux}') \leftarrow \text{RegPK}(\text{pp}, \text{aux}, pk, x)$. The challenger updates its auxiliary data by setting $\text{aux} := \text{aux}'$ and replies to A with $(\text{ctr}[\text{reg}], \text{mpk}_{\text{ctr}[\text{reg}]}, \text{aux})$.
 - **Register target key query:** In a target-key registration query, the adversary specifies an attribute $x^* \in \{0, 1\}^\ell$. If the adversary has previously made a target-key registration query, then the challenger replies with $\perp$. Otherwise, the challenger increments the counter $\text{ctr}[\text{reg}] := \text{ctr}[\text{reg}] + 1$, samples $(pk^*, sk^*) \leftarrow \text{KeyGen}(\text{pp}, \text{aux})$, and registers $(\text{mpk}_{\text{ctr}[\text{reg}]}, \text{aux}') \leftarrow \text{RegPK}(\text{pp}, \text{aux}, pk^*, x^*)$. It computes the helper decryption key $\text{hsk}^* \leftarrow \text{Update}(\text{pp}, \text{aux}, pk^*)$. The challenger updates its auxiliary data by setting $\text{aux} := \text{aux}'$, stores the index of the target identity $\text{ctr}[\text{reg}]^* := \text{ctr}[\text{reg}]$, and replies to A with $(\text{ctr}[\text{reg}], \text{mpk}_{\text{ctr}[\text{reg}]}, \text{aux}, pk^*, \text{hsk}^*, sk^*)$.
 - **Encryption query:** In an encryption query, A submits an index $i \in [\text{ctr}[\text{reg}]^*, \text{ctr}[\text{reg}]]$ for a public key, a message $\text{msg}_{\text{ctr}[\text{enc}]} \in \mathcal{M}_\lambda$, and a policy $P_{\text{ctr}[\text{enc}]} \in \mathcal{P}_\lambda$. If the adversary has not yet registered a target key, or if the target set of attributes x^* do not satisfy the policy $P_{\text{ctr}[\text{enc}]}$, the challenger replies with $\perp$. Otherwise, the challenger increments the counter $\text{ctr}[\text{enc}] := \text{ctr}[\text{enc}] + 1$ and computes $\text{ct}_{\text{ctr}[\text{enc}]} \leftarrow \text{Enc}(\text{mpk}_i, P_{\text{ctr}[\text{enc}]}, \text{msg}_{\text{ctr}[\text{enc}]})$. The challenger replies to A with $(\text{ctr}[\text{enc}], \text{ct}_{\text{ctr}[\text{enc}]})$.
 - **Decryption query:** In a decryption query, the adversary submits a ciphertext index $j \in [\text{ctr}[\text{enc}]]$. The challenger computes $\text{msg}'_j \leftarrow \text{Dec}(\text{mpk}, sk^*, \text{hsk}^*, \text{ct}_j)$. If $\text{msg}'_j = \text{GetUpdate}$, then the challenger computes an updated helper decryption key $\text{hsk}^* \leftarrow \text{Update}(\text{pp}, \text{aux}, pk^*)$ and recomputes $\text{msg}'_j \leftarrow \text{Dec}(\text{mpk}, \text{hsk}^*, \text{ct}_j)$. If $\text{msg}'_j \neq \text{msg}_j$, the experiment halts and outputs 0.

If the adversary has finished making queries and the experiment has not halted (as a result of a decryption query), then the experiment outputs the bit $b = 1$. We require the following properties for all (potentially unbounded) adversaries A making at most a polynomial number of queries:

- **Correctness:** There exists a negligible function negl such that for all $\lambda \in \mathbb{N}$, $\Pr[b = 1] \geq 1 - \text{negl}(\lambda)$ in the above game. We say that the scheme satisfies perfect correctness if $\Pr[b = 1] = 1$.
- **Compactness:** Let N be the number of registration queries the adversary makes in the above game. There exists a university polynomial $p(\cdot, \cdot, \cdot)$ such that for all $i \in [N]$, $|\text{mpk}_i| \leq p(\lambda, \ell(\lambda), \log i)$. We also require that the size of the helper decryption key hsk^* satisfy $|\text{hsk}^*| \leq p(\lambda, \ell(\lambda), \log N)$ at all times during the experiment.
- **Update efficiency:** Let N be the number of registration queries the adversary makes in the above game. Then, in the course of the above game, the challenger invokes the update algorithm Update at most $O(\log N)$ times, where each invocation runs in time at most $\text{poly}(\log N)$ in the RAM model of computation. Specifically, we model Update as a RAM program that has random access to its input, so the running time of Update may be smaller than the input length.

Definition A.3 (Registered ABE Security). Let $\text{rABE} = (\text{Setup}, \text{KeyGen}, \text{RegPK}, \text{Enc}, \text{Update}, \text{Dec})$ be a registered ABE scheme for attributes of length $\ell = \ell(\lambda)$, policy space $\mathcal{P} = \{\mathcal{P}_\lambda\}_{\lambda \in \mathbb{N}}$, and message space $\mathcal{M} = \{\mathcal{M}_\lambda\}_{\lambda \in \mathbb{N}}$. We say that rABE satisfies *adaptive security* if for all stateful PPT adversaries A , there exists a negligible function negl such that for all $\lambda \in \mathbb{N}$

$$\Pr[\text{Expt}_A^{\text{rABE}}(\lambda) = 1] \leq 1/2 + \text{negl}(\lambda),$$

where $\text{Expt}_A^{\text{rABE}}(\lambda)$ is defined via the following security game between the adversary A and a challenger on common input 1^λ :

- **Setup phase:** The challenger samples the public parameters $\text{pp} \leftarrow \text{Setup}(1^\lambda)$. It then initializes the auxiliary input $\text{aux} := \perp$, a master public key $\text{mpk} := \perp$, a counter $\text{ctr} := 0$ for the number of honest key-registration queries the adversary has made, an empty set of keys $\mathcal{C} = \emptyset$ (to keep track of corrupted public keys), and an empty dictionary mapping public keys to registered attribute sets $\mathcal{D}$. The challenger sends pp to A .

- **Query phase:** The adversary A can now issue the following queries:
 - **Register honest key query:** In an honest key-generation query, the adversary specifies an attribute $x \in \{0, 1\}^\ell$. The challenger increments $\text{ctr} := \text{ctr} + 1$, samples $(\text{pk}, \text{sk}) \leftarrow \text{KeyGen}(\text{pp}, \text{aux})$, and registers $(\text{mpk}', \text{aux}') \leftarrow \text{RegPK}(\text{pp}, \text{aux}, \text{pk}, x)$. The challenger updates its public key $\text{mpk} := \text{mpk}'$, its auxiliary data $\text{aux} := \text{aux}'$, and $D[\text{pk}] := D[\text{pk}] \cup \{x\}$. The challenger replies to A with $(\text{ctr}, \text{mpk}, \text{aux}, \text{pk})$.
 - **Corrupt honest key query:** In a corrupt-honest-key query, the adversary specifies an index $i \in [\text{ctr}]$. Let $(\text{pk}_i, \text{sk}_i)$ be the i^{th} public/secret key query the challenger samples when responding to the i^{th} honest key-registration query. The challenger adds pk_i to C and replies to A with sk_i .
 - **Register corrupted key query:** In a corrupted key-registration query, the adversary A specifies a public key pk and an attribute $x \in \{0, 1\}^\ell$. The challenger registers the key by computing $(\text{mpk}', \text{aux}') \leftarrow \text{RegPK}(\text{pp}, \text{aux}, \text{pk}, x)$. The challenger updates its copy of the public key $\text{mpk} := \text{mpk}'$, its auxiliary data $\text{aux} := \text{aux}'$, adds pk to C , and updates $D[\text{pk}] \leftarrow D[\text{pk}] \cup \{x\}$. It replies to A with (mpk, aux) .
- **Challenge phase:** The adversary A sends two messages $\text{msg}_0, \text{msg}_1 \in \mathcal{M}_\lambda$ and an access policy $P^* \in \mathcal{P}_\lambda$. The challenger samples a bit $b \xleftarrow{\mathcal{R}} \{0, 1\}$, known as the challenge bit, and replies with the challenge ciphertext $\text{ct}^* \leftarrow \text{Enc}(\text{mpk}, P^*, \text{msg}_b)$.
- **Output phase:** Let $\mathcal{X} = \{x \in D[\text{pk}] : \text{pk} \in C\}$ be the set of corrupted attributes. We say that an adversary A is admissible if the challenger policy $P^*(x) = 0$ for all $x \in \mathcal{X}$. The experiment outputs 0 if A is not admissible. Finally, A outputs a bit b' and the experiment outputs 1 if $b' = b$ and 0 otherwise.

Selective security. Similar to the setting of slotted registered ABE (Definitions 6.2 and 6.3), we will also the following selective notions of security:

- **Policy-selective security:** This is the analog of Definition 6.2. In particular, in the policy-selective security game, $\text{Expt}_A^{\text{rABE}}(\lambda)$ is modified so that the adversary A commits to its challenge policy P^* during the setup phase before it receives the public parameters pp .
- **Policy-selective security without corruptions:** This is the analog of Definition 6.3, where in addition to committing to the challenge policy P^* at the beginning of the security game, the adversary is also *not* allowed to make any corrupt-honest-key queries.

From slotted registered ABE to registered ABE. Previously [HLWW23] showed how to construct a full (non-slotted) registered ABE from any slotted registered ABE scheme. The transformation preserves the policy family of the underlying scheme and only incurs overhead that is logarithmic in the number of registered users. We state the theorem below:

Theorem A.4 (Registered ABE from Slotted Registered ABE [HLWW23, §6]). *Suppose there exists a slotted registered ABE scheme srABE for attributes of length $\ell = \ell(\lambda)$, policy space $\mathcal{P} = \{\mathcal{P}_\lambda\}_{\lambda \in \mathbb{N}}$, and message space $\mathcal{M} = \{\mathcal{M}_\lambda\}_{\lambda \in \mathbb{N}}$. Then, there exists a registered ABE scheme rABE for attributes of the same length $\ell = \ell(\lambda)$, policy space $\mathcal{P}$, and message space $\mathcal{M}$. If srABE satisfies adaptive security (resp., policy-selective security or policy-selective security without corruptions), then rABE satisfies adaptive security (resp., policy-selective security or policy-selective security without corruptions).*

Remark A.5 (Stateless vs. Stateful Key Generation). The key-generation algorithm Definition A.1 is allowed to depend on the auxiliary input aux . We can define a stronger notion of registered ABE where the key-generation algorithm is *independent* of the auxiliary data. We refer to such schemes as supporting *stateless* key generation; namely, users in such schemes can generate their keys *independently* of existing users. Earlier works on registration-based encryption (i.e., registered identity-based encryption) all considered stateless key generation [GHMR18, GHM⁺19].

On the contrary, we refer to schemes where the key-generation algorithm can depend on the auxiliary input aux (as in Definition A.1) as supporting *stateful* key generation. In the generic compilation from slotted registered ABE to registered ABE from [HLWW23], users have to know the current number of registered users when generating their keys; as such, the compiler produces a registered ABE scheme with a *stateful* key-generation procedure. However, we

observe that if the underlying slotted registered ABE scheme srABE has a slot-independent key-generation algorithm (i.e., KeyGen in srABE does not depend on the slot index i), then the [HLWW23] compiler (i.e., [Theorem A.4](#)) yields a registered ABE scheme with stateless key generation. Stateless key generation is necessary for using a registered ABE scheme to obtain a flexible broadcast encryption ([Section 7](#)).

B Flexible Broadcast Encryption with Adaptive Security

In this section, we show how to apply the Gentry-Waters transformation [GW09] to upgrade a flexible broadcast encryption scheme with semi-static security into one that satisfies full adaptive security in the random oracle model. The transformation from [GW09] applied to traditional broadcast encryption, but the underlying two-key technique directly applies to the setting of flexible broadcast encryption.

Construction B.1 (Adaptively-Secure Flexible Broadcast). Let $\lambda \in \mathbb{N}$ be a security parameter. Let $m = m(\lambda)$ be any function and $\mathcal{M} = \{\mathcal{M}_\lambda\}_{\lambda \in \mathbb{N}}$ be a message space with messages of length m . Let n be the number of users the scheme supports. Our construction relies on the following primitives:

- Let $\text{FBE}_{\text{SS}} = (\text{FBE}_{\text{SS}}.\text{Setup}, \text{FBE}_{\text{SS}}.\text{KeyGen}, \text{FBE}_{\text{SS}}.\text{Enc}, \text{FBE}_{\text{SS}}.\text{Dec})$ be a flexible broadcast encryption scheme with message space $\mathcal{M}$ that satisfies semi-static security.
- Let $\mathcal{O}: \{0, 1\}^\lambda \rightarrow \{0, 1\}^n$ be a hash function (that we will model as a random oracle in the security analysis).

We construct an (adaptively-secure) flexible broadcast encryption scheme $\text{FBE} = (\text{Setup}, \text{KeyGen}, \text{Enc}, \text{Dec})$ as follows:

- $\text{Setup}(1^\lambda, n)$: On input the security parameter λ and the number of users n , output $\text{pp} \leftarrow \text{FBE}_{\text{SS}}.\text{Setup}(1^\lambda, n)$.
- $\text{KeyGen}(\text{pp})$: On input the public parameters pp , sample two keys $(\text{pk}_0, \text{sk}_0) \leftarrow \text{FBE}_{\text{SS}}.\text{KeyGen}(\text{pp})$ and $(\text{pk}_1, \text{sk}_1) \leftarrow \text{FBE}_{\text{SS}}.\text{KeyGen}(\text{pp})$ for the underlying flexible broadcast encryption scheme. Sample a random bit $a \xleftarrow{\mathbb{R}} \{0, 1\}$ and output $\text{pk} = (\text{pk}_0, \text{pk}_1)$ and $\text{sk} = (a, \text{sk}_a)$.
- $\text{Enc}(\text{pp}, \text{msg}, (\text{pk}_1, \dots, \text{pk}_k))$: On input the public parameters pp , the message $\text{msg} \in \mathcal{M}_\lambda$, and a tuple of public keys $(\text{pk}_1, \dots, \text{pk}_k)$, where each $\text{pk}_i = (\text{pk}_{i,0}, \text{pk}_{i,1})$, the encryption algorithm samples a random $r \xleftarrow{\mathbb{R}} \{0, 1\}^\lambda$ and computes $(t_1, \dots, t_n) = \mathcal{O}(r) \in \{0, 1\}^n$. Next, it constructs two ciphertexts

$$\begin{aligned} \text{ct}_0 &\leftarrow \text{FBE}_{\text{SS}}.\text{Enc}(\text{pp}, \text{msg}, (\text{pk}_{1,t_1}, \dots, \text{pk}_{k,t_k})) \\ \text{ct}_1 &\leftarrow \text{FBE}_{\text{SS}}.\text{Enc}(\text{pp}, \text{msg}, (\text{pk}_{1,1-t_1}, \dots, \text{pk}_{k,1-t_k})), \end{aligned}$$

and outputs the ciphertext $\text{ct} = (\text{ct}_0, \text{ct}_1, r)$.

- $\text{Dec}(\text{pp}, \text{ct}, (j, \text{sk}_j), (\text{pk}_1, \dots, \text{pk}_k))$: On input the public parameters pp , the ciphertext $\text{ct} = (\text{ct}_0, \text{ct}_1, r)$, the index j , the secret key $\text{sk}_j = (a, \text{sk})$, and a tuple of public keys $(\text{pk}_1, \dots, \text{pk}_k)$ where each $\text{pk}_i = (\text{pk}_{i,0}, \text{pk}_{i,1})$, the decryption algorithm starts by computing $(t_1, \dots, t_n) = \mathcal{O}(r) \in \{0, 1\}^n$.
 - If $a = t_j$, the decryption algorithm outputs $\text{msg} = \text{FBE}_{\text{SS}}.\text{Dec}(\text{pp}, \text{ct}_0, (j, \text{sk}), (\text{pk}_{1,t_1}, \dots, \text{pk}_{k,t_k}))$.
 - If $a = 1 - t_j$, the decryption algorithm outputs $\text{msg} = \text{FBE}_{\text{SS}}.\text{Dec}(\text{pp}, \text{ct}_1, (j, \text{sk}), (\text{pk}_{1,1-t_1}, \dots, \text{pk}_{k,1-t_k}))$.

Correctness and security analysis. We now show that FBE of [Construction B.1](#) is a correct flexible broadcast encryption scheme that satisfies adaptive security.

Theorem B.2 (Correctness and Efficiency). *Assuming FBE_{SS} has succinct ciphertexts and is correct, then [Construction B.1](#) has succinct ciphertexts and is correct.*

Proof. We consider each property separately:

- **Succinct ciphertexts:** Consider a ciphertext ct output by the encryption algorithm. By construction, $\text{ct} = (\text{ct}_0, \text{ct}_1, r)$ where ct_0, ct_1 are ciphertexts for FBE_{SS} and $|r| = \lambda$ by construction. Thus, if the underlying flexible broadcast encryption scheme FBE_{SS} has succinct ciphertexts, then so does FBE .

- **Correctness:** Take any $\lambda \in \mathbb{N}$, $n \in \text{poly}(\lambda)$, $k \leq n(\lambda)$, $j \in [k]$, and $\text{msg} \in \mathcal{M}_\lambda$. Then, we have the following:
 - Let $\text{pp} \leftarrow \text{Setup}(1^\lambda, n(\lambda))$, and for each $i \in [k]$, sample $(\text{pk}_i, \text{sk}_i) \leftarrow \text{KeyGen}(\text{pp})$. Note that pp is simply the public parameters for FBE_{SS} . Each public key pk_i consists of two public keys $(\text{pk}_{i,0}, \text{pk}_{i,1})$ for FBE_{SS} , and $\text{sk}_i = (a, \text{sk}_{i,a})$ is the corresponding secret key for one of the two public keys.
 - Let $\text{ct} \leftarrow \text{Enc}(\text{pp}, \text{msg}, (\text{pk}_1, \dots, \text{pk}_k))$. Note that $\text{ct} = (\text{ct}_0, \text{ct}_1, r)$ consists of two ciphertexts for FBE_{SS} . The first ciphertext ct_0 is encrypted with respect to the public keys $(\text{pk}_{1,t_1}, \dots, \text{pk}_{k,t_k})$ while the second ct_1 is encrypted to the public keys $(\text{pk}_{1,1-t_1}, \dots, \text{pk}_{k,1-t_k})$, and $(t_1, \dots, t_k) = \mathcal{O}(r)$.
 - We argue that $\text{Dec}(\text{pp}, \text{ct}, (j, \text{sk}_j), (\text{pk}_1, \dots, \text{pk}_k)) = \text{msg}$. Since $\text{sk}_j = (a, \text{sk}_{j,a})$ and $\text{sk}_{j,a}$ is the secret key associated with public key $\text{pk}_{j,a}$, we can appeal to correctness of FBE_{SS} to conclude that $\text{sk}_{j,a}$ can be used to decrypt ciphertext ct_0 if $a = t_j$ and ct_1 if $a = 1 - t_j$. $\square$

Theorem B.3 (Adaptive Security). *Assuming FBE_{SS} satisfies semi-static security and $\mathcal{O}$ is modeled as a random oracle, then [Construction B.1](#) satisfies adaptive security.*

Proof. Suppose by way of contradiction that adaptive security does not hold. Namely, there exists a PPT adversary A , an efficiently computable function $n \in \text{poly}(\lambda)$, a polynomial q , and an infinite set $\Lambda \subseteq \mathbb{N}$ such that for all $\lambda \in \Lambda$, it holds that

$$\Pr \left[\text{Expt}_{A,n(\lambda)}^{\text{FBE}}(\lambda) = 1 \right] > 1/2 + 1/q(\lambda).$$

Towards reaching a contradiction, we consider a series of hybrid experiments as follows:

- $H_{b,0}(\lambda)$: This experiment is identical to $\text{Expt}_{A,n(\lambda)}^{\text{FBE}}(\lambda)$ where the challenge bit is fixed to b .
- $H_{b,1}(\lambda)$: This experiment is identical to $H_{b,0}(\lambda)$ except the challenger encrypts the lexicographically first message $\text{msg}^* \in \mathcal{M}_\lambda$ instead of msg_b for ct_0^* in the challenge ciphertext $\text{ct}^* = (\text{ct}_0^*, \text{ct}_1^*)$. Specifically, the challenger computes $\text{ct}_0^* \leftarrow \text{FBE}_{\text{SS}}.\text{Enc}(\text{pp}, \text{msg}^*, (\text{pk}_{1,t_1}, \dots, \text{pk}_{k,t_k}))$. The ciphertext ct_1^* is computed as in the real scheme (i.e., as in $H_{b,0}$).
- $H_{b,2}(\lambda)$: In this experiment, the challenger encrypts the lexicographically first message $\text{msg}^* \in \mathcal{M}_\lambda$ instead of msg_b for ct_1^* in the challenge phase. Specifically, when constructing the challenge ciphertext $\text{ct}^* = (\text{ct}_0^*, \text{ct}_1^*)$, the challenger computes $\text{ct}_1^* \leftarrow \text{FBE}_{\text{SS}}.\text{Enc}(\text{pp}, \text{msg}^*, (\text{pk}_{1,1-t_1}, \dots, \text{pk}_{k,1-t_k}))$. The ciphertext ct_0^* is constructed as in $H_{b,1}$.

Since the challenge bit is sampled uniformly (i.e., $b \xleftarrow{\mathcal{R}} \{0, 1\}$), the success probability of A on any security parameter λ is equal to

$$\Pr[A \text{ wins}] = \frac{1}{2} \cdot (\Pr[H_{0,0}(\lambda) = 1] + \Pr[H_{1,0}(\lambda) = 1]).$$

For each $b \in \{0, 1\}$ and $i \in \{1, 2\}$, we define

$$\delta_{b,i}(\lambda) = \Pr[H_{b,i-1}(\lambda) = 1] - \Pr[H_{b,i}(\lambda) = 1].$$

It follows by our assumption that for all $\lambda \in \Lambda$,

$$\begin{aligned} \frac{1}{2} + \frac{1}{q(\lambda)} &\leq \frac{1}{2} \cdot (\Pr[H_{0,0}(\lambda) = 1] + \Pr[H_{1,0}(\lambda) = 1]) \\ &\leq \frac{1}{2} \cdot (\Pr[H_{0,2}(\lambda) = 1] + \delta_{0,1}(\lambda) + \delta_{0,2}(\lambda) \\ &\quad + \Pr[H_{1,2}(\lambda) = 1] + \delta_{1,1}(\lambda) + \delta_{1,2}(\lambda)). \end{aligned}$$

As $H_{b,2}(\lambda)$ is independent of b for each $b \in \{0, 1\}$, it holds that $\Pr[H_{b,2}(\lambda) = 1] \leq 1/2$. Thus, for all $\lambda \in \Lambda$,

$$\frac{1}{q(\lambda)} \leq \frac{1}{2} \cdot (\delta_{0,1}(\lambda) + \delta_{0,2}(\lambda) + \delta_{1,1}(\lambda) + \delta_{1,2}(\lambda)).$$

As Λ is an infinitely large subset of $\mathbb{N}$, it must be the case that for some $b \in \{0, 1\}$ and $i \in \{1, 2\}$, $\delta_{b,i}(\lambda) \geq 1/(2q(\lambda))$ for infinitely many $\lambda \in \mathbb{N}$. In the subsequent claims, we show that this contradicts semi-static security of FBE_{SS} . We prove the claims for $b = 0$ without loss of generality.

Claim B.4. *If $\delta_{0,1}(\lambda) \geq 1/(2q(\lambda))$ for infinitely many $\lambda \in \mathbb{N}$, then FBE_{SS} does not satisfy semi-static security.*

Proof. Given an efficient adversary A that breaks adaptive security of FBE, we construct an efficient adversary B that breaks semi-static security of FBE_{SS} as follows. In the security reduction, we model $\mathcal{O}$ as a random oracle (which algorithm B simulates for A):

- **Random oracle queries:** Whenever A makes an oracle query to $\mathcal{O}$, algorithm B responds with a random string of $n(\lambda)$ bits and stores the response so it can respond consistently to any future queries.
- **Setup phase:** In the setup phase, algorithm B receives the public parameters pp from the FBE_{SS} challenger, which it forwards to A . Algorithm B initializes a counter $\text{ctr} := 0$, a dictionary D , and a set $C = \emptyset$.
- **Pre-challenge query phase:** In the pre-challenge query phase, algorithm B simulates the queries for A as follows:
 - **Key-generation query:** Whenever A makes a key-generation query, algorithm B increments $\text{ctr} := \text{ctr} + 1$ and samples a bit $a \xleftarrow{\mathbb{R}} \{0, 1\}$. Algorithm B then samples $(\text{pk}_a, \text{sk}_a) \leftarrow \text{FBE}_{\text{SS}}.\text{KeyGen}(\text{pp})$. In addition, it makes a key-generation query to its challenger to obtain a public key pk_{1-a} . Algorithm B sends A the key $(\text{pk}_0, \text{pk}_1)$ and adds the mapping $\text{ctr} \mapsto (a, (\text{pk}_0, \text{pk}_1), \text{sk}_a)$ to D .
 - **Corruption query:** Whenever A makes a corruption query for counter value $c \in [\text{ctr}]$, algorithm B retrieves $(a, (\text{pk}_0, \text{pk}_1), \text{sk}_a) := D[c]$ and returns (a, sk_a) . Algorithm B adds c to C .
- **Challenge phase:** In the challenge phase, A sends two messages $\text{msg}'_0, \text{msg}'_1 \in \mathcal{M}_\lambda$ and an ordered list $S^* = (i_1, \dots, i_{k^*}) \subseteq [\text{ctr}]$ to B . Algorithm B proceeds as follows:
 - For each index $i_j \in S^*$, algorithm B retrieves $(a, (\text{pk}_0, \text{pk}_1), \text{sk}_a) := D[i_j]$, and sets $t_j = 1 - a$. For indices $j \in \{k^* + 1, \dots, n\}$, algorithm B samples $t_j \xleftarrow{\mathbb{R}} \{0, 1\}$.
 - Algorithm B samples a random value $r^* \xleftarrow{\mathbb{R}} \{0, 1\}^\lambda$ and programs the random oracle so that on input r , the value of $\mathcal{O}(r^*) = (t_1, \dots, t_{k^*}) \in \{0, 1\}^n$. If adversary A has previously queried $\mathcal{O}$ on r^* , then algorithm B aborts with output $\perp$.
 - Algorithm B sends the same set S^* and messages $\text{msg}_0 = \text{msg}'_0$ and $\text{msg}_1 = \text{msg}'_1$, which is the lexicographically first message in $\mathcal{M}_\lambda$, to the FBE_{SS} challenger. Let ct_0^* be the challenge ciphertext constructed by the challenger. By construction, the challenger samples a bit $\hat{b} \xleftarrow{\mathbb{R}} \{0, 1\}$ and replies with an encryption $\text{msg}_{\hat{b}}$ under the honestly-generated public keys for S^* .
 - Algorithm B computes a ciphertext for the message msg_0 using the public keys for S^* that it generated itself. Algorithm B sends $\text{ct}^* = (\text{ct}_0^*, \text{ct}_1^*, r^*)$ to A .
- **Output phase:** At the end of the game, algorithm A outputs a bit $b' \in \{0, 1\}$, which B also outputs.

Algorithm B is efficient as long as A is efficient, so it suffices to analyze its success probability. First, we define $\text{QE}(\lambda)$ to be the event where A queries $\mathcal{O}(r^*)$ before the challenge phase (in which case algorithm B aborts). As A runs in polynomial-time, there exists some polynomial $Q(\lambda)$ bounding the number of queries that A makes to the random oracle before the challenge phase. Since algorithm B samples r^* uniformly at random from $\{0, 1\}^\lambda$, it follows that

$$\Pr[\text{QE}(\lambda)] \leq Q(\lambda)/2^\lambda.$$

Conditioned on $\text{QE}(\lambda)$ not occurring, we observe that $t_1, \dots, t_{k^*}$ are uniformly distributed given the view of algorithm A in the security game (this is because algorithm B samples $a \xleftarrow{\mathbb{R}} \{0, 1\}$ when responding to each key-generation query). We proceed to analyze the probability that B outputs the correct guess b' for the challenge bit $\hat{b}$, conditioned on $\text{QE}(\lambda)$ not holding. If the challenge bit $\hat{b} = 0$, then the view of A is distributed according to $H_{0,0}(\lambda)$. This means that B outputs the correct bit whenever A wins since $b' = b = 0$. If the challenge bit $\hat{b} = 1$, then the view of A is

distributed according to $H_{0,1}(\lambda)$. So, B outputs the correct bit whenever A loses since $b' \neq b = 0$. It follows that the success probability of B is at least

$$\begin{aligned} \Pr[B \text{ wins}] &\geq \Pr[\neg \text{QE}(\lambda)] \cdot \frac{1}{2} \cdot (\Pr[H_{0,0}(\lambda) = 1 \mid \neg \text{QE}(\lambda)] + \Pr[H_{0,1}(\lambda) = 0 \mid \neg \text{QE}(\lambda)]) \\ &\geq \frac{1}{2} + \frac{1}{2} \cdot (\Pr[H_{0,0}(\lambda) = 1] - \Pr[H_{0,1}(\lambda) = 1]) - \Pr[\text{QE}(\lambda)]. \end{aligned}$$

As $\delta_{0,1}(\lambda) > 1/(2q(\lambda))$ by assumption and $\Pr[\text{QE}(\lambda)] \leq Q(\lambda)/2^\lambda$ is negligible, this violates the semi-static security of FBE_{SS} . $\square$

Claim B.5. *If $\delta_{0,2}(\lambda) \geq 1/(2q(\lambda))$ for infinitely many $\lambda \in \mathbb{N}$, then FBE_{SS} does not satisfy semi-static security.*

Proof. The proof of this claim is essentially identical to the proof of [Claim B.4](#), except the reduction algorithm B programs the random oracle so that $t_j = a$ instead of $1 - a$. It then follows that the honestly-generated keys correspond to ct_1^* instead of ct_0^* . In this case, the challenge ciphertext is used to simulate ct_1^* instead of ct_0^* in the reduction. Algorithm B constructs ct_0^* itself (according to the specification of $H_{0,1}$). The rest of the proof then follows identically to the previous proof. $\square$

The claim now follows via [Claims B.4](#) and [B.5](#). $\square$

C Slotted Registered ABE with Policy-Selective Security

In this section, we show how to transform a slotted registered ABE scheme that satisfies policy-selective security *without* corruptions into one that satisfies policy-selective security *with* corruptions in the random oracle model. As was the case with flexible broadcast encryption ([Appendix B](#)), our transformation is a simple adaptation of the two-key approach from [\[GW09\]](#). We note that the random oracle in our transformation is only used for key aggregation. We could alternatively consider a notion of slotted registered ABE where we allow for a randomized key aggregation procedure. This would yield a registered ABE scheme that satisfies policy-selective security in the plain model, albeit with a randomized (though public-coin) aggregation procedure. We discuss this in more detail in [Remark C.6](#).

Construction C.1 (Slotted Registered ABE with Policy-Selective Security). Let $\lambda \in \mathbb{N}$ be a security parameter and L be the number of slots. Let $\ell = \ell(\lambda)$ be an attribute length and $\mathcal{P} = \{\mathcal{P}_\lambda\}_{\lambda \in \mathbb{N}}$ be a policy space on ℓ -bit inputs (i.e., $\mathcal{P}_\lambda$ is a set of functions $P: \{0, 1\}^{\ell(\lambda)} \rightarrow \{0, 1\}$). Let $m = m(\lambda)$ and $\mathcal{M} = \{\mathcal{M}_\lambda\}_{\lambda \in \mathbb{N}}$ be a message space with message length m . Our construction relies on the following primitives:

- First, let $\text{srABE}_{\text{PSWC}} = (\text{srABE}_{\text{PSWC}}.\text{Setup}, \text{srABE}_{\text{PSWC}}.\text{KeyGen}, \text{srABE}_{\text{PSWC}}.\text{Aggregate}, \text{srABE}_{\text{PSWC}}.\text{Enc}, \text{srABE}_{\text{PSWC}}.\text{Dec})$ be a slotted registered ABE scheme for attributes of length ℓ , policy space $\mathcal{P}$, and message space $\mathcal{M}$. We require that $\text{srABE}_{\text{PSWC}}$ satisfies policy-selective security without corruptions.
- Let $\mathcal{O}: \{0, 1\}^* \rightarrow \{0, 1\}^L$ be a hash function (that we will model as a random oracle in the security analysis). For simplicity of notation, we assume $\mathcal{O}$ can take an arbitrary bit-string (of unbounded length) as input; however, it suffices to have a hash function on bounded-length inputs of the form $(\text{pp}, (\text{pk}_1, x_1), \dots, (\text{pk}_L, x_L))$ where pp is the public parameters for $\text{srABE}_{\text{PSWC}}$, and each pk_i consists of a two public keys for $\text{srABE}_{\text{PSWC}}$ along with a λ -bit string, and each $x_i \in \{0, 1\}^\ell$ is an attribute.

We construct a slotted registered ABE scheme $\text{srABE} = (\text{Setup}, \text{KeyGen}, \text{Aggregate}, \text{Enc}, \text{Dec})$ as follows:

- **Setup**($1^\lambda, L$): On input the security parameter λ and the number of slots L , the setup algorithm outputs $\text{pp} \leftarrow \text{srABE}_{\text{PSWC}}.\text{Setup}(1^\lambda, L)$.
- **KeyGen**(pp, i): On input the public parameters pp and an index $i \in [L]$, the key-generation algorithm samples two public keys $(\text{pk}_0, \text{sk}_0) \leftarrow \text{srABE}_{\text{PSWC}}.\text{KeyGen}(\text{pp}, i)$ and $(\text{pk}_1, \text{sk}_1) \leftarrow \text{srABE}_{\text{PSWC}}.\text{KeyGen}(\text{pp}, i)$. It also samples a random bit $a \xleftarrow{\mathcal{R}} \{0, 1\}$ and $r \xleftarrow{\mathcal{R}} \{0, 1\}^\lambda$ and outputs $\text{pk} = (\text{pk}_0, \text{pk}_1, r)$ and $\text{sk} = (a, \text{sk}_a)$.

- $\text{Aggregate}(\text{pp}, (\text{pk}_1, x_1), \dots, (\text{pk}_L, x_L))$: On input the public parameters pp , a tuple of public keys $\text{pk}_i = (\text{pk}_{i,0}, \text{pk}_{i,1})$ and associated attributes $x_i \in \{0, 1\}^\ell$, the aggregation algorithm first computes $(t_1, \dots, t_L) = \mathcal{O}(\text{pp}, (\text{pk}_1, x_1), \dots, (\text{pk}_L, x_L))$. Then, it prepares two sets of public keys and secret keys as follows:

$$\begin{aligned} (\text{mpk}_0, \text{hsk}_{1,0}, \dots, \text{hsk}_{L,0}) &= \text{srABE}_{\text{PSWC}}.\text{Aggregate}(\text{pp}, (\text{pk}_{1,t_1}, x_1), \dots, (\text{pk}_{L,t_L}, x_L)) \\ (\text{mpk}_1, \text{hsk}_{1,1}, \dots, \text{hsk}_{L,1}) &= \text{srABE}_{\text{PSWC}}.\text{Aggregate}(\text{pp}, (\text{pk}_{1,1-t_1}, x_1), \dots, (\text{pk}_{L,1-t_L}, x_L)). \end{aligned}$$

It outputs the master public key $\text{mpk} = (\text{mpk}_0, \text{mpk}_1)$ and the helper decryption keys $\text{hsk}_i = (t_i, \text{hsk}_{i,0}, \text{hsk}_{i,1})$ for each $i \in [L]$.

- $\text{Enc}(\text{mpk}, P, \text{msg})$: On input the master public key $\text{mpk} = (\text{mpk}_0, \text{mpk}_1)$, the policy $P \in \mathcal{P}_\lambda$, and the message $\text{msg} \in \mathcal{M}_\lambda$, the encryption algorithm constructs two ciphertexts $\text{ct}_0 \leftarrow \text{srABE}_{\text{PSWC}}.\text{Enc}(\text{mpk}_0, P, \text{msg})$ and $\text{ct}_1 \leftarrow \text{srABE}_{\text{PSWC}}.\text{Enc}(\text{mpk}_1, P, \text{msg})$. It outputs $\text{ct} = (\text{ct}_0, \text{ct}_1)$.
- $\text{Dec}(\text{mpk}, \text{hsk}, \text{sk}, \text{ct})$: On input the master public key $\text{mpk} = (\text{mpk}_0, \text{mpk}_1)$, the helper decryption key $\text{hsk} = (t, \text{hsk}_0, \text{hsk}_1)$, the secret key $\text{sk} = (a, \text{sk}_a)$, and the ciphertext $\text{ct} = (\text{ct}_0, \text{ct}_1)$, the decryption algorithm proceeds as follows:
 - If $a = t$, output $\text{msg} = \text{Dec}(\text{mpk}_0, \text{hsk}_0, \text{sk}_a, \text{ct}_0)$.
 - If $a = 1 - t$, output $\text{msg} = \text{Dec}(\text{mpk}_1, \text{hsk}_1, \text{sk}_a, \text{ct}_1)$.

Correctness and security analysis. We now show that srABE of [Construction C.1](#) is a correct slotted registered ABE scheme that satisfies policy-selective security (with corruptions).

Theorem C.2 (Correctness and Efficiency). *Assuming $\text{srABE}_{\text{PSWC}}$ is correct and compact, then [Construction C.1](#) is correct and compact.*

Proof. We consider each property separately:

- **Compactness:** The master public key mpk consists of two master public keys $(\text{mpk}_0, \text{mpk}_1)$ for $\text{srABE}_{\text{PSWC}}$. By compactness of $\text{srABE}_{\text{PSWC}}$, $|\text{mpk}_0|, |\text{mpk}_1| \in \text{poly}(\lambda, \ell, \log L)$ so the same holds for $|\text{mpk}|$. Similarly, each helper decryption key hsk_i consists of two helper secret keys $(\text{hsk}_{i,0}, \text{hsk}_{i,1})$ for $\text{srABE}_{\text{PSWC}}$ along with a single additional bit. Again by compactness of $\text{srABE}_{\text{PSWC}}$, it follows that $|\text{hsk}_i| \in \text{poly}(\lambda, \ell, \log L)$.
- **Correctness:** Take any $\lambda \in \mathbb{N}, L \in \mathbb{N}, i \in [L]$, message $\text{msg} \in \mathcal{M}_\lambda$, attributes $x_1, \dots, x_L \in \{0, 1\}^\ell$, policy $P \in \mathcal{P}_\lambda$ where $P(x_i) = 1$. Then, we have the following:
 - Let $\text{pp} \leftarrow \text{Setup}(1^\lambda, L)$, $(\text{pk}_i, \text{sk}_i) \leftarrow \text{KeyGen}(\text{pp}, i)$, and $\{\text{pk}_j\}_{j \neq i}$ be any collection of public keys (which may be arbitrarily correlated with pk_i). By construction, pp is simply the public parameters for the underlying scheme $\text{srABE}_{\text{PSWC}}$ and the public key $\text{pk}_i = (\text{pk}_{i,0}, \text{pk}_{i,1}, r)$ consists of a pair of public keys $(\text{pk}_{i,0}, \text{pk}_{i,1})$ for $\text{srABE}_{\text{PSWC}}$ along with a random string $r \in \{0, 1\}^\lambda$. The associated secret key $\text{sk}_i = (a, \text{sk}_{i,a})$ is the corresponding secret key for $\text{pk}_{i,a}$.
 - Let $(\text{mpk}, \text{hsk}_1, \dots, \text{hsk}_L) = \text{Aggregate}(\text{pp}, (\text{pk}_1, x_1), \dots, (\text{pk}_L, x_L))$ and $\text{ct} \leftarrow \text{Enc}(\text{mpk}, P, \text{msg})$. By construction, $\text{ct} = (\text{ct}_0, \text{ct}_1)$ consists of two ciphertexts for $\text{srABE}_{\text{PSWC}}$. The first ciphertext ct_0 is with respect to mpk_0 which was obtained by aggregating the public keys $(\text{pk}_{1,t_1}, \dots, \text{pk}_{L,t_L})$ while the second ciphertext ct_1 is with respect to mpk_1 which was obtained by aggregating the public keys $(\text{pk}_{1,1-t_1}, \dots, \text{pk}_{L,1-t_L})$. Here, the bits $t_1, \dots, t_L$ are derived by evaluating $\mathcal{O}(\text{pp}, (\text{pk}_1, x_1), \dots, (\text{pk}_L, x_L))$. In particular, we note that $\text{pk}_{i,a}$ is aggregated in mpk_0 if $a = t_i$ and mpk_1 if $a = 1 - t_i$.
 - We argue that $\text{Dec}(\text{pp}, \text{hsk}_i, \text{sk}_i, \text{ct}) = \text{msg}$. Note that $\text{sk}_i = (a, \text{sk}_{i,a})$. By construction, the helper decryption key hsk_i contains the bit t_i that specifies which of $\text{pk}_{i,0}$ and $\text{pk}_{i,1}$ was aggregated in mpk_0 and which was aggregated in mpk_1 . Since $P(x_i) = 1$, by correctness of $\text{srABE}_{\text{PSWC}}$, either $a = t_i$ in which case $\text{sk}_{i,a}$ correctly decrypts ct_0 , or $a = 1 - t_i$, in which case $\text{sk}_{i,a}$ correctly decrypts ct_1 . $\square$

Theorem C.3 (Policy-Selective Security). *Assuming $\text{srABE}_{\text{PSWC}}$ satisfies policy-selective security without corruptions and $\mathcal{O}$ is modeled as a random oracle, then [Construction C.1](#) satisfies policy-selective security.*

Proof. Suppose there exists a PPT adversary A , an efficiently computable function $L \in \text{poly}(\lambda)$, a polynomial q , and an infinite set $\Lambda \subseteq \mathbb{N}$ such that for all $\lambda \in \Lambda$, it holds that

$$\Pr \left[\text{Expt}_{A, L(\lambda)}^{\text{srABE, PS}}(\lambda) = 1 \right] > 1/2 + 1/q(\lambda).$$

Towards reaching a contradiction, we consider a series of hybrid experiments as follows:

- $H_{b,0}(\lambda)$: This experiment is identical to $\text{Expt}_{A, L(\lambda)}^{\text{srABE, PS}}(\lambda)$ where the challenge bit is fixed to b .
- $H_{b,1}(\lambda)$: This experiment is identical to $H_{b,0}(\lambda)$ except the challenger encrypts the lexicographically first message $\text{msg}^* \in \mathcal{M}_\lambda$ instead of msg_b for ct_0^* when constructing the challenge ciphertext $\text{ct}^* = (\text{ct}_0^*, \text{ct}_1^*)$. Specifically, the challenger computes $\text{ct}_0^* \leftarrow \text{srABE}_{\text{PSWC}}.\text{Enc}(\text{mpk}_0, P^*, \text{msg}^*)$. The ciphertext ct_1^* is computed as in the real scheme (i.e., as in $H_{b,0}$).
- $H_{b,2}(\lambda)$: In this experiment, the challenger encrypts the lexicographically first message $\text{msg}^* \in \mathcal{M}_\lambda$ instead of msg_b for ct_1^* in the challenge phase. Specifically, when constructing the challenge ciphertext $\text{ct}^* = (\text{ct}_0^*, \text{ct}_1^*)$, the challenger computes $\text{ct}_1^* \leftarrow \text{srABE}_{\text{PSWC}}.\text{Enc}(\text{mpk}_1, P^*, \text{msg}^*)$. The ciphertext ct_1^* is constructed as in $H_{b,1}$.

Since the challenge bit $b \xleftarrow{\mathcal{R}} \{0, 1\}$ is sampled uniformly, the success probability of A on any security parameter λ is equal to

$$\Pr[A \text{ wins}] = \frac{1}{2} \cdot (\Pr[H_{0,0}(\lambda) = 1] + \Pr[H_{1,0}(\lambda) = 1]).$$

For each $b \in \{0, 1\}$ and $i \in \{1, 2\}$, we define

$$\delta_{b,i}(\lambda) = \Pr[H_{b,i-1}(\lambda) = 1] - \Pr[H_{b,i}(\lambda) = 1].$$

It follows by our assumption that for all $\lambda \in \Lambda$,

$$\begin{aligned} \frac{1}{2} + \frac{1}{q(\lambda)} &\leq \frac{1}{2} \cdot (\Pr[H_{0,0}(\lambda) = 1] + \Pr[H_{1,0}(\lambda) = 1]) \\ &\leq \frac{1}{2} \cdot (\Pr[H_{0,2}(\lambda) = 1] + \delta_{0,1}(\lambda) + \delta_{0,2}(\lambda) \\ &\quad + \Pr[H_{1,2}(\lambda) = 1] + \delta_{1,1}(\lambda) + \delta_{1,2}(\lambda)). \end{aligned}$$

As $H_{b,2}(\lambda)$ is independent of b for each $b \in \{0, 1\}$, it holds that $\Pr[H_{b,2}(\lambda) = 1] \leq 1/2$. Thus, for all $\lambda \in \Lambda$,

$$\frac{1}{q(\lambda)} \leq \frac{1}{2} \cdot (\delta_{0,1}(\lambda) + \delta_{0,2}(\lambda) + \delta_{1,1}(\lambda) + \delta_{1,2}(\lambda)).$$

As Λ is an infinitely large subset of $\mathbb{N}$, it must be the case that for some $b \in \{0, 1\}$ and $i \in \{1, 2\}$, $\delta_{b,i}(\lambda) \geq 1/(2q(\lambda))$ for infinitely many $\lambda \in \mathbb{N}$. In the subsequent claims, we show that this contradicts the policy-selective security of $\text{srABE}_{\text{PSWC}}$ without corruption queries. We prove the claims for $b = 0$ without loss of generality.

Claim C.4. *If $\delta_{0,1}(\lambda) \geq 1/(2q(\lambda))$ for infinitely many $\lambda \in \mathbb{N}$, then $\text{srABE}_{\text{PSWC}}$ does not satisfy policy-selective security without corruption queries.*

Proof. Let $Q = Q(\lambda)$ be a polynomial that upper bounds the number of pre-challenge random oracle queries the adversary A makes. Additionally, we assume without loss of generality that the following conditions holds:

- Algorithm A always queries $\mathcal{O}$ on the input to the Aggregate algorithm prior to the challenge phase.
- Algorithm A does not query the random oracle on the same input multiple times.

If A does not satisfy the first requirement, we can use A to construct a new adversary that makes one additional random oracle query and which succeeds with the same advantage. Similarly, if A is an algorithm that queries $\mathcal{O}$ on the same input multiple times, we can transform A into an adversary that makes unique queries to $\mathcal{O}$ (and remembers the responses to each oracle query so as to simulate future queries on a repeated input). This change also does not affect the advantage of the adversary. We now use A to construct an adversary B for the policy-selective security game for $\text{srABE}_{\text{PSWC}}$ *without* corruption queries:

- **Setup phase:** Algorithm B starts running algorithm A who starts by committing to a challenge policy P^* . Algorithm B forwards the policy P^* to the challenger and receives the public parameters pp from the challenger, which it forwards to A . Algorithm B then initializes a counter $\text{ctr} := 0$, a dictionary D , and a set $C = \emptyset$. Algorithm B also samples a random index $i^* \xleftarrow{R} [Q]$.
- **Random oracle queries:** When A makes a random oracle query on a tuple $(\text{pp}', (pk_1, x_1), \dots, (pk_L, x_L))$, algorithm B proceeds as follows. If this is not the $(i^*)^{\text{th}}$ pre-challenge random oracle made by A , then algorithm B responds with a uniform random string of length L . If this is the $(i^*)^{\text{th}}$ pre-challenge random oracle made by A , then algorithm B proceeds as follows:
 - If $\text{pp}' \neq \text{pp}$, algorithm B outputs a uniform random bit $b' \xleftarrow{R} \{0, 1\}$.
 - For each $i \in [L]$, algorithm B computes $t_i \in \{0, 1\}$ as follows:
 - * If $P^*(x_i) = 0$, algorithm B samples $t_i \xleftarrow{R} \{0, 1\}$.
 - * If $P^*(x_i) = 1$, B checks if there is a tuple (i, a_i, pk_i, sk_i) in D associated with some counter value c_i . If no such tuple exists, B samples $t_i \xleftarrow{R} \{0, 1\}$. Otherwise, B sets $t_i = 1 - a_i$.
 - Algorithm B replies with $(t_1, \dots, t_L) \in \{0, 1\}^L$.
- **Pre-challenge query phase:** In the pre-challenge query phase, algorithm B responds to queries as follows:
 - **Key-generation query:** Whenever A makes a key generation query for slot $i \in [L]$, algorithm B increments $\text{ctr} := \text{ctr} + 1$ and samples a bit $a \xleftarrow{R} \{0, 1\}$ and string $r \xleftarrow{R} \{0, 1\}^\lambda$. Algorithm B samples a key $(pk_a, sk_a) \leftarrow \text{srABE}_{\text{PSWC}}.\text{KeyGen}(\text{pp}, i)$. It also makes a key-generation query to its own challenger to obtain a public key pk_{1-a} . Finally, algorithm B samples $r \xleftarrow{R} \{0, 1\}^\lambda$ and replies to A with the public key $pk = (pk_0, pk_1, r)$. It also adds the mapping $\text{ctr} \mapsto (i, a, (pk_0, pk_1, r), sk_a)$ to D .
 - **Corruption query:** Whenever A makes a corruption query on an index $c \in [\text{ctr}]$, algorithm B retrieves $(i, a, (pk_0, pk_1, r), sk_a) := D[c]$, and returns (a, sk_a) .
- **Challenge phase:** In the challenge phase, algorithm A chooses two messages $\text{msg}'_0, \text{msg}'_1 \in \mathcal{M}_\lambda$ and for each slot $i \in [L]$, it specifies a tuple (c'_i, x_i, pk_i^*) . Algorithm B now constructs the public keys $(pk_1, \dots, pk_L)$ according to the specification of the adaptive security game:
 - If $c'_i \in \{1, \dots, \text{ctr}\}$, algorithm B sets $(i', a_i, (pk_{i,0}, pk_{i,1}, r_i), sk_i) := D[c'_i]$. If $i \neq i'$, then algorithm B aborts and outputs a uniform random bit $b' \xleftarrow{R} \{0, 1\}$. Otherwise, algorithm B sets $pk_i = (pk_{i,0}, pk_{i,1}, r_i)$. If the adversary previously issued a corruption query on counter c'_i , then algorithm B also adds the slot index i to the corrupted set C .
 - If $c'_i = \perp$, then algorithm B sets $pk_i := pk_i^*$ and adds the slot index i to the corrupted set C .

Algorithm B now checks that for all $i \in C$, it holds that $P^*(x_i) = 0$. If not, algorithm B aborts and outputs a uniform random bit $b' \xleftarrow{R} \{0, 1\}$. Otherwise, if $P^*(x_i) = 0$ for all $i \in C$, algorithm B constructs the challenge ciphertext as follows:

- First, it checks whether algorithm A has made at least i^* queries to the random oracle. If not, then algorithm B halts with output $\perp$.
- If algorithm A has made at least i^* queries to the random oracle and its $(i^*)^{\text{th}}$ query was *not* on input $(\text{pp}, (pk_1, x_1), \dots, (pk_L, x_L))$, then algorithm B aborts and outputs a uniform random bit $b' \xleftarrow{R} \{0, 1\}$. Otherwise let $(t_1, \dots, t_L) := \mathcal{O}(\text{pp}, (pk_1, x_1), \dots, (pk_L, x_L))$ be the value algorithm B used to respond to the $(i^*)^{\text{th}}$ random oracle query.

- For each $i \in [L]$, algorithm B sets $c_i := c'_i$.
- Algorithm B outputs $\text{msg}_0 = \text{msg}'_0$ and $\text{msg}_1 = \text{msg}^*$ as its challenge message and $(c_i, x_i, \text{pk}_{i,t_i})$ for each $i \in [L]$ as its public keys. The challenger replies with a challenge ciphertext ct_0^* . Specifically, the challenger samples a bit $\hat{b} \xleftarrow{\mathcal{R}} \{0, 1\}$ and sends back an encryption of $\text{msg}_{\hat{b}}$.
- Finally, algorithm B also computes $\text{mpk}_1 = \text{srABE}_{\text{PSWC}}.\text{Aggregate}(\text{pp}, (\text{pk}_{1,1-t_1}, x_1), \dots, (\text{pk}_{L,1-t_L}, x_L))$ and $\text{ct}_1^* \leftarrow \text{srABE}_{\text{PSWC}}.\text{Enc}(\text{mpk}_1, P^*, \text{msg}_0)$.
- Algorithm B sends $\text{ct}^* = (\text{ct}_0^*, \text{ct}_1^*)$ to A .

- **Output phase:** Algorithm A outputs a bit $b' \in \{0, 1\}$, which B also outputs.

If A is efficient, then algorithm B is also efficient by construction. It suffices to analyze the success probability of algorithm B . First, we note that if algorithm B aborts before the output phase, then it always outputs a uniform random bit $b' \in \{0, 1\}$. In this case, the advantage of algorithm B is exactly $1/2$. Suppose algorithm B does not abort early. Then it must be the case that the following properties hold:

- Algorithm A makes at least i^* queries to the random oracle prior to the challenge phase, and moreover, the $(i^*)^{\text{th}}$ query algorithm A makes to the random oracle is on the tuple $(\text{pp}, (\text{pk}_1, x_1), \dots, (\text{pk}_L, x_L))$ that algorithm B constructs in the challenge phase.
- Every tuple $(c'_i, x_i, \text{pk}_i^*)$ that algorithm A provides in the challenge phase has the property that if $c'_i \in \{1, \dots, \text{ctr}\}$, then $i = i'$ where $(i', a_i, (\text{pk}_{i,0}, \text{pk}_{i,1}, r_i)) := \text{D}[c'_i]$. Similarly, for all $i \in C$, it holds that $P^*(x_i) = 0$.

By construction, this means that algorithm B is admissible. Next, let $\text{QE}(\lambda)$ be the event where the challenge query contains an honestly-generated public key pk_i , but algorithm A makes a challenge query before pk_i is actually generated and inserted into D . Since each public key pk_i contains a random string $r \leftarrow \{0, 1\}^\lambda$ and A makes at most $Q(\lambda) = \text{poly}(\lambda)$ queries to $\mathcal{O}$, it follows that

$$\Pr[\text{QE}(\lambda)] \leq Q(\lambda)/2^\lambda = \text{negl}(\lambda).$$

We now show that conditioned on event QE *not* happening, algorithm B correctly simulates either hybrid $H_{0,0}$ or hybrid $H_{0,1}$. For this to be the case, ct_0^* should be sampled according to $\text{srABE}_{\text{PSWC}}.\text{Enc}(\text{mpk}_0, P^*, \text{msg}_{\hat{b}})$ where $\text{mpk}_0 = \text{srABE}_{\text{PSWC}}.\text{Aggregate}(\text{pp}, (\text{pk}_{1,t_1}, x_1), \dots, (\text{pk}_{L,t_L}, x_L))$ and each t_i is uniformly random given A 's view. For each slot $i \in [L]$, we show that the challenger uses the key pk_{i,t_i} as input to $\text{srABE}_{\text{PSWC}}.\text{Aggregate}$ and the corresponding bit t_i output by $\mathcal{O}$ is uniformly random conditioned on A 's view. We consider two cases:

- Suppose $c'_i = \perp$. In this case, the challenger uses the public key pk_{i,t_i} provided by B as the i^{th} public key when computing $\text{srABE}_{\text{PSWC}}.\text{Aggregate}$. This means algorithm A chose the public key itself. In this case, if A is admissible, then it must be the case that $P^*(x_i) = 0$. By construction of algorithm B , it sampled $t_i \xleftarrow{\mathcal{R}} \{0, 1\}$ in this case.
- Suppose $c'_i \neq \perp$. Since $c'_i \neq \perp$, then $\text{D}[c'_i]$ must contain a tuple $(i, a_i, \text{pk}_i, \text{sk}_i)$ where $\text{pk}_i = (\text{pk}_{i,0}, \text{pk}_{i,1}, r_i)$. In this case, algorithm B sets $\text{pk}_{i,t_i} = \text{pk}_{i,1-a_i}$. By construction, $\text{pk}_{i,1-a_i}$ is the key chosen by the challenger in response to the $(c'_i)^{\text{th}}$ key-generation query. Correspondingly, the challenger in this case uses pk_{i,t_i} to construct the aggregated key. We again consider two cases:
 - If $P^*(x_i) = 0$, then t_i is uniform by construction.
 - If $P^*(x_i) = 1$, then conditioned on $\text{QE}(\lambda)$ not occurring, it must be the case that $t_i = 1 - a_i$. Since $P^*(x_i) = 1$ and assuming A is admissible, then algorithm A does not make a corruption query on index $c_i = c'_i$. In this case, the value of a_i is information-theoretically hidden from the view of A . Since algorithm B samples $a_i \xleftarrow{\mathcal{R}} \{0, 1\}$ and sets $t_i = 1 - a_i$, the bit t_i is correctly distributed in this case.

Since t_i is appropriately distributed for each slot $i \in [L]$, it also follows that ct_0^* is distributed according to $H_{0,0}$ when $\hat{b} = 0$ and according to $H_{0,1}$ when $\hat{b} = 1$. Moreover, algorithm B constructs ct_1^* exactly according to the specification

of $H_{0,0}$ and $H_{0,1}$. Therefore, it follows that if $\hat{b} = 0$, algorithm B wins whenever $H_{0,0}(\lambda) = 1$ (since $b = 0$), and if $\hat{b} = 1$, algorithm B wins whenever $H_{0,1}(\lambda) = 0$. To complete the proof, we now compute the overall success probability of B . Let $E(\lambda)$ be the event that A queries the challenge query on query i^* . By assumption, algorithm A is required to query the random oracle on its challenge query at some point prior to the challenge phase, so if A makes exactly $Q = Q(\lambda)$ queries, then

$$\Pr[E(\lambda)] = 1/|Q|.$$

Let $\overline{QE}(\lambda)$ denote the complement of $QE(\lambda)$. For notational convenience, in the following, we omit the explicit dependence on the security parameter when it is clear from context. From the above analysis, we have that

$$\begin{aligned} \Pr[B \text{ wins} \mid E, \overline{QE}] &= \frac{1}{2} \cdot (\Pr[H_{0,0}(\lambda) = 1] + \Pr[H_{0,1}(\lambda) = 0]) \\ &= \frac{1}{2} + \frac{1}{2} \cdot (\Pr[H_{0,0}(\lambda) = 1] - \Pr[H_{0,1}(\lambda) = 1]) \\ &= \frac{1}{2} + \frac{1}{2} \delta_{0,1}(\lambda). \end{aligned}$$

Moreover, conditioned on $\overline{E}$, we have by construction that $\Pr[B \text{ wins} \mid \overline{E}] = 1/2$. Putting everything together, we can now write

$$\begin{aligned} \Pr[B \text{ wins}] &\geq \Pr[B \text{ wins} \mid E, \overline{QE}] \cdot \Pr[E \mid \overline{QE}] \cdot \Pr[\overline{QE}] + \Pr[B \text{ wins} \mid \overline{E}] \cdot \Pr[\overline{E}] \\ &\geq \left(\frac{1}{2} + \frac{1}{2} \delta_{0,1}(\lambda) \right) \cdot \frac{1}{Q} \cdot \left(1 - \frac{Q}{2^\lambda} \right) + \frac{1}{2} \cdot \left(1 - \frac{1}{Q} \right) \\ &\geq \frac{1}{2} + \frac{1}{2Q} \delta_{0,1}(\lambda) - \text{negl}(\lambda), \end{aligned}$$

whenever $Q = Q(\lambda)$ is polynomially-bounded. Thus, algorithm B breaks policy-selective security of $\text{srABE}_{\text{PSWC}}$ without corruption queries. $\square$

Claim C.5. *If $\delta_{0,2}(\lambda) \geq 1/(2q(\lambda))$ for infinitely many $\lambda \in \mathbb{N}$, then $\text{srABE}_{\text{PSWC}}$ does not satisfy policy-selective security without corruptions.*

Proof. The proof of this claim is essentially identical to the proof of Claim C.4 with the following minor modifications:

- The reduction algorithm B now programs $t_i := a_i$, instead of $t_i = 1 - a_i$ when programming the random oracle for the challenge query.
- When answering key-generation queries, after sampling the bit $a \xleftarrow{\mathcal{R}} \{0, 1\}$, algorithm B now samples pk_{1-a} itself (using the honest key-generation algorithm) and makes a key-generation query to its own challenger to obtain pk_a .
- When generating the challenge ciphertext, algorithm B constructs ct_0^* itself (according to the specification of $H_{0,1}$ and $H_{0,2}$). It uses the challenger to construct ct_1^* using the public keys $\text{pk}_{i,1-t_i}$ for each slot $i \in [L]$ instead of pk_{i,t_i} .

The rest of the proof follows identically to the proof of Claim C.4. $\square$

Combining Claims C.4 and C.5 completes the proof of the theorem. $\square$

Remark C.6 (Registered ABE without Random Oracles). We note that the random oracle in Construction C.1 is only invoked in the Aggregate algorithm. We could alternatively consider a notion of (slotted) registered ABE where we allow key aggregation to be randomized. This yields a scheme satisfying policy-selective security in the *plain* model. However, the security proof in this case would only hold for a *semi-honest* key curator that samples a uniform random string for key aggregation. Security does not necessarily hold if the key curator behaves maliciously, and as such, we find this notion significantly less compelling from a definitional perspective. A key motivation for considering notions like registration-based encryption and registered ABE is that we do *not* need to trust any central authority. As such, it is more natural to model the key curator as a public, deterministic entity whose behavior can be easily audited.